AUTHOR REVIEW COPY NOT FOR JOURNAL SUBMISSION

RankCloak V2 to final V3 tracked comparison

RankCloak Conceals the Surface Form of Synthetic Cryptographic Artifacts in Language Model Generated Text

Comparison scope

Main manuscript:

`paperV2/scientific_reports/main2.tex`
versus
`paperV3/scientific_reports/main3.tex`

Supplementary Information:

`paperV2/scientific_reports/supplementary2.tex`
versus
`paperV3/scientific_reports/supplementary3.tex`

Legend

- Added final V3 text
- ~~Deleted V2 text~~
- Unchanged text is black.
- Added figures are outlined in blue; deleted figures, when present, are reduced and crossed in red.

This combined PDF is an internal author-review artifact.
Do not upload it as part of the journal submission package.

~~RankCloak: Hiding Synthetic Cryptographic Artifacts in Plain English with Language Models~~

RankCloak Conceals the Surface Form of Synthetic Cryptographic Artifacts in Language Model Generated Text

Alexander V. Mantzaris[1,*]

[1]School of Data, Mathematical, and Statistical Sciences, University of Central Florida, Orlando, FL, United States of America
[*]Correspondence: alexander.mantzaris@ucf.edu

ABSTRACT

Deleted V2 abstract

~~RankCloak transcodes synthetic cryptographic artifacts into natural language using language-model next-token ranks (leveraging an LLM) and realizes those ranks as generated cover text. We compare direct subword representation with bounded ASCII-byte and hex-nibble codecs, including non-segmented generation and segmented forced spans followed by non-payload natural tails. The completed evaluation used 480 public deterministic artifacts from eight cryptographic classes, three open-source model families, and 18 English prompt templates. The secondary recovery tests used Spanish and Simplified Chinese prompts. All 6,480 primary payload trials recovered the serialized payload under saved-token exact-copy replay (Wilson 95% confidence interval $[0.999408, 1.000000]$), and all 576 multilingual trials recovered under the same condition. This mapping contract is restrictive. Detokenizing and retokenizing visible text recovered 61.1%; normalization and formatting changes reduced recovery further. Using markdown transfer, paraphrase, and cross-model decoding recovered none. Automated readability diagnostics were protocol-dependent and are not participant ratings. Neural detectors strongly distinguished RankCloak covers from the controls (matched ROC-AUC 0.990190 for TextCNN and 0.999834 for DeBERTa-v3-base), providing adverse evidence for concealment. RankCloak therefore provides a reproducible framework for measuring rank pressure, capacity, replay requirements, cover cost, and detectability. It does not establish a secure or deployment-ready covert channel.~~

Added final V3 abstract

Cryptographic artifacts such as hashes, nonces, identifiers, and ciphertexts are conspicuous when transmitted in their usual textual forms. RankCloak conceals this recognizable form by encoding each synthetic artifact through language-model token choices, producing cover text that does not directly display the artifact. We evaluated 480 public deterministic artifacts from eight classes with three open-source model families and 18 English prompt templates. The decoder recovered every artifact across 6,480 primary trials when the original token sequence and shared generation configuration were preserved. Recovery from rendered text was 61.1 percent; markdown transfer, paraphrasing, and decoding with another model failed in the tested robustness conditions. Stronger detection tests separated duplicate and near-duplicate material before training, withheld model families, included human-written comparisons, and used generating-model probabilities; encoded covers were often detectable. Restricting embedding to positions with several plausible next words reduced some detection signals but also reduced capacity and increased cover length. Detecting a possible encoding does not itself display or reconstruct the concealed artifact. RankCloak demonstrates artifact-specific concealment and recovery under a shared configuration while exposing detectability and transmission limitations; it does not establish hiding arbitrary natural-language messages.

Introduction

Generative language models provide a new medium for text steganography because a prompt can define a next-token distribution and a topic direction in which generated text appears. In the Calgacus rank-transcoding construction[1], a sender tokenizes an arbitrary hidden text, records each token's context-dependent rank, and generates a cover text by selecting the correspondingly ranked tokens under a shared cover prompt; the receiver reverses these steps under the same contexts. Calgacus thereby transcodes one hidden-text token into one cover-text token. RankCloak retains that rank-transfer mechanism but also maps exact serialized cryptographic artifacts through direct-subword and bounded codecs, for which payload and cover token counts need not equal the Calgacus baseline. This mechanism connects classical work on covert channels, information-theoretic steganography, information hiding, and provably secure steganography[2–5] with neural and LLM-based linguistic steganography, where hidden information is embedded through choices made during language generation[6–14]. Related work on LLM watermarking also treats token selection as a carrier of hidden signals, although watermarking ~~is usually aimed at attribution or detection rather than arbitrary payload recovery[15].~~ generally asks whether text carries a detectable mark rather than whether a chosen payload can be recovered[15]. Entropy-guided watermarking selects suitable high-entropy positions for test-time marking[16]; that idea motivates the selective-embedding experiment here, but RankCloak instead evaluates artifact-specific payload recovery, embedding rate, completion, cover length, and steganalysis.

A long-running goal in this area is to make arbitrary, encrypted, or otherwise high-entropy data appear as ordinary communication. Early mimic-function work used statistical source models and inverse compression ideas to generate outputs with distributional properties similar to a target source[17,18]. Hiding the Hidden and related linguistic-steganography systems transformed ciphertext into innocuous natural-language text that could later be transformed back into the original ciphertext[19,20]. Other text methods carried bits through synonym choice or contextual word substitution[21], while network-camouflage systems generalized a similar objective to protocol formats by making encrypted traffic resemble allowed or innocuous traffic[22,23]. RankCloak follows this same broad goal in an LLM setting. Instead of using a fixed grammar, synonym table, or protocol template, it uses the token choices made by a shared language model. The payload is first converted into ranks, and the cover prompt determines which English tokens occupy those ranks at each step. This also makes the method useful as a measurement tool since it exposes how a hidden artifact changes rank pressure, generated text quality, and the traces available for later detection.

This measurement focus also has defensive value for detecting the presence of such hidden encodings. If hashes, nonces, UUID-like values, or ciphertext strings are carried by generated tokens rather than displayed in their original syntax, surface scanners cannot rely only on visible hexadecimal, base64, UUID, or credential-shaped patterns. Detection must therefore look for indirect evidence, such as low-probability forced tokens, tokenizer artifacts, formatting fragments, unusually regular token choices, segmentation patterns, or differences between payload-bearing spans and non-payload continuations. Covert-channel and stegomalware surveys emphasize that hiding the existence of communication creates detection, capacity-limitation, and countermeasure problems across text and network media[24,25]. Text steganalysis and generated-text detection work similarly shows that statistical, neural, graph-based, and probability-based features can reveal some generated or steganographic text, while also showing that robust detection is setting-dependent and vulnerable to distribution shift or paraphrase[26–31]. The same issue is relevant to AI-safety monitoring, where hidden message passing and secret collusion among agents have been studied as ways that generated text might evade oversight[32,33]. RankCloak ~~therefore treats hiding and detection as two sides of the same measurement problem. A useful methodology should recover payloads under declared exact-copy conditions while also producing transparent features, controls, and failure modes for steganalysis~~evaluates these as distinct endpoints. Artifact concealment asks whether the cryptographic artifact and its recognizable syntax are directly visible in the cover text. Payload recovery asks whether an authorized receiver using the shared recovery configuration can reconstruct that artifact, whereas steganalysis asks whether an observer can classify the cover as likely encoded. A positive steganalysis decision does not itself display or recover the concealed artifact, although high detectability weakens covert-channel stealth.

RankCloak proposes a methodology for a specific case, hiding synthetic cryptographic artifacts in generated English text. The payload classes resemble values commonly encountered in security, including hashes, authentication tags, nonces, identifiers, ciphertexts, and signatures[34–36]. These strings are challenging payloads for rank transcoding because they are not natural-language messages and do not display high probability sequences. Direct subword tokenization can represent them compactly, but the resulting payload tokens may occupy low-probability positions in the model's next-token distribution[37,38]. A cover generator forced to select such tokens can preserve exact recoverability under matched replay while producing public text with visible artifacts or unusual statistical features. RankCloak gives an approach by which a hash-like string can be carried by generated cover tokens rather than shown as hexadecimal characters. The main methodological development of this paper is that cryptographic-artifact payloads can be treated as representations to be measured, not merely as strings to be passed through an LLM tokenizer. RankCloak therefore compares direct subword rank measurement with bounded-rank payload codecs, including ASCII-byte fixed-radix encodings and a hex-nibble encoding for lowercase hex-like payloads. This design makes the capacity to quality trade-off explicit. Direct subword ranks can be short but may impose large rank pressure, whereas bounded encodings constrain the allowed rank range while increasing the number of generated cover tokens. This representation view also separates payload recovery from public-message quality. RankCloak keeps the cover-side model, tokenizer, prompt family, deterministic rank-ordering rule, and replay procedure fixed while varying payload-side encodings and protocol variants. Non-segmented variants emit one cover token per payload rank under a single prompt context. Segmented variants split a payload into short rank chunks, emit payload-bearing forced spans across multiple public messages, and append non-payload natural tails after those forced spans. Because tails do not carry payload ranks, RankCloak reports forced-span and full-message quality proxies separately.

The resulting evaluation introduces RankCloak as a reproducible methodology for measuring how synthetic cryptographic artifacts can be represented as bounded rank sequences and expressed as LLM-generated cover text~~under exact-copy replay~~. The study ~~is organized around how artifact-like payloads affect~~ examines rank pressure, ~~how bounded encodings change~~ bounded-codec capacity and cover length, ~~and how segmentationand natural tails change the relationship between payload-bearing spans and full messages. Exact-copy replay remains the supported recovery condition, so the sender and receiver must share the same model artifact, tokenizer, prompt rendering, filtering, deterministic configuration, codec, boundaries, and generated token identifiers. This condition is consistent with recent work showing that tokenization and retokenization mismatches can disrupt LLM-based steganography and watermarking[39]. Visible-text detokenization and retokenization is a separate, weaker recovery condition. The expanded evaluation spans public cryptographic test vectors, prompt and model families~~ segmentation, and ~~secondary-language conditions, and examines robustness, detectability, automated readability diagnostics, capacity, and computational overhead. RankCloak 's contribution is therefore a controlled methodology for hiding and measuring synthetic cryptographic artifacts while making representation, recovery , cover-quality, and detection trade-offs explicit. It does not establish practical robustness, security, undetectability, or human-perceived naturalness~~non-payload natural tails while preserving the artifact-hiding and recovery question. Saved-token exact-copy recovery is one controlled condition for exact inversion with tokenization and generation history fixed; rendered visible-text transfer and transformed-text transfer test stricter communication assumptions. The focused revision adds strict pre-feature deduplication, human-authored secondary controls, detector testing on withheld model families and with generating-model probabilities, analysis at low false-positive rates, entropy-gated embedding, and matched quantization tests. RankCloak contributes an artifact-specific hiding and recovery method with transparent empirical boundaries; it does not provide a cryptographic secrecy proof or claim universal undetectability.

Methods

RankCloak measures whether synthetic artifacts that do not normally look like language can be represented as LLM token ranks and then expressed as generated cover text while remaining exactly recoverable under a declared replay contract. The motivating examples are hashes, authentication tags, nonces, hexadecimal tokens, UUID identifiers, authenticated-encryption outputs, and signatures. In their original visible forms, many of these strings are recognizable by simple pattern matching (even by humans), but RankCloak asks whether the same serialized artifact can instead be carried by the token choices of an LLM-generated message. Conceptually, the method proceeds in four steps. First, the payload is converted into a sequence of integer ranks. Second, a cover prompt defines the next-token distribution used for public-text generation. Third, the sender emits the token at each required rank under that cover context. Fourth, the receiver replays the same cover context and recovers the rank of each exact saved token, following the rank-transcoding idea described by Norelli and Bronstein[1]. If the recovered rank sequence matches the encoded payload ranks and the codec reconstructs the declared serialized bytes, the payload has been decoded exactly. The algorithms below implement two RankCloak procedures, a non-segmented procedure that maps one payload into one cover sequence and decodes the complete generated sequence, and a segmented procedure that maps a hex-like payload into several short payload-bearing spans, appends non-payload natural tails, and decodes only the forced spans. These procedures support both construction-side and detection-side measurement because they provide exact-copy recovery tests, rank-pressure measurements, cover-quality proxies, labeled cover examples, and rank-derived features that can be audited by steganalysis methods.

Sender and receiver are assumed to share a common overall LLM configuration, denoted as K_{common}. ~~This configuration includes~~ It specifies the exact model ~~artifact~~revision, tokenizer, quantization and backend~~settings~~, prompt rendering, deterministic ~~rank-ordering~~ rank rule, payload codec, ~~segmentation rule where used, tail and~~ token filter, span convention, and any segmentation, tail, or lead-in ~~policies where used, token filterwhere used, forced-span boundaries where used, and saved payload-bearing token IDs. Protocol labels select predeclared variants of this shared configuration. They are not modeled as cryptographic keys, authentication tags, signatures, or operational commands.~~

~~Recovery is evaluated under an exact-copy replay requirement for deterministic context reproduction. This means that the receiver consumes the saved generated token IDs and replays the same model, tokenizer, prompt configuration, rank-ordering rule, codec, filter, and segmentation metadata. Visible-text detokenization and retokenization is a separate, weaker replay condition. Edits, paraphrases, whitespace normalization, moderation rewrites, truncation of~~ policy. Selecting such a configuration is analogous to selecting a common codec or protocol implementation and need not create a new per-message negotiation; a prompt phrase may be shared contextual information, but it is not treated as a formal cryptographic key. In the saved-token experimental condition, retained payload-bearing ~~tokens, retokenization, model changes, quantization changes, prompt changes, or filter changes can alter ranks and are outside the supported exact-copy condition~~token IDs hold tokenization and generation history fixed while the decoder still reconstructs every contextual rank and inverts the selected payload representation; visible-text and transformed-text conditions separately test transport without preserving that original token sequence.

The exact-copy assumptions used throughout the experiments are:

- Shared configuration: sender and receiver use the same ~~model artifact~~pinned protocol definition, including model revision, tokenizer, quantization and backend, prompt rendering, rank rule, codec, filters, span convention, segmentation policy, and tail or lead-in policy.
- Deterministic rank ordering: ranks are one-indexed and computed from descending logits with token-ID tie-breaking.
- Payload codecs: ASCII-byte fixed-radix coding applies to exact serialized bytes; hex-nibble coding applies only to canonical lowercase hexadecimal payloads.
- Non-segmented decoding: the receiver replays the complete saved cover-token sequence.
- Segmented decoding: the receiver decodes only the saved payload-bearing forced spans; natural tails are public text but carry no payload ranks.
- Endpoint validation: rank recovery, representation recovery, serialized byte length, and SHA-256 identity are recorded separately, with exact serialized-payload equality as the primary endpoint.

The reader-facing protocol labels distinguish non-segmented variants, where recovery replays the complete generated cover-token sequence, from segmented variants, where recovery decodes only the payload-bearing forced span and ignores non-payload tails.

Basic rank definition

Rank-transcoding uses the rank of a token in an LLM next-token ordering as the transferable object. A rank is defined by sorting candidate tokens under a model context. Exact recovery is possible only when the sender and receiver replay the same model, tokenizer, prompt, rank rule, and generated tokens. Let M denote an autoregressive language model with vocabulary $\mathscr{V}$. For a token context h, let $z_M(v \mid h)$ be the next-token logit for token $v \in \mathscr{V}$, and let $\mathrm{id}(v)$ be its deterministic token id. The model probability is:

$$p_M(v \mid h) = \frac{\exp z_M(v \mid h)}{\sum_{u \in \mathscr{V}} \exp z_M(u \mid h)}. \tag{1}$$

Ranks are one-indexed; token ID breaks ties deterministically:

$$u \prec_h v \Longleftrightarrow z_M(u \mid h) > z_M(v \mid h) \text{ or } [z_M(u \mid h) = z_M(v \mid h) \text{ and } \mathrm{id}(u) < \mathrm{id}(v)]. \tag{2}$$

The rank of a token and the inverse rank selector are,

$$\rho_h(v) = 1 + |\{u \in \mathscr{V} : u \prec_h v\}|, \qquad \mathrm{token}_h(r) = v \text{ such that } \rho_h(v) = r. \tag{3}$$

Here $\mathrm{token}_h(r)$ means the token at rank r in the deterministic ordering for context h. Because softmax is order-preserving, the released implementation computes this order directly from 64-bit logit values and breaks exact ties by increasing token ID.

Calgacus rank-transcoding baseline

The starting point is the Calgacus rank-transcoding construction, in which ranks rather than token identities are transferred between contexts[1]. In the Calgacus procedure, a hidden text can be tokenized as $e_1, e_2, \ldots$. For each hidden-text token e_i, the sender computes its rank under a payload-side context g_i, where g_i is determined by the preceding hidden-text tokens and any payload-side conditioning used by the protocol. The sender then emits a cover token by selecting the token with the same rank under a cover-side context h_i, where h_i is determined by the cover prompt or key and the previously emitted cover tokens.

For hidden-text token e_i, payload-side context g_i, cover-side context h_i, emitted cover token y_i, and transferred rank r_i, the baseline rank-transcoding step is:

$$r_i = \rho_{g_i}(e_i), \qquad y_i = \mathrm{token}_{h_i}(r_i), \qquad \hat{r}_i = \rho_{h_i}(y_i). \tag{4}$$

Under exact replay with the same model, tokenizer, prompt configuration, and deterministic rank rule, the recovered rank satisfies $\hat{r}_i = r_i$. The receiver can then replay the payload-side context and reconstruct the hidden-text token as $\hat{e}_i = \mathrm{token}_{g_i}(\hat{r}_i)$. This is a same-token-length arbitrary-text baseline; one hidden-text token determines one cover token. The construction shows that a rank sequence can be carried by generated text without directly emitting the hidden tokens. Direct application to cryptographic-artifact strings, however, can impose high rank pressure. Such strings are often poorly predicted by a language model, so their payload-side tokens can occupy low-probability, high-rank positions, and selecting those same ranks under a cover prompt can degrade model-likelihood and surface diagnostics even when recovery remains exact.

RankCloak keeps the cover-side rank-transfer idea from Calgacus but changes the payload-side representation. Instead of relying only on direct LLM tokenization of the displayed artifact, RankCloak maps serialized artifact strings into explicit bounded rank sequences. In the RankCloak notation used below, the displayed artifact is x, the payload codec is C, and the encoded rank sequence is $R = C(x)$. Recovery therefore proceeds by reconstructing $\hat{R}$ from the cover tokens and decoding $\hat{x} = C^{-1}(\hat{R})$, rather than by replaying a natural-language hidden-text stream token by token. The direct-subword condition retains the Calgacus payload-side ranks as a separate measurement arm. The contribution relative to the baseline is the measurement framework around artifact-specific representations: direct subword rank diagnostics, bounded ASCII-byte and hex-nibble codecs, deterministic token filtering, segmented forced spans, non-payload tails, and separate forced-span versus full-message metrics.

RankCloak bounded payload-codec methods

RankCloak changes the payload side by applying explicit codecs to displayed synthetic payload strings. The goal is to avoid treating a hash-like or ciphertext serialization as an ordinary natural-language token sequence. Instead, the displayed artifact is converted into a controlled sequence of ranks before cover generation begins. This makes rank pressure measurable by design. A codec can trade cover length against rank magnitude so that smaller rank alphabets constrain forced token choices but require more generated tokens, while more compact representations can reduce length but may impose larger rank pressure. For payload string x, codec C, encoded rank sequence R, and recovered rank sequence $\hat{R}$,

$$R = C(x) = (r_1, \ldots, r_n), \qquad \hat{x} = C^{-1}(\hat{R}). \tag{5}$$

The direct-subword arm is not folded into Eq. (5); it tokenizes the literal serialization under the Calgacus payload context and retains the resulting ranks without imposing a fixed rank alphabet. The released implementation disables special-token insertion and accepts only a reversible tokenization whose detokenization differs from the literal UTF-8 payload by an explicitly recorded leading-space prefix. Any other affix or payload-byte change makes that representation unavailable before generation; complete framing metadata appears in Supplementary Note S1.

Non-segmented cover generation and replay recovery then use the same rank selector as the Calgacus baseline, but the ranks come from the payload codec rather than only from direct payload-side LLM token ranks:

$$y_i = \mathrm{token}_{h_i}(r_i), \qquad \hat{r}_i = \rho_{h_i}(y_i), \qquad \hat{R} = (\hat{r}_1, \ldots, \hat{r}_n). \tag{6}$$

Codec-level exact recovery requires both rank recovery and an invertible codec for the displayed payload:

$$\hat{x} = x \quad \text{when} \quad \hat{R} = R \quad \text{and} \quad C^{-1}(C(x)) = x. \tag{7}$$

Hex-nibble coding applies to canonical lowercase hexadecimal payloads. It maps each displayed hexadecimal character to one bounded rank:

$$\texttt{0} \mapsto 1, \quad \texttt{1} \mapsto 2, \quad \ldots, \quad \texttt{9} \mapsto 10, \quad \texttt{a} \mapsto 11, \quad \ldots, \quad \texttt{f} \mapsto 16. \tag{8}$$

Thus each lowercase hex character maps to one rank in $1, \ldots, 16$. A 64-character SHA-256 hexadecimal serialization becomes 64 ranks under this codec. Base64 and hyphenated UUID display forms are rejected rather than silently normalized. The bounded alphabet controls maximum requested rank while increasing the number of forced positions; its relationship to observed cover diagnostics is evaluated rather than assumed. ASCII-byte fixed-radix coding applies to arbitrary displayed payload strings. It first expresses the displayed UTF-8 byte string as base-B digits d_i, then maps each digit to rank $d_i + 1$,

$$d_i \in \{0, \ldots, B-1\}, \qquad r_i = d_i + 1. \tag{9}$$

The released codec concatenates the complete serialized byte bitstream, appends zero bits only at its end to reach a multiple of $\log_2 B$, and records the original byte length and terminal-padding count. Decoding subtracts one from each rank, validates the digit range, removes only that recorded padding, and reconstructs exactly the declared number of bytes. Consequently, the $B = 8$ condition uses $\lceil 8m/3 \rceil$ ranks for m serialized bytes rather than independently padding every byte; the $B = 16$ condition uses two ranks per byte. Supplementary Note S1 gives the complete metadata and edge cases.

When a deterministic token filter is configured, ranks are computed within the allowed token set A_h:

$$\rho_h^F(v) = 1 + |\{u \in A_h : u \prec_h v\}|, \qquad A_h \subseteq \mathscr{V}. \tag{10}$$

Here A_h is the allowed token set after filtering. If no filter is configured, $A_h = \mathscr{V}$. The inverse selector $\text{token}_h^F(r)$ denotes the token at rank r in the same ordered allowed set. Sender and receiver must apply the identical mask and ordering because removing one candidate shifts every lower choice, a trial is unavailable if the allowed set cannot supply a requested rank.

Algorithm 1 gives the non-segmented bounded-codec RankCloak procedure. The algorithm first encodes the displayed artifact into ranks. It then generates one cover token per rank under the cover prompt. Recovery replays the same prompt and exact saved token sequence, recomputes each token's rank, and decodes the recovered ranks back to the displayed payload. This algorithm is the basic measurement unit for the non-segmented cover-generation trials.

Algorithm 1 Non-segmented RankCloak bounded-codec trial for encoding a displayed synthetic artifact as ranks, generating one cover token per rank, and recovering by exact-copy replay.

Require: Synthetic payload x; model M; cover prompt p; payload codec C; optional deterministic token filter F; shared configuration K_{common}.
Ensure: Cover text, recovered payload $\hat{x}$, exact-recovery flag, and recorded cover metrics.

$R \leftarrow C.\text{encode}(x)$
context $\leftarrow$ tokenize(p)
cover_tokens $\leftarrow$ [ ]
for each rank $r_i \in R$ **do**
    logits $\leftarrow M.$next_logits(context)
    if F is configured **then**
        allowed_tokens $\leftarrow F$(logits)
    else
        allowed_tokens $\leftarrow$ vocabulary(M)
    end if
    $y_i \leftarrow$ token with one-indexed rank r_i among allowed_tokens
    append y_i to cover_tokens
    append y_i to context
end for
cover_text $\leftarrow$ detokenize(cover_tokens)

context $\leftarrow$ tokenize(p)
recovered_ranks $\leftarrow$ [ ]
for each exact saved token y_i in cover_tokens **do**
    logits $\leftarrow M.$next_logits(context)
    if F is configured **then**
        allowed_tokens $\leftarrow F$(logits)
    else
        allowed_tokens $\leftarrow$ vocabulary(M)
    end if
    append rank_of$(y_i$, logits, allowed_tokens) to recovered_ranks
    append y_i to context
end for
$\hat{x} \leftarrow C.$decode(recovered_ranks)
exact_recovery $\leftarrow (\hat{x} = x)$
return cover_text, $\hat{x}$, exact_recovery

Before Algorithm 1 is executed, the selected representation validates payload syntax, exact byte length, codec applicability, and the source digest. The retained record saves the requested ranks, cover-token IDs, prompt and configuration identities, codec metadata, and failure position where applicable. After decoding, exact rank and representation equality are distinguished from exact serialized-payload equality, which additionally requires the declared byte length, bytes, syntax, and SHA-256 digest to match. The same generation and replay loop also applies when the validated direct-subword representation supplies R; its payload-side reconstruction and framing checks are detailed in Supplementary Note S1.

RankCloak segmented multi-cover method

The segmented multi-cover method extends RankCloak from one cover sequence to several shorter public messages. Long sequences of forced ranks can accumulate topic drift, formatting artifacts, or low-probability continuations. Segmentation restarts the cover context for each chunk, allowing the effect of distributing the payload across several messages to be measured without claiming that those messages are human-natural. The method is also useful analytically because it separates the payload-bearing forced span from the rest of the public message.

For a full rank sequence R, segmentation splits the payload into m chunks:

$$R = R^{(1)} \| R^{(2)} \| \cdots \| R^{(m)}. \tag{11}$$

Here $\|$ denotes sequence concatenation, and each $R^{(j)}$ is an ordered, non-overlapping rank chunk. The completed design includes frozen segmented schedules and an ablation over 4-, 8-, 16-, and 32-rank chunks rather than treating one chunk size as universally preferred.

For segment j, the forced span emits one token per payload rank. Recovery uses only those forced tokens:

$$y_k^{(j)} = \text{token}_{h_k^{(j)}}\left(r_k^{(j)}\right), \qquad \hat{r}_k^{(j)} = \rho_{h_k^{(j)}}\left(y_k^{(j)}\right). \tag{12}$$

When a filter is configured, both operations use the filtered ordering defined after Eq. (10). After the forced span $y^{(j)}$, the sender appends a greedy non-payload tail $z^{(j)}$. The public message is the tokenizer rendering of the concatenated token sequence $y^{(j)} \| z^{(j)}$. The receiver ignores $z^{(j)}$ and decodes only $y^{(j)}$:

$$m_j = \text{text}\left(y^{(j)} \| z^{(j)}\right), \qquad \hat{R} = \hat{R}^{(1)} \| \cdots \| \hat{R}^{(m)}. \tag{13}$$

Here $\text{text}(\cdot)$ means ordinary tokenizer rendering of the token sequence; it is not an additional cryptographic operation. The canonical expression in Eq. (13) has no lead-in. For the evaluated lead-in extension, a non-payload prefix is generated before $y^{(j)}$, and the record stores exact lead-in, forced, tail, and full token-ID arrays, a token-role mask, and half-open forced-span offsets. The decoder uses those declared offsets and does not infer a payload boundary from visible prose.

A core component of this method is the forced-span and tail separation. The payload is carried only by the forced span, while the tail is a greedy continuation whose effect on the complete public message is measured separately. This distinction prevents a misleading quality result: full-message diagnostics can change because of the non-payload tail even if the payload-bearing forced span remains low probability or exhibits surface artifacts. For that reason, the Results report forced-span metrics and full-message metrics separately.

Algorithm 2 describes the segmented multi-cover protocol. It encodes the hex-like payload, splits the rank sequence into short chunks, generates one forced span per chunk, appends the selected tail-policy output after each forced span, and recovers the payload by replaying only the forced spans. Each segment is its own replay component: generation starts from its declared prompt, and recovery reads the exact saved token IDs within its declared forced-span boundary.

Algorithm 2 Segmented multi-cover RankCloak trial for distributing a hex-nibble payload across payload-bearing forced spans with non-payload natural tails.

Require: Synthetic hex-like payload x; model M; hex-nibble codec C_{hex}; segment size s; prompt schedule $P = (p_1, \ldots, p_m)$; deterministic token filter F; tail policy T; shared configuration K_{common}.
Ensure: Public messages $(\text{message}_1, \ldots, \text{message}_m)$, recovered payload $\hat{x}$, exact-recovery flag, forced-span metrics, and full-message metrics.

$R \leftarrow C_{\text{hex}}.\text{encode}(x)$
split R into chunks $(R_1, \ldots, R_m)$, each with at most s ranks
for each segment $j = 1, \ldots, m$ **do**
    context $\leftarrow$ tokenize$(P[j])$
    forced_tokens$_j \leftarrow [\,]$
    for each rank $r \in R_j$ **do**
        logits $\leftarrow M.$next_logits(context)
        allowed_tokens $\leftarrow F$(logits)
        $y \leftarrow$ token with one-indexed rank r among allowed_tokens
        append y to forced_tokens$_j$
        append y to context
    end for
    tail_tokens$_j \leftarrow$ TAILPOLICY$(M, \text{context}, F, T)$
    message$_j \leftarrow$ detokenize(concat(forced_tokens$_j$, tail_tokens$_j$))
end for
emit public messages $(\text{message}_1, \ldots, \text{message}_m)$

recovered_chunks $\leftarrow [\,]$
for each segment $j = 1, \ldots, m$ **do**
    context $\leftarrow$ tokenize$(P[j])$
    read the exact saved tokens within the declared forced-span boundary for R_j
    ignore all tail tokens
    segment_ranks $\leftarrow [\,]$
    for each forced token y in the payload-bearing span **do**
        logits $\leftarrow M.$next_logits(context)
        allowed_tokens $\leftarrow F$(logits)
        append rank_of(y, logits, allowed_tokens) to segment_ranks
        append y to context
    end for
    append segment_ranks to recovered_chunks
end for
$\hat{x} \leftarrow C_{\text{hex}}.\text{decode}(\text{concat}(\text{recovered\_chunks}))$
exact_recovery $\leftarrow (\hat{x} = x)$
return $(\text{message}_1, \ldots, \text{message}_m), \hat{x}$, exact_recovery

The non-lead-in form in Algorithm 2 supplies the canonical segment layout. The lead-in extension inserts a rank-1 prefix before the forced span; its exact token IDs must be replayed before forced-rank recovery, and its saved length shifts the half-open forced boundary. Lead-in lengths 2, 4, 8, 16, and 32 were compared with the no-lead-in condition. Single-topic segmentation reuses its assigned prompt, whereas multi-topic segmentation rotates deterministically through the declared prompt categories. Segment order, prompts, boundaries, and token roles are retained metadata rather than information inferred by the decoder.

Algorithm 3 implements the fixed natural-tail policy available to Algorithm 2. The tail is generated greedily, begins only after the payload-bearing forced span, and is not decoded. Its purpose is to measure how a non-payload continuation changes the complete public message while preserving a separate measurement of the forced span. This tail extension is a sentence-completion step rather than part of the hidden payload. A forced span can end abruptly because its tokens were selected to carry payload ranks, not necessarily to finish a phrase or sentence. The minimum length and sentence-boundary rule allow a continuation before the hard cap prevents non-payload text from growing without bound; no claim of human-perceived completeness follows from this proposed heuristic.

Algorithm 3 Greedy natural-tail policy used after each segmented forced span; the generated tail carries no payload ranks and is ignored by the decoder.

Require: Model M; current context after the forced span; deterministic token filter F.
Ensure: Non-payload tail tokens.

```
tail_tokens ← []
while |tail_tokens| < 60 do
    choose the rank-1 token z under M.next_logits(context) among tokens allowed by F
    append z to tail_tokens
    append z to context
    if |tail_tokens| ≥ 20 and detokenize(tail_tokens) ends at a sentence boundary then
        break
    end if
end while
return tail_tokens
```

Algorithm 3 realizes TAILPOLICY as a fixed sentence completion of 20 to 60 tokens. The completed primary matrix uses a separately frozen dynamic condition as its reference level: after at least eight greedy tokens, stopping requires terminal punctuation or a double newline, balanced delimiters, and no declared dangling connective or punctuation, with a 256-token emergency cap recorded as censoring. A no-tail condition returns an empty tail and all three policies are non-payload conditions ignored during decoding (their complete stopping definitions appear in Supplementary Note S1).

Evaluation corpus, controls, and metrics

The experiments use deterministic synthetic payloads that resemble cryptographic or operational artifacts while avoiding real secret material. The corpus contains 480 public deterministic test vectors, 60 in each of eight algorithm-defined classes; SHA-256 hexadecimal digests, HMAC-SHA256 hexadecimal tags, deterministic 96-bit nonces, 128-bit hexadecimal tokens, UUIDv4 identifiers, AES-256-GCM base64 outputs, ChaCha20-Poly1305 base64 outputs, and Ed25519 base64 signatures[34–36,40–43]. Authenticated-encryption and signature rows are genuine outputs of the declared algorithms under reproducible public test inputs. No real credentials, API keys, private keys, account identifiers, operational secrets, commands, private user data, or deployed traffic are used.

Each payload is evaluated first as a representation problem and then, where applicable, as a cover-generation problem. The representation diagnostics measure how the exact serialization behaves under direct subword tokenization and how many ranks are required by the bounded codecs. Cover-generation conditions include direct subword ranks, ASCII-byte fixed-radix coding with $B = 8$, ASCII-byte fixed-radix coding with $B = 16$, raw hex-nibble coding for eligible lowercase hexadecimal payloads, and single- and multi-topic segmented hex-nibble protocols. All generation, recovery, and diagnostic rows use the deterministic rank-ordering rule in Eqs. (2) and (3) as part of the shared replay configuration.

Generation used Meta-Llama-3-8B-Instruct, Qwen2.5-7B-Instruct, and Mistral-7B-Instruct-v0.3 in pinned Q4_K_M configurations, an approximately 4-bit quantized format, with their embedded GGUF tokenizers. Eighteen English templates span six prompt categories. Secondary Spanish and Simplified Chinese conditions evaluate saved-token recovery but do not establish multilingual human naturalness. The primary matrix contains 14,400 generated units; 6,480 RankCloak payload trials and 7,920 prompt, model, representation, and length-matched ordinary controls. The complete ledger reconciles 38,504 declared work units across the primary, robustness, ablation, multilingual, evaluator, and detector stages. Complete corpus construction, model and tokenizer identities and hashes, prompt schedules, and work-unit accounting appear in Supplementary Note S2.

The protocol variants define how encoded ranks become public text. Non-segmented trials use Algorithm 1 and replay the complete saved cover-token sequence. Segmented trials use Algorithm 2; Algorithm 3 or another declared tail condition follows each forced span, while recovery uses only the saved forced tokens. Prompt-matched controls provide comparison text for automated diagnostics and detector training; they are not human-written naturalness baselines. The deterministic safe-text filter rejects declared control, replacement, code, markup, URL, escape, and formatting fragments. The ablation compares it with no filter and an isolated roundtrip-stable filter, lead-in lengths 0, 2, 4, 8, 16, and 32, chunk sizes 4, 8, 16, and 32, dynamic, fixed, and no-tail policies, and frozen single- versus multi-topic schedules.

The Results report exact serialized-payload recovery, representation and rank diagnostics, cover length, requested-rank pressure, token log probability, deterministic surface artifacts, and automated readability diagnostics. A balanced 504-stimulus subset covers seven generation conditions and all 18 English templates; its surface measures include Flesch Reading Ease[44], frozen artifact flags, and TF-IDF prompt similarity, with intervals bootstrapped over prompt templates. Source-model and held-out-evaluator likelihoods are reported as model diagnostics, not participant ratings. No human study was performed.

For segmented rows, every cover-quality measurement is assigned to one of two scopes. Forced-span metrics are computed only over payload-bearing tokens decoded by the receiver. Full-message metrics are computed over the complete rendered message, including non-payload tails and any experimental lead-in. This scope distinction is necessary because tails can change full-message proxies while carrying no payload ranks. Declared unavailable combinations are excluded from the applicable estimand and reported separately rather than classified as observed failures. Complete metric definitions, transformation coverage, ablation schedules, and exclusion rules appear in Supplementary Notes S3 through S6.

For paired segmented trials, tail gain is the full-message mean token log probability minus the forced-span mean token log probability, $G_{\text{tail}} = \bar{\ell}_{\text{full}} - \bar{\ell}_{\text{forced}}$. Positive values mean that averaging in the non-payload continuation raises the full-message model score; they do not imply a change in the payload-bearing span. The empirical frontier reports forced-span and full-message views separately on the four-class common hexadecimal support, with payload-bootstrap uncertainty and automated surface-flag burden detailed in Supplementary Note S5.

Recovery, robustness, and capacity endpoints

The ~~strongest recovery endpoint passes the original saved token IDs~~ saved-token endpoint supplies the retained payload-bearing token sequence to the decoder under K_{common}~~. Saved IDs are an exact-copy record, not an ordinary text channel~~, thereby testing exact contextual rank reconstruction and codec inversion while tokenization fidelity and generation history are held fixed; retaining IDs does not make those inverse computations unnecessary or expose the payload serialization. Visible-text replay ~~first~~ instead detokenizes and retokenizes the rendered string, ~~so an apparently unchanged string can yield different IDs or boundaries[39]. Arbitrary editing, paraphrasing, formatting changes, truncation of payload-bearing tokens, cross-model decoding, and prompt or filter search are not supported recovery modes. Controlled secondary tests separately evaluate unmodified visible text,~~ and controlled transformed-text tests additionally apply final-tail truncation, Unicode NFKC, quote conversion, whitespace and line-ending changes, deterministic character and token edits, markdown copy/paste, paraphrase, limited canonicalization, ~~and~~ or cross-model decoding~~. Transformations are frozen as separate conditionsrather than composed into an uncontrolled channel~~[39]. These are distinct transmission conditions: token-preserving delivery can retain the exact token stream, ordinary rendered-text delivery does not, and paraphrased delivery changes the visible string. Final-tail truncation removes only declared non-payload tokens and is not evidence of arbitrary truncation tolerance~~. First-divergence~~; first-divergence labels localize the first recorded mismatch but do not establish its cause.

For a bounded representation containing H source bits and a rank alphabet of size B, the minimum fixed-radix forced length and nominal forced-position rate are

$$n_B = \left\lceil \frac{H}{\log_2 B} \right\rceil, \qquad R_B = \frac{H}{n_B} \le \log_2 B. \tag{14}$$

Here H is the information actually presented to that codec, eight bits per exact serialized byte for the byte codecs and four bits per canonical hexadecimal character for the nibble codec. For the byte bitstream, $n_B = \lceil 8m/\log_2 B \rceil$, consistent with Eq. (9). Direct subword ranks do not have a fixed B and are excluded from this bounded-capacity identity.

For representation comparisons on a common artifact support, we additionally report effective artifact bits per forced token, $R_{\text{art}} = H_{\text{art}}/n_{\text{forced}}$. Here H_{art} is held fixed across representations, four bits per canonical hexadecimal character in the common-hex analysis, rather than set to the codec's serialized input width. This descriptive rate permits direct and bounded representations to be compared on the same artifacts; it is not a cryptographic-security capacity.

If segment j has ℓ_j lead-in tokens and t_j tail tokens, and $\delta \ge 0$ denotes any other counted non-payload extension under the retained accounting convention, full cover length and effective rate are,

$$N_{\text{full}} = n_{\text{forced}} + \sum_{j=1}^{J} (\ell_j + t_j) + \delta, \qquad R_{\text{eff}} = \frac{H}{N_{\text{full}}} \le \frac{H}{n_{\text{forced}}} = R_B \quad (n_{\text{forced}} = n_B). \tag{15}$$

Segmentation changes context resets and auxiliary text, not the number of forced codec positions. Positive cover-length residuals are overhead rather than payload capacity.

For eligible-token probabilities $p_{(1)}(h) \ge \cdots \ge p_{(B)}(h)$, any bounded selection $r \le B$ obeys the same-context conditional quality relation

$$-\log p_{(1)}(h) \le -\log p_{(r)}(h) \le -\log p_{(B)}(h), \qquad Q_B = \frac{1}{n_B} \sum_{i=1}^{n_B} -\log p_M(y_i \mid h_i). \tag{16}$$

Some tables report mean log probability $\bar{\ell} = -Q_B$, for which larger values are less negative. These bounds concern model likelihood under identical filtered histories; they are not statements about grammar, meaning, human naturalness, security, or detectability.

The confirmatory endpoint is exact equality between the decoded serialized payload bytes and the declared source, including length, syntax, and SHA-256 digest. Exact rank and representation equality are retained as diagnostics. Trial-level Wilson 95% intervals are reported[45]; a strict payload group passes only if every constituent prompt, model, and eligible protocol trial succeeds, and a group defined by model and payload applies the same rule within one model. Segments are never counted as independent payload trials. Grouped bootstrap procedures use payload or prompt template as the resampling unit according to the estimand. Declared unavailable combinations never enter a failure denominator.

Neural steganalysis, statistical analysis, and computational overhead

~~The frozen detector corpus contains~~ Strict corpus preparation preceded feature extraction and tokenization. Unicode NFKC normalization, case folding, Unicode-whitespace collapse, and hashing identified one normalized exact-duplicate group; its complete matched pair was removed from the 15,~~840 rows, balanced between 7, 920 RankCloak covers and 7~~840-row source corpus, leaving 15,838 observations. Declared near duplicates had cosine similarity at least 0.95 under L2-normalized character-boundary TF-IDF with 3–5 character n-grams and minimum document frequency two. Payload groups connected by an exact or near-duplicate edge formed immutable components assigned to 9,904 training, 2,~~920 matched ordinary controls and grouped by the 480 payloads. A TextCNN-equivalent classifier[26,46] uses lowercase tokens, a 30~~952 validation, and 2~~,000-term vocabulary,length 256, 300-dimensional embeddings,100 filters at widths 3 and 5, dropout 0.5, and Adam learning rate 0.001. A DeBERTa-v3-base classifier uses length 256,learning rate 2×10^{-5}, weight decay 0.01, and three epochs[47]. Payload groups, never individual rows, define train/test splits~~982 test observations. The audit found zero payload, pair, normalized-text, component, or declared near-duplicate links across partitions; it cannot identify semantic paraphrases below the normalization and threshold rule.

~~Each architecture is fit on one matched split, 18 held-out-template splits, three leave-one-model splits, and six leave-one-codec splits, for 56 completed fits and 55, 440 predictions. The split definitions and seeds~~ TextCNN and a pinned DeBERTa-v3-base revision were trained specifically on RankCloak versus matched ordinary-model controls using the predeclared architectures. An exact-model-aware logistic detector, representing an attacker with likelihood information from the same model that generated the text, used summary surprisal features computed from saved token log probabilities for both labels, with regularization selected by validation ROC-AUC; ordinary-control ranks were unavailable and therefore excluded. Leave-one-family-out fits excluded the target model family from both training and validation. A secondary test selected 250 human-written responses from the Databricks Dolly corpus across 16 available templates without using human labels for fitting or threshold selection; it is a corpus comparison, not a participant study or naturalness rating.

ROC-AUC used tie-aware empirical ranks. Thresholds for low false-positive-rate operating points (low-FPR) maximized validation sensitivity subject to the exact allowed false-positive count, were conservatively tie-broken, and were frozen before ~~fitting. Primary endpoints are ROC-AUC and balanced accuracy; precision,Brier score, and sensitivity at~~ test evaluation; empirical 1~~% false-positive rate are exploratory. Payload-group bootstrap intervals use~~ % or 0.1% results required at least 100 or 1,000 validation and test negatives, respectively, and unavailable estimates were not interpolated. Confidence intervals used 2,000 resamples [48]~~. Audits test exact visible-text~~ of complete deduplication clusters. The entropy extension fixed ungated, median-gate, and 75th-percentile strict-gate thresholds from ordinary top-p development traces before evaluation and consumed payload ranks only at eligible positions. Its fixed-payload capacity is conditional on completion. The paired quantization experiment held one pinned Qwen2.5-7B-Instruct revision, tokenizer, prompt, payload, codec, filter, target length, sampler, and backend fixed while comparing Q4_K_M and Q8_0, approximately 4-bit and ~~token-sequence identity, then diagnose normalized character n-gram near-duplicates at the declared threshold. Excluding affected test payload groups without retraining is a sensitivity check and does not eliminate the near-duplicate limitation. Fixed-seed repeats establish same-device CUDA repeatability only, CPU/GPU equivalence was not tested~~8-bit quantized versions of the same pinned Qwen model; paired recovery, grouped bootstrap, exact McNemar, and bidirectional cross-quantization detector transfer were prespecified. Supplementary Notes S11–S15 give the complete calibration, partition, threshold, count, and provenance details.

Statistical inference respects payload and prompt-template grouping, with segments nested within covers. Paired ablations use payload as the pairing and bootstrap unit. Mixed models include payload and prompt grouping where supported, negative-binomial models for overdispersed artifact counts, and Gaussian models for continuous outcomes[49,50]. Holm adjustment controls selected contrast families; Benjamini-Hochberg values accompany exploratory families. Convergence, dispersion, zero-inflation, singularity, and residual diagnostics are retained. Three of five fitted mixed models were singular, no undeclared fixed-effects fallback was substituted, and compact ablation, topic, auxiliary-detector, and near-duplicate analyses remain labeled exploratory or post-outcome where applicable. Complete model formulas, bootstrap seeds, split definitions, diagnostics, and multiplicity details are in Supplementary Notes S5 through S7.

Generation overhead uses inclusive wrapper wall time retained by the harness, including the reported setup, generation, and supported replay scopes. Payload throughput is representation bits divided by generation time, and effective rate follows Eq. (15). Detector benchmarks report fixed-device CUDA wall/training time and peak allocated VRAM. CPU time, repeated warm-up measurements, and cross-device hardware generalization were unavailable and were not inferred. Complete timing cells and component scopes appear in Supplementary Note S9.

Results

Expanded evaluation and exact-copy recovery

The completed primary design produced 14,400 work units across three model families and 18 English prompt templates. Of these 6,480 were payload bearing RankCloak trials spanning the six protocol/codec configurations and 7,920 were ordinary controls. All 6,480 decoded serialized payloads exactly under saved-token replay (1.000000; Wilson 95% CI $[0.999408, 1.000000]$; Table 1). The stricter aggregation also passed for all 480 payload groups (CI $[0.992061, 1.000000]$) and all 1,440 groups defined by model and payload (CI $[0.997339, 1.000000]$). These grouped endpoints prevent repeated prompts and protocols from inflating the apparent number of independently generated artifacts.

The recovery question was whether codec inversion remained exact after forcing the ranks into cover contexts, rather than whether emitted ranks could merely be read back from the same loop. Success was defined at the original serialization and digest, and every model and eligible protocol stratum reached that endpoint. The all-success result therefore supports deterministic implementation fidelity across the evaluated prompt/model matrix. Its uncertainty is one-sided in substance; the data bound the rate of error under this completed design but cannot show that an unobserved replay environment or transmission channel would also be error-free.

Secondary exact-copy tests recovered 288/288 Spanish trials and 288/288 Simplified Chinese trials, or 576/576 in total. They establish recovery for those retained prompts under the same saved-token contract; they do not establish multilingual naturalness or robustness to ordinary text transfer. Across primary, ablation, robustness, multilingual, evaluator, and detector work, the final ledger reconciled 38,504/38,504 units: 38,072 succeeded and 432 were declared unavailable. No unit remained and none was classified as an execution failure. Recovery strata and secondary-language endpoints appear in Supplementary Tables S5 and S6.

Design or endpoint	Scope	Result
Payload corpus	Eight algorithm-defined classes, 60 public deterministic artifacts per class	480 artifacts
Model and prompt coverage	Three open-source 7B to 8B families; 18 English templates in six categories	14,400 primary work units (6,480 payload; 7,920 control)
Primary exact-copy recovery	Serialized payload, saved-token replay	6,480/6,480; 1.000000 (95% CI $[0.999408, 1.000000]$)
Strict grouped sensitivity	Payload groups; groups defined by model and payload	480/480 (CI $[0.992061, 1.000000]$); 1,440/1,440 (CI $[0.997339, 1.000000]$)
Spanish secondary recovery	Saved-token replay	288/288 (CI $[0.986837, 1.000000]$)
Simplified Chinese secondary recovery	Saved-token replay	288/288 (CI $[0.986837, 1.000000]$)
Complete work ledger	All declared revision matrices	38,504/38,504 reconciled: 38,072 success; 432 unavailable; 0 execution failure

Table 1. Evaluation design and recovery endpoints. Confidence intervals are trial- or group-level Wilson 95% intervals as identified. Recovery requires saved-token exact-copy replay with the declared shared configuration. Unavailable work units are configuration/retained-output absences, not observed decoding failures.

Payload representation, rank pressure, and capacity tradeoff

Figure 1 compares payload representations across 480 unique artifacts in eight classes and three models. The 1,440 direct observations across payloads and models contained 60,861 direct-rank positions. Direct-subword representations required 11 to 77 forced symbols, with class medians ranging from 20 to 66, and the largest observed direct rank was 145,498. These counts and ranks depend on the model and tokenizer rather than defining a fixed alphabet. The deterministic codecs instead imposed rank ceilings of 8 or 16 while generally increasing forced length. On the four-class common hexadecimal support, median effective artifact rates were approximately 4.571 for direct subword, 4.000 for hex nibble, 2.000 for ASCII $B = 16$, and 1.497 for ASCII $B = 8$ artifact bits per forced token. These descriptive rates compare representations on identical eligible artifacts;

they are not cryptographic-security capacities. The whiskers in Fig. 1 are observed 5th to 95th percentiles, not confidence intervals. The detailed theoretical capacity and tail validation tests a different question: whether retained records satisfy the declared algebraic identities and conditional bounds (in Supplementary Fig. S6 and Table S14). The common-support empirical capacity and quality frontier follows in Fig. 2.

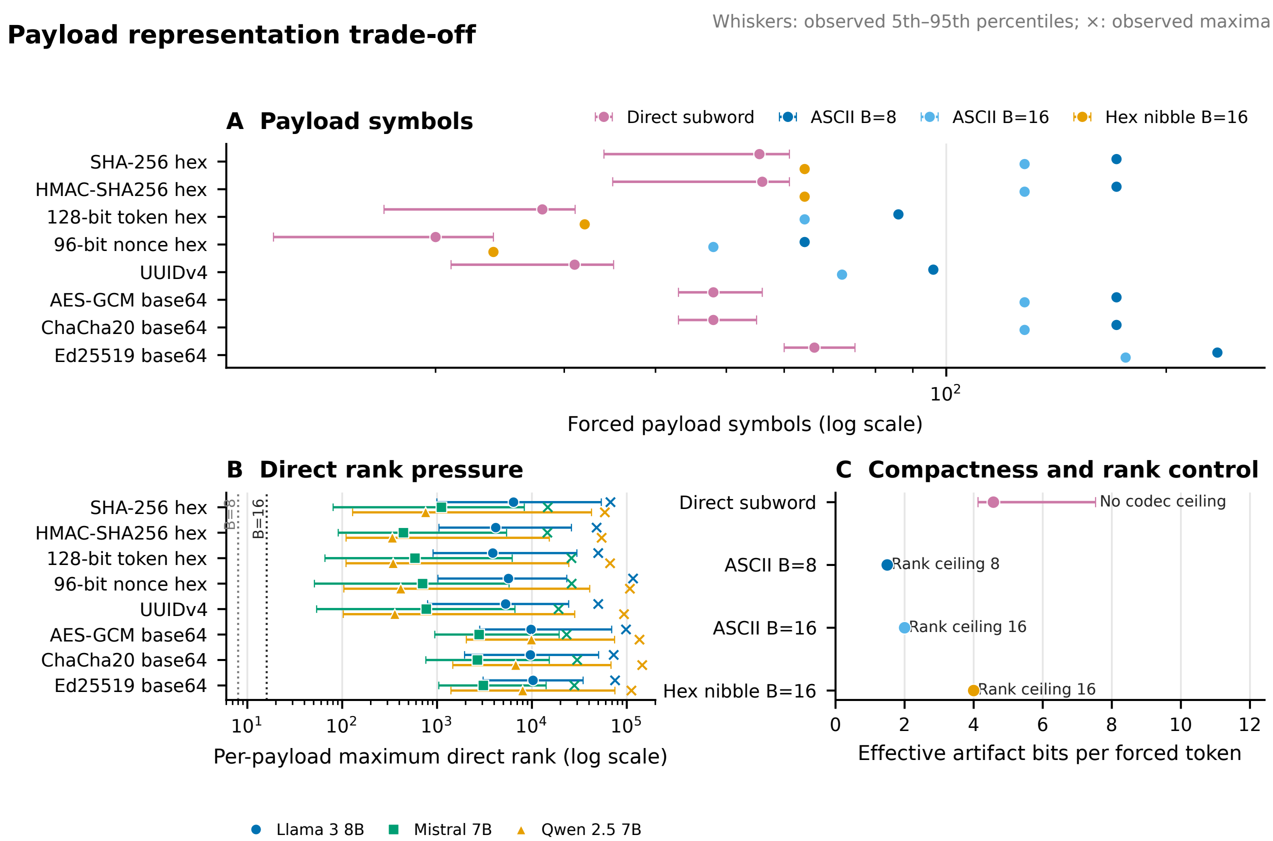


Figure 1. Payload representation, rank pressure, and capacity tradeoff. Direct-subword symbol counts and rank pressure are summarized for 1,440 observations across payloads and models comprising 60,861 direct-rank positions. Their values are tokenizer and model dependent. Bounded codecs constrain requested ranks to at most 8 or 16, at the cost of more forced positions. Effective artifact-rate comparisons use the four-class common hexadecimal support and are descriptive rather than cryptographic-security capacities. Whiskers show observed 5th to 95th percentiles, not confidence intervals.

Figure 2 compares mean token count with mean source-model token log probability over 4,320 trials on the four-class common hexadecimal support. Three models and six protocols form 18 model and protocol cells, and separate forced-span and full-message views give 36 plotted cells. Compact direct-subword spans generally occupy the low-length region but can have poorer source-model likelihood, whereas bounded protocols trade longer forced spans for controlled rank alphabets. Segmented full-message points move toward better full-message likelihood by adding non-payload natural-tail length; this movement is not improved likelihood on the payload-bearing forced span. Higher mean token log probability is better; point area represents mean automated surface-flag burden, and uncertainty uses 2,000-resample payload-bootstrap 95% confidence intervals. The complete primary matrix contains no lead-in condition, so exploratory lead-in rows were not pooled. These automated diagnostics are not human-perceived judgments, and the empirical frontier is distinct from the theoretical capacity and tail validation.

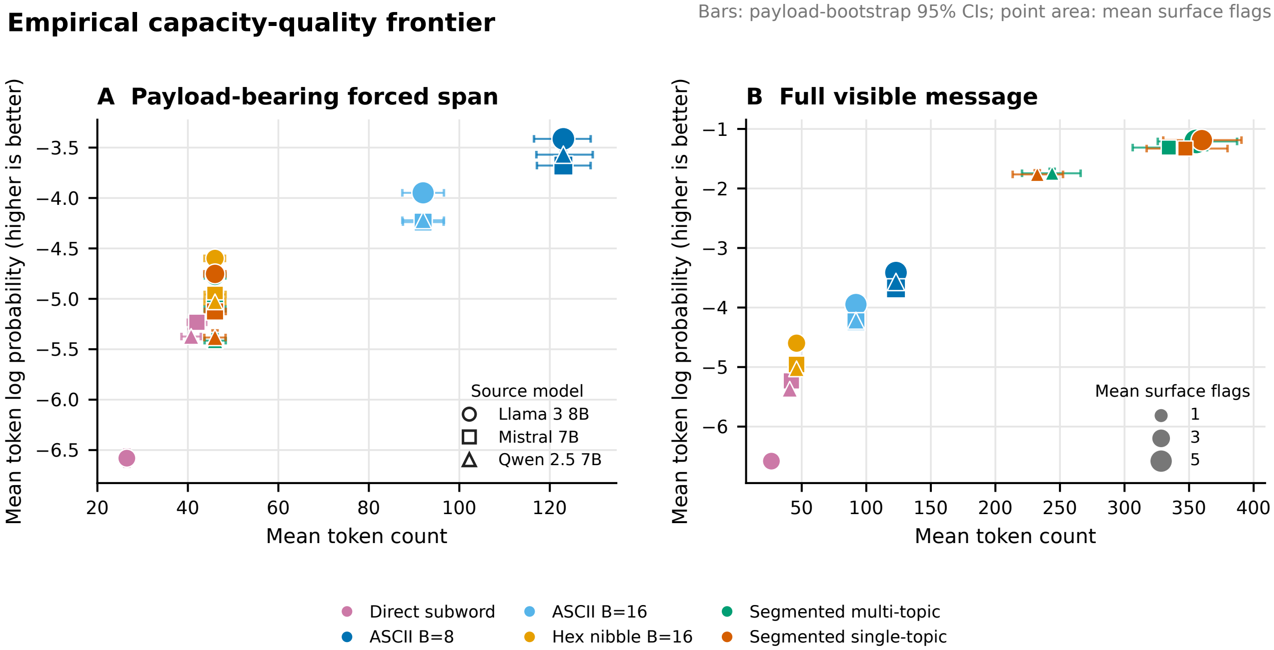


Figure 2. Empirical capacity and quality frontier. Mean token count is shown against mean source-model token log probability for 4,320 trials on four-class common hexadecimal support, spanning three models, six protocols, 18 model and protocol cells, and 36 forced-span or full-message cells. Higher mean token log probability is better; point area represents mean automated surface-flag burden. Error bars are 2,000-resample payload-bootstrap 95% confidence intervals. The primary matrix contains no lead-in condition. Likelihood and surface flags are automated diagnostics, not human judgments, and this empirical frontier is distinct from the theoretical capacity and tail validation in Supplementary Fig. S6.

Exact-copy replay and transmission fragility

Replay mode materially changed recovery (Fig. 3). Saved-token replay recovered 144/144 robustness covers, whereas detokenizing the same covers and retokenizing their visible text recovered 88/144 (0.611111). So even an unedited visible string did not guarantee the original token sequence. Raw unmodified transmission recovered 144/144, but common transformations were damaging. Unicode NFKC recovered 82/144 (0.569444), quote conversion 56/144 (0.388889), and whitespace trimming 28/144 (0.194444). Markdown copy/paste and paraphrase recovered 0/144, and the two cross-model decoder outcomes for each cover recovered 0/288.

These conditions address a different question from the primary result of whether a cover can leave its saved-token record and still recreate the forced ranks. The answer depended sharply on the frozen channel definition. The saved-ID, visible-retokenization, and raw-transmission rows are distinct replay procedures and should not be pooled. Their contrast shows why the receiver contract must name the tokenization path, not merely say that the visible text was copied exactly. Cross-model zero recovery further shows that similar prompt intent does not substitute for model and tokenizer identity.

Deleting only the declared final 10% of a message recovered 144/144 because that operation was constrained to non-payload tail tokens. It is not evidence of arbitrary truncation tolerance; removing a forced token or earlier context would change the decoded sequence. The broader matrix also showed low recovery after token or character edits and line-ending conversion (Supplementary Note S4). Limited canonicalization preserved unmodified controls but did not recover the available transformed cells, so it did not demonstrate a general mitigation.

Tail-only truncation is consequently a boundary check on the decoder definition rather than a successful error-correction result. The decoder is designed to ignore tails, so removing only tail labels should be harmless. By contrast, prefix changes and edits inside a forced span can alter a segment's initial tokenization or model history and propagate. The failed limited-canonicalization cells show that normalizing a small declared set after the fact was insufficient in this matrix; no broader mitigation is inferred.

Relative to the reviewer-requested inventory, five transformation classes were tested directly, three had only partial proxies, and three were untested. In particular, arbitrary prefix/suffix insertion, general case conversion, and general punctuation edits were not tested; line wrapping, email/markdown quoting, and sentence-boundary changes were only partially represented. First-divergence labels locate the first observed mismatch among tokenization, context, rank, or payload stages, but are descriptive and cannot by themselves establish why a transformation failed (Supplementary Fig. S1 and Supplementary Tables S7 and S8).

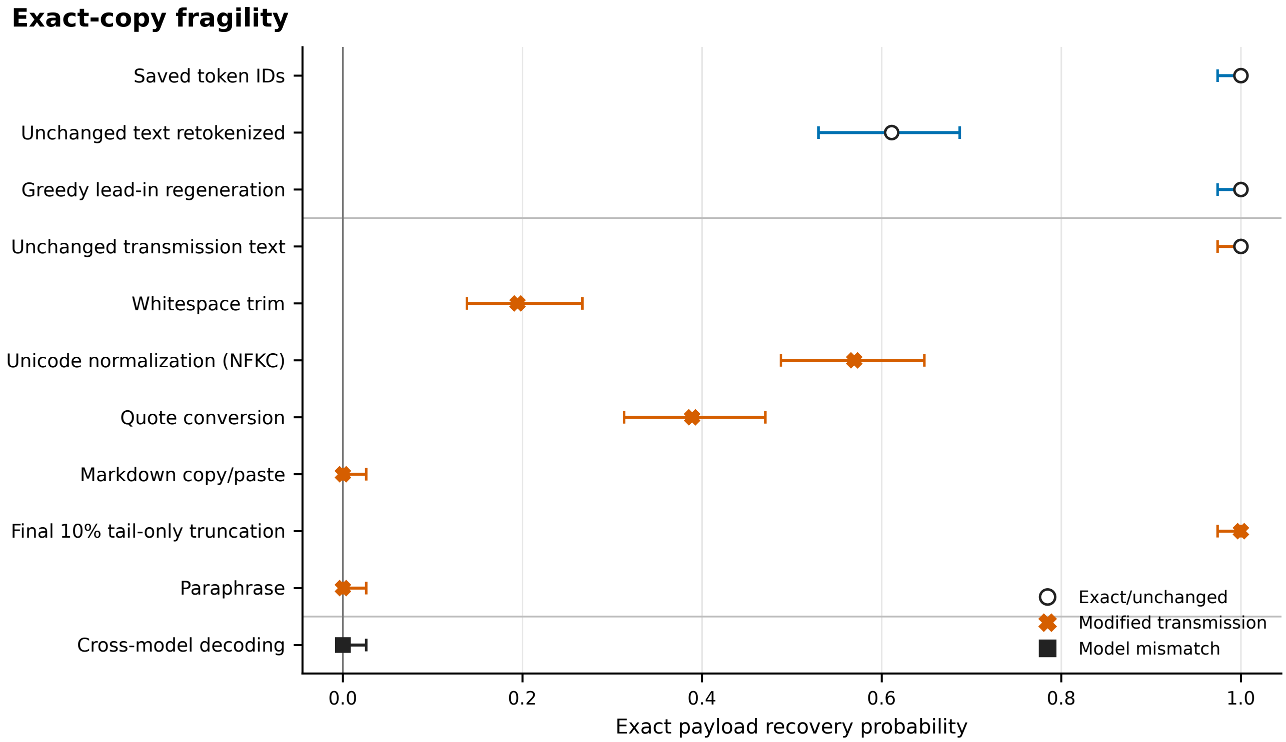


Figure 3. Exact-copy fragility. Recovery proportions and Wilson 95% intervals are shown for the principal replay and transmission conditions. Saved token IDs preserve the strongest replay contract. Visible-text detokenization and retokenization already reduce recovery. NFKC normalization, quote conversion, and trimming reduce it further, while markdown copy/paste, paraphrase, and cross-model decoding yield no successful payloads. Final 10% tail truncation removes only declared non-payload tail tokens and must not be interpreted as arbitrary truncation tolerance. The full transformation matrix, denominators, limited canonicalization cells, and first-divergence diagnostics appear in Supplementary Figure S1 and Supplementary Note S4.

Forced span versus full message

Across 1,440 paired segmented trials, overall forced-span mean token log probability was -5.090904 and overall full-message mean token log probability was -1.424774, for an overall paired gain of +3.666130 (Fig. 4). Each of the six model and topic schedule cells contained 240 paired trials, and all six 2,000-resample paired payload-bootstrap confidence intervals for the gain excluded zero.

Natural tails contain no payload ranks and add no payload capacity. The higher full-message average therefore reflects the inclusion of non-payload greedy continuations and must not be interpreted as improved quality on the payload-bearing forced span. Automated readability and surface diagnostics remain in Supplementary Fig. S2 and Note S5; they are not human evaluation.

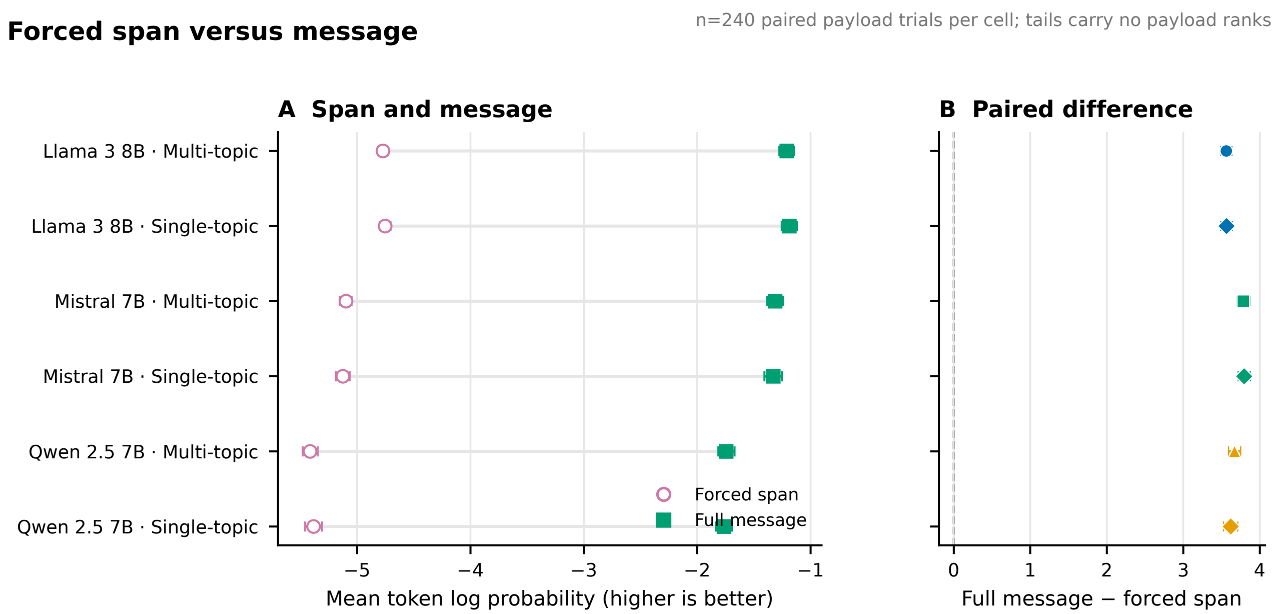


Figure 4. Forced span versus full message. Mean token log probability is paired within 1,440 segmented trials across three models and single-topic or multi-topic schedules, with 240 trials per model schedule cell. Error bars are 2,000-resample paired payload-bootstrap 95% confidence intervals; all six intervals for the full-minus-forced gain exclude zero. Natural tails contain no payload ranks and add no payload capacity, so the improved full-message average is not an improvement to the payload-bearing forced span.

Ablations, dynamic stopping, and prompt-topic effects

The locked compact ablation extraction compared 48 paired payload groups across three models and is exploratory/post-outcome (Table 2). A 32-token lead-in, relative to no lead-in, reduced mean log probability by 1.652327 (95% CI $[-1.749091, -1.553103]$) and added 159.590278 full-message tokens (CI $[125.829861, 191.030035]$). This adverse result is consistent with added context and boundary cost; the contrast does not prove a single failure mechanism. Increasing the segment size from 8 to 32 ranks increased effective rate by 1.231461 bits per full token (CI $[1.073116, 1.388188]$) and reduced full length by 174.500000 tokens (CI $[-208.405035, -140.573264]$), trading fewer tails for longer forced spans. The lead-in and segment results locate different costs. A lead-in precedes the forced span, consumes visible tokens, and changes the distribution used for every encoded rank; it therefore cannot be treated as a harmless stylistic prefix. Larger segments keep the forced count fixed but require fewer prompt resets and tail opportunities. Their higher effective rate is consistent with reduced overhead, while the forced span itself lasts longer before an ordinary greedy continuation begins.

Removing the safe-text filter changed mean log probability by approximately -0.0018 (canonical CSV estimate -0.001797; CI $[-0.037738, 0.035394]$) and effective rate by -0.062969 bits per full token (CI $[-0.134265, 0.007651]$); both intervals included zero. The Mistral roundtrip-stable-filter cell was unavailable for 48 work units, although the no-filter contrast included all three models. The filter results therefore do not establish a quality benefit. The near-zero selected filter estimates are important null evidence. The deterministic filter remains necessary to define an agreed candidate set for the canonical protocol, but this ablation did not show that it improved the selected likelihood or rate endpoints. Nor can the unavailable stable-filter Mistral condition be treated as evidence for or against that condition and it is an absence caused by the frozen isolated-vocabulary criterion. Compared with dynamic stopping, no tail increased effective rate by 3.171551 bits per full token and shortened messages by 254.951389 tokens. This arithmetic consequence of removing non-payload text is not evidence of better cover quality. A fixed sentence-tail policy of 20 to 60 tokens shortened messages by 65.708333 tokens relative to dynamic stopping, whereas its selected effective-rate interval included zero. Dynamic stopping remains a declared heuristic with an emergency cap, not a semantic judgment by readers.

The tail ablation also prevents a misleading optimization conclusion. Maximizing bits per full token alone selects no tail because every added non-payload token enlarges the denominator. RankCloak introduced tails for a different, unverified purpose: allowing a forced fragment to continue to a rule-defined stopping point. Without human ratings, the experiment measures the length/rate price of that choice but cannot determine whether the price purchases perceived naturalness. Across 15 locked topic contrasts, one Llama artifact-count comparison excluded the null; multi-topic versus single-topic expected count ratio 1.276361 (95% CI [1.114743, 1.461409]; Holm-adjusted $p = 0.003294$). The other 14 adjusted p-values were at least 0.396248. This compact extraction was performed post-outcome without refitting models and is exploratory as it does not support a general prompt-topic advantage. The solitary non-null row concerns one model and one artifact-count outcome. It did not recur across the other model/outcome combinations after adjustment. Accordingly, prompt diversity is retained as design coverage rather than promoted as a performance-enhancing intervention. Reviewer-relevant ablations, exploratory tail-gain subgroups, and all topic contrasts appear in Supplementary Figs. S3 and S4 and Supplementary Tables S10 and S11; the complete 60-row contrast table remains machine-readable in the public repository.

Analysis	Sample/group	Principal estimate	Uncertainty or adjusted test	Classification	Main limitation
32-token lead-in vs none	48 paired payloads; 3 models	Mean log probability -1.652327; full length +159.590278	CIs $[-1.749091, -1.553103]$; $[125.829861, 191.030035]$	Exploratory, post-outcome	Boundary/context interpretation is not causal
Segments of 32 versus 8 ranks	Same	Effective rate +1.231461 bits/full token; length -174.500000	CIs $[1.073116, 1.388188]$; $[-208.405035, -140.573264]$	Exploratory, post-outcome	Fewer tails accompany longer forced spans
No filter vs safe filter	Same	Log probability -0.001797; rate -0.062969	CIs $[-0.037738, 0.035394]$; $[-0.134265, 0.007651]$	Exploratory, post-outcome	Both selected intervals include zero; stable-filter Mistral cell unavailable
Tail policy	Same	No tail: rate +3.171551, length -254.951389; fixed tail: length -65.708333	Selected 95% CIs in Supplementary Note S6	Exploratory, post-outcome	Rate gain from deleting non-payload text is not quality gain
Topic schedule	15 locked contrasts	Llama artifact-count ratio 1.276361	CI $[1.114743, 1.461409]$; Holm $p = 0.003294$; other $p_{\mathrm{Holm}} \geq 0.396248$	Exploratory, post-outcome	One selected model/outcome contrast excludes null
Generation overhead	18 model and protocol cells; 12,144 matched rows	Mean $[1.674368, 16.087628]$ s; $[13.631855, 100.851772]$ bit/s	Payload-bootstrap intervals in Supplementary Note S9	Supporting measurement	Inclusive wall time; no CPU time or warm-up repeats

Table 2. Compact reviewer-requested analysis and operational summary. Differences are level minus canonical unless otherwise stated. Ablation and topic rows are a locked exploratory/post-outcome extraction from retained analyses; no new model was fit. Reviewer-relevant ablations, complete topic rows, availability, and timing cells are in the Supplementary Information; the full 60-row contrast table remains machine-readable in the repository.

Neural steganalysis

~~Both classifiers strongly separated RankCloak covers from matched controls (Fig. 5). TextCNN matched~~ On the rebuilt strictly deduplicated matched test, TextCNN ROC-AUC was ~~0.990190 (payload-group bootstrap~~ 0.9868 (95% CI ~~[0.987740, 0.992682]) with balanced accuracy 0.951010 (CI [0.943456, 0.958466]).~~ [0.9819, 0.9913]), DeBERTa-v3-base ~~ROC-AUC was 0.999834 (CI [0.999696, 0.999939])with balanced accuracy 0.991919 (CI [0.987654, 0.995665]). High performance is adverse evidence for concealment, not a security feature of RankCloak. This test directly challenges the premise that removing visible artifact syntax is sufficient for concealment. The corpus~~was ~~balanced between RankCloak and matched ordinary controls and split by payload group, so the matched estimates cannot be attributed to class imbalance or row-level payload leakage. Both a compact convolutional model and a pretrained transformer found strong signal. The evidence is limited to this constructed corpus and training procedure, but within that scope it contradicts any claim of practical undetectability.~~ 0.9994 (CI [0.9984, 1.0000]), and the exact-model-aware surprisal attacker was 0.999995 (CI [0.999970, 1.000000]; Fig. 5). Their normalized empirical 0–1% partial AUCs were 0.6652, 0.9948, and 0.999460. These values show that the encoding is often detectable; classification of a cover does not reveal the concealed artifact or restore its recognizable surface syntax.

~~Generalization depended on architecture and split. Across 18 held-out templates, TextCNN~~ Leave-one-family-out ROC-AUC ~~range was [0.720421, 0.981705] and its balanced-accuracy range was [0.565909, 0.919318]; DeBERTa ranges were [0.996291, 0.999979] and [0.952273, 0.995455]. Leave-one-model~~ ranged from 0.9186 to 0.9784 for TextCNN, 0.9972 to 1.0000 for DeBERTa, and 0.999943 to 1.000000 for the model-aware detector. These fits exclude the target family from training and validation and test unseen-family transfer within the three-family corpus, not arbitrary future models. At validation-frozen nominal 1% FPR thresholds, matched test TPR/observed FPR was 0.8317/0.0127 (19/1,491 false positives), 0.9913/0.0007 (1/1,491), and 0.9987/0.0020 (3/1,491), respectively. The matched validation and test sizes support empirical 0.1% reporting; at that target the corresponding TPRs were 0.4829, 0.9886, and 0.9987, with 2, 1, and 3 observed false positives. Family holdouts had only 497 test negatives and did not support 0.1% estimates; at frozen 1% thresholds their observed false-positive counts reached 35, 28, and 3, showing calibration drift.

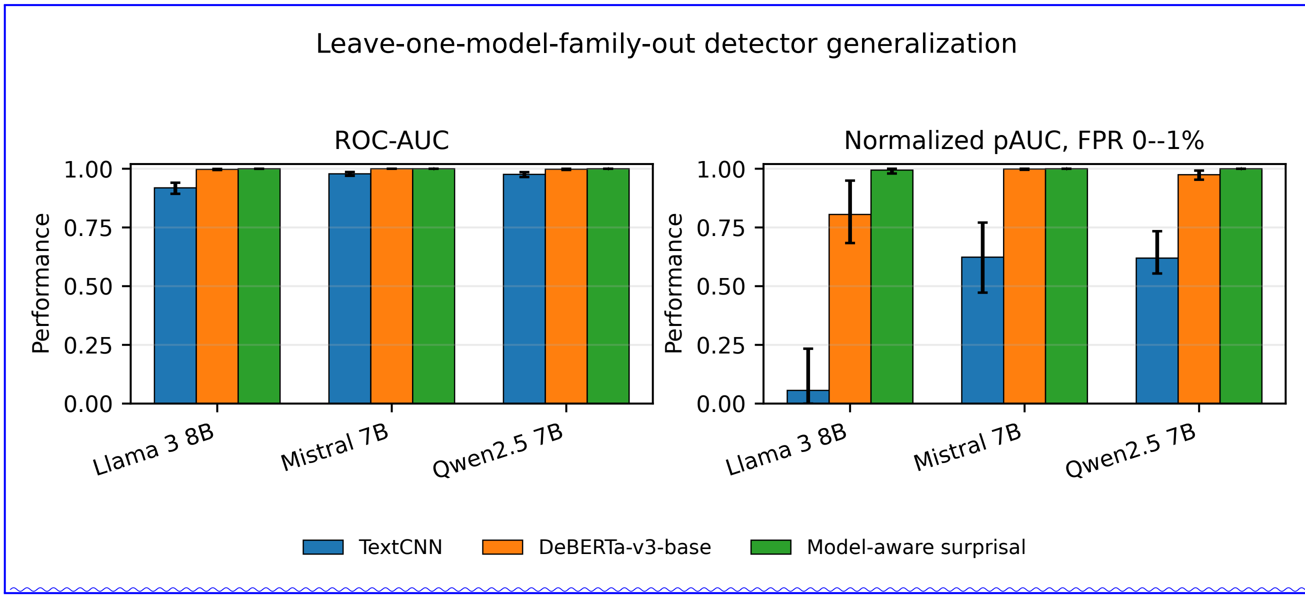


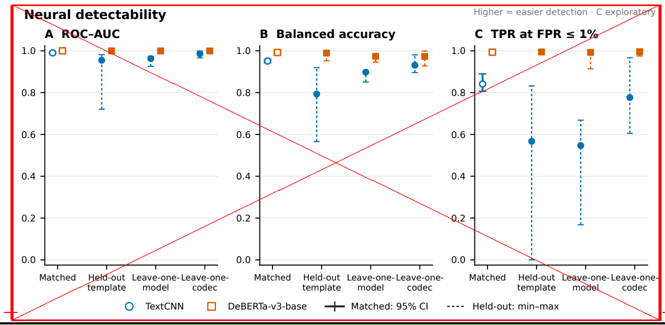
Figure 5. Strictly deduplicated detector evaluation. Matched and leave-one-family-out ROC-AUC results are shown for two data-adaptive text-only detectors and the exact-model-aware surprisal attacker. Higher performance indicates that encoded covers can be distinguished from matched controls. Components connected by declared exact or near-duplicate relations do not cross partitions; the rule does not detect semantic paraphrases below its threshold. Supplementary Note S12 reports full split counts, validation-frozen low-FPR operating points, exact false-positive counts, and unavailable 0.1% conditions.

Human controls, entropy gating, and quantization

The bounded human-authored secondary analysis selected 250 Dolly responses across 16 available templates, with no exact or declared near-duplicate links within that corpus or to the primary corpus. Against these negatives, ROC-AUC ~~ranges were [0.925623, 0.973650]~~ was 0.7537 for TextCNN and ~~[0.996772, 0.999979] for DeBERTa; leave-one-codec ranges were [0.966532, 0.998634]~~ 0.9846 for DeBERTa. At thresholds selected only on matched ordinary-model validation controls, 152/250 and ~~[0.997917, 1.000000]. These are ranges across distinct held-out splits, not confidence intervals for one pooled estimate. The TextCNN template range shows meaningful distribution sensitivity ; a high matched score alone does not specify behavior for every new prompt. DeBERTa remained near perfect over the evaluated held-out regimes, which strengthens the adverse conclusion for these model, prompt, and codec families. It still does not establish performance on arbitrary human text, unseen generation systems, or future RankCloak modifications. The ranges deliberately preserve heterogeneity instead of averaging unlike test sets~~97/250 human texts were false positives. These high rates reveal detector-domain sensitivity and may reflect generic machine-versus-human differences; the analysis is not a participant study, human naturalness rating, or evidence that readers cannot notice RankCloak covers.

~~Exact text-identity~~ Entropy gating produced a capacity–length–detectability tradeoff (Table 3). From ungated through median and strict gates, mean fixed-payload capacity conditional on completion declined from 7.4001 to 5.7920 and 4.0513 bits/token, while mean length ratio increased from 1.0000 to 1.3743 and ~~token-sequence leakage checks passed for all 28 frozen splits. A stricter lexical diagnostic nevertheless found 53 normalized near-duplicate pairs at the declared threshold, including 44 cross-payload pairs, affecting 16 splits. Excluding affected test payload groups without retraining changed ROC-AUC by -0.001925 to +0.004568 for TextCNN and -0.000038 to +0.000166 for DeBERTa; balanced-accuracy changes were similarly small. This sensitivity does not remove the design limitation because it is diagnostic, threshold-dependent, and does not rebuild the training sets. Fixed-seed repetition on the same CUDA device reproduced scores exactly for both architectures. It supplies same-device repeatability only; no CPU~~2.7615. The strict condition completed 114/~~GPU equivalence experiment was completed. Small exclusion deltas show that the observed metrics were not numerically overturned by this particular thresholded removal, but they cannot prove independence of lexical structure. The sensitivity discarded affected test payload groups after training and did not reconstruct all split assignments. Supplementary precision,Brier score,~~120 payloads; its six maximum-budget failures are retained outcomes, and its capacity denominator is the 114 completed cases. Text-only ROC-AUC declined under the strict gate to 0.8850 and ~~low-false-positive sensitivity are likewise labeled exploratory. Same-device exact repetition verifies the saved CUDA procedure, not portability across processor types (Supplementary Fig. S5 and Supplementary Tables S12 and S13).~~ 0.8893, but exact-model-aware ROC-AUC remained 0.9904. Visible-text recovery was 62/120, 65/120, and 68/120 across the three conditions, so gating neither established undetectability nor reliable visible-text transport.

For the single pinned Qwen revision, saved-ID recovery was 960/960 for both Q4_K_M and Q8_0, whereas visible-text recovery was 732/960 (76.25%) and 758/960 (78.96%). The paired Q8-minus-Q4 difference was 2.71 percentage points (grouped-bootstrap 95% CI $[-0.73, 6.35]$); paired outcomes were 588 both, 144 Q4 only, 170 Q8 only, and 58 neither, with exact two-sided McNemar $p = 0.1582$. Thus there was no statistically supported Q8 recovery advantage. Independently generated paths diverged substantially (mean positionwise match 0.0155), while cross-quantization detection remained very strong in both directions.



Analysis or condition	Recovery or completion	Capacity and length	Detection or paired inference
Strict detector split	15,838 retained rows; 9,904/2,952/2,982 train/validation/test	Deduplication before features; zero declared cross-partition leakage	Matched AUC 0.9868/0.9994/0.999995
Human-authored controls	250 frozen test negatives; no human labels used for fitting	Length-matched within 16 available templates	TextCNN/DeBERTa AUC 0.7537/0.9846; false positives 152/250 and 97/250
Entropy ungated	Complete/saved ID 120/120; visible 62/120	7.4001 bit/token, $n = 120$, conditional on completion; length ratio 1.0000	AUC TextCNN/DeBERTa/model-aware 0.9953/0.9999/1.0000
Entropy median gate	Complete/saved ID 120/120; visible 65/120	5.7920 bit/token, $n = 120$, conditional on completion; length ratio 1.3743	AUC 0.9825/0.9785/0.9999
Entropy strict gate	Complete/saved ID 114/120; visible 68/120; six budget failures	4.0513 bit/token, $n = 114$, conditional on completion; length ratio 2.7615	AUC 0.8850/0.8893/0.9904
Q4_K_M versus Q8_0	Saved ID 960/960 each; visible 732/960 versus 758/960	Independent-path position match 0.0155	Difference 2.71 points, CI $[-0.73, 6.35]$; McNemar $p = 0.1582$
Cross-quantization detection	Q4-to-Q8 and Q8-to-Q4 held-payload tests	One Qwen revision; two quantizations	AUC TextCNN 0.9927/0.9944; DeBERTa and model-aware 1.0000/1.0000

Table 3. ~~**Neural detectability.**~~ ~~Matched ROC-AUC~~ **Compact summary of the focused V3 analyses.** Entropy capacity is fixed-payload bits per generated token and ~~balanced accuracy are shown with payload-group bootstrap 95% confidence intervals~~is conditional on completion; ~~held-out-template~~entropy recovery proportions use all 120 attempts. Detector triples are TextCNN, ~~leave-one-model~~DeBERTa-v3-base, and ~~leave-one-codec results are summarized as ranges across frozen splits~~exact-model-aware surprisal. ~~Higher performance is adverse~~ Empirical 0.1% operating points are unavailable for ~~RankCloak concealment~~ the 250-negative human comparison and every entropy and quantization subgroup because ~~it indicates that covers are distinguishable from matched controls. Both classifiers completed all 28 regimes. Exact identity leakage checks passed~~each has fewer than 1,~~but a declared lexical near-duplicate diagnostic affected 16 splits; exclusion sensitivity produced small observed changes without eliminating that limitation~~000 negatives. ~~The full six-panel view and exploratory precision, Brier~~Detailed denominators, ~~low-false-positive~~intervals, ~~leakage~~thresholds, and ~~repeatability~~condition-level results appear in Supplementary ~~Figure S5~~Notes S11–S15.

Computational overhead and generalizability

Across the 18 primary model and protocol cells, mean inclusive generation time ranged from 1.674368 to 16.087628 seconds. Mean payload throughput ranged from 13.631855 to 100.851772 bits/s, and effective rate from 0.710004 to 7.322003 bits per full token. These saved wall times include wrapper setup and decoding scope defined by the harness; CPU time and repeated warm-up measurements were not retained.

The approximately order-of-magnitude spread across cells shows that representation and model choice materially affect operational cost in the retained environment. Compact direct ranks can reduce forced work, whereas byte expansion and repeated segmented continuations increase it. Throughput and effective rate answer different questions: the first normalizes payload bits by wall time, while the second normalizes by visible full-message tokens. Neither is a deployment benchmark outside the recorded wrapper and device. On the fixed CUDA benchmark, representative TextCNN wall/training times were 91.358/9.498 seconds with 306.25 MB peak allocated VRAM; DeBERTa-v3-base times were 1,536.661/1,425.063 seconds with 6.38 GB. Those device-specific measurements do not generalize across hardware. Evaluation across three model families and exact-copy Spanish and Simplified Chinese trials broadens the controlled design.

The detector timings also illustrate the cost of the stronger adverse test: the pretrained architecture required substantially more saved time and memory than TextCNN in the single retained benchmark. CPU neural training was infeasible and was not used as a comparison point. These measurements support transparent study accounting, not a claim that the approach is efficient enough for an operational sender, receiver, or monitor (Supplementary Fig. S7 and Supplementary Tables S15 and S16).

Discussion

RankCloak provides a reproducible method for converting serialized cryptographic artifacts into language-model next-token ranks and ~~measuring the cover textthat realizes them~~realizing those ranks as cover text. Its representation choices expose a central ~~trade-off~~tradeoff: direct subwords can be compact but leave ~~the~~ rank pressure uncontrolled, whereas ASCII-byte and hex-nibble codecs bound the permitted rank by spending more forced positions. Segmentation redistributes those positions across contexts, and natural tails add non-payload length. The formal and empirical capacity relationships make these costs explicit rather than treating longer cover text as additional payload capacity. ~~Complete recovery in the evaluated primary and secondary-language matrices establishes deterministic codec and replay fidelity under the saved-token exact-copy contract. It does not establish an ordinary text channel. Detokenized visible text could change tokenization without an apparent edit, and common normalization, formatting, rewriting, or model-mismatch conditions sharply reduced or eliminated recovery . The contrast between perfect~~

The principal demonstrated transformation is that the recognizable surface representation of a synthetic cryptographic artifact can be absent from the generated cover while an authorized receiver reconstructs it under the supported shared configuration. In the saved-token ~~replay and fragile~~ condition, retaining the token stream isolates tokenization fidelity, but recovery still requires contextual rank reconstruction and codec inversion; ordinary visible-text ~~transmission is therefore not a secondary implementation detail; it defines the boundary of the recovery claim~~transfer is the stricter and materially less reliable condition. Steganalysis addresses another endpoint: a detector can often flag the presence of encoding, weakening covert-channel stealth, without thereby observing the original artifact or recovering its serialization.

Cover-quality evidence remained protocol-dependent. Full-message mean log probability was higher than forced-span mean across paired segmented trials, but the added natural tails carry no ranks and the gain does not improve the payload-bearing positions or add capacity. Automated readability, surface diagnostics, and model-based contrasts remain computational proxies rather than human judgments, and no participant evidence supports a claim of human-perceived naturalness. ~~Supporting analyses reinforce this interpretation without identifying a single optimum.~~ Larger segments improved efficiency by reducing repeated tail overhead, while lead-ins worsened likelihood and length in the selected contrasts; removing tails mechanically improved rate but did not test perceived quality. ~~These exploratory comparisons show that optimization depends on which cost is prioritized rather than establishing a universally preferable protocol. Evaluation across three model families, diverse prompts, and two secondary languages broadens the controlled setting but cannot be extrapolated to arbitrary models, languages, or deployed channels.~~

~~Neural steganalysis supplies the clearest adverse finding. Both detector architectures achieved very high matchedperformance, and the pretrained detector remained highly effective across the evaluated held-out regimes. Although split sensitivity and lexical overlap limit extrapolation, the observed distinguishability directly argues against practical concealment in this corpus. Exact recovery cannot compensate for a cover distribution that trained classifiers can identify with near-perfect accuracy. Despite these adverse results, the study contributes a controlled framework for separating representation, recovery assumptions, capacity~~The focused steganalysis revision strengthens the adverse evidence without changing that endpoint distinction. Strict pre-feature component deduplication found zero leakage under the declared exact and character-*n*-gram rules, yet matched, unseen-family, and exact-model-aware detectors remained strong. Validation-frozen low-FPR results show that high ranking performance does not ensure threshold calibration after family or quantization shift. The human-authored comparison is informative but secondary: its high false-positive rates expose control-domain sensitivity and may reflect general machine-versus-human cues.

Entropy gating reduced text-only detectability in the tested matrix, but lower capacity, greater cover length, ~~automated quality, transformation sensitivity, and detectability. The expanded payload, prompt, model, and secondary-language design shows how these quantities can be evaluated without equating model likelihood with reader judgment or deterministic replay with channel robustness. RankCloak therefore establishes a measurement framework and a set of reproducible failure surfaces for rank-based hiding, but it does~~ six strict-gate completion failures, and continued strong exact-model-aware detection prevent treating it as a uniformly superior protocol. Likewise, both tested quantizations preserved saved-ID inversion, but visible-text recovery remained weaker, independently generated token paths diverged, and the paired analysis did not support a Q8 recovery advantage. These studies identify measurable design tradeoffs rather than eliminating detectability or transport limitations.

RankCloak therefore contributes an artifact-specific method, evaluation methodology, and empirical evidence about concealing and recovering synthetic cryptographic artifacts whose original syntax would be conspicuous. The experiments do not establish a ~~secure, undetectable, robust, or deployment-ready covert communication system~~cryptographic secrecy proof, universal steganographic security, or robustness under every text transformation. They instead define when rank-based artifact recovery succeeds, when visible transport fails, and how current attackers detect the encoded covers.

Responsible use and limitations

RankCloak is dual use and can encode sensitive-looking artifacts, but it does not ~~itself~~ provide encryption, authentication, confidentiality, or a ~~cryptographic-security proof; any protected payload still requires~~ cryptographic secrecy proof; protected payloads require an external cryptographic system and an explicit threat model. ~~Exact recovery depends on saved-token replay, whereas~~ The study does not claim universal undetectability. Saved-token inversion depends on the declared protocol configuration, while visible-text ~~transmission and common transformations are fragile and the tested perturbations do not establish broad edit robustness . Human-perceived naturalness, broad multilingual behavior, real-world deployment, and performance across arbitrary future models or communication channels were not established; the automated~~ recovery is weaker and markdown transfer, paraphrase, and cross-model decoding failed in the bounded robustness sample. The payloads are synthetic cryptographic artifacts, so the evidence does not establish hiding or recovery for arbitrary natural-language messages. Automated readability and likelihood measures are not participant ratings, ~~and detector performance~~ no participant study was performed, and the 250-response Dolly control is a bounded secondary corpus that may expose generic machine-versus-human differences.

Strict deduplication is limited to the ~~evaluated corpus. The evidence therefore supports RankCloak as a controlled research framework, not as a secure, undetectable, robust, or deployment-ready covert channel; detailed~~ declared normalization and character-n-gram threshold and cannot rule out semantic paraphrases. The three-family holdouts, two learned text detectors, and exact-model-aware attacker are not exhaustive. Small family, human, entropy, and quantization test sets cannot resolve empirical 0.1% operating points, and frozen thresholds can drift across family or quantization. The entropy experiment uses 120 attempts per gate and two templates; its strict gate left six payloads incomplete at the maximum budget, and its reported capacity is conditional on 114 completions. The quantization comparison covers one Qwen revision, one backend, and only Q4_K_M and Q8_0. Broader models, tokenizers, quantizations, languages, transformations, communication systems, and adversaries remain untested; complete boundaries appear in Supplementary ~~Table S17~~Notes S11–S15.

Data and Code availability

The RankCloak source code and computational data ~~supporting this study~~ are available at the GitHub repository, `https://github.com/mantzaris/llm-rankcloak`~~, and are archived in Zenodo~~. The currently published Zenodo record is available at `https://doi.org/10.5281/zenodo.21987450`. ~~The archive contains the public payload corpus, analysis inputs, detector predictions and metrics, source tables, figures, configurations, and provenance required to inspect the reported computational results~~Before submission, the final V3 revision will be synchronized to Zenodo, the archived version will be checked for the cited computational revision, and the DOI wording will be updated to match the deposited version; this statement does not claim that those final synchronization steps have already occurred.

References

1. Norelli, A. & Bronstein, M. M. LLMs can hide text in other text of the same length. In *The Fourteenth International Conference on Learning Representations* (2026).

2. Simmons, G. J. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of CRYPTO '83*, 51–67, DOI: 10.1007/978-1-4684-4730-9_5 (Springer, 1984).

3. Cachin, C. An information-theoretic model for steganography. In *Information Hiding*, 306–318, DOI: 10.1007/3-540-49380-8_21 (Springer, 1998).
4. Hopper, N. J., Langford, J. & von Ahn, L. Provably secure steganography. In *Advances in Cryptology – CRYPTO 2002*, 77–92, DOI: 10.1007/3-540-45708-9_6 (Springer, 2002).
5. Petitcolas, F. A. P., Anderson, R. J. & Kuhn, M. G. Information hiding: A survey. *Proc. IEEE* **87**, 1062–1078, DOI: 10.1109/5.771065 (1999).
6. Fang, T., Jaggi, M. & Argyraki, K. Generating steganographic text with LSTMs. In *Proceedings of ACL 2017, Student Research Workshop*, 100–106, DOI: 10.18653/v1/P17-3017 (Association for Computational Linguistics, 2017).
7. Yang, Z.-L., Guo, X.-Q., Chen, Z.-M., Huang, Y.-F. & Zhang, Y.-J. RNN-Stega: Linguistic steganography based on recurrent neural networks. *IEEE Transactions on Inf. Forensics Secur.* **14**, 1280–1295, DOI: 10.1109/TIFS.2018.2871746 (2019).
8. Ziegler, Z. M., Deng, Y. & Rush, A. M. Neural linguistic steganography. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 1210–1215, DOI: 10.18653/v1/D19-1115 (Association for Computational Linguistics, 2019).
9. Dai, F. Z. & Cai, Z. Towards near-imperceptible steganographic text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 4303–4308, DOI: 10.18653/v1/P19-1422 (Association for Computational Linguistics, 2019).
10. Shen, J., Ji, H. & Han, J. Near-imperceptible neural linguistic steganography via self-adjusting arithmetic coding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 303–313, DOI: 10.18653/v1/2020.emnlp-main.22 (Association for Computational Linguistics, 2020).
11. Zhang, S., Yang, Z., Yang, J. & Huang, Y. Provably secure generative linguistic steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 3046–3055, DOI: 10.18653/v1/2021.findings-acl.268 (Association for Computational Linguistics, 2021).
12. Wang, Y., Song, R., Li, L., Zhang, R. & Liu, J. Dynamically allocated interval-based generative linguistic steganography with roulette wheel. *Appl. Soft Comput.* **176**, 113101, DOI: 10.1016/j.asoc.2025.113101 (2025).
13. Wu, J., Wu, Z., Xue, Y., Wen, J. & Peng, W. Generative text steganography with large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, 10345–10353, DOI: 10.1145/3664647.3680562 (Association for Computing Machinery, 2024).
14. Lin, K., Luo, Y., Zhang, Z. & Luo, P. Zero-shot generative linguistic steganography. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 5168–5182, DOI: 10.18653/v1/2024.naacl-long.289 (Association for Computational Linguistics, 2024).
15. Kirchenbauer, J. *et al.* A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, 17061–17084 (PMLR, 2023).
16. Cai, S., Ding, L. & Tao, D. Entropy-guided watermarking for LLMs: A test-time framework for robust and traceable text generation. *arXiv preprint arXiv:2504.12108* DOI: 10.48550/arXiv.2504.12108 (2025).
17. Wayner, P. Mimic functions. *Cryptologia* **16**, 193–214, DOI: 10.1080/0161-119291866883 (1992).
18. Wayner, P. Strong theoretical steganography. *Cryptologia* **19**, 285–299, DOI: 10.1080/0161-119591883962 (1995).
19. Chapman, M. & Davida, G. I. Hiding the hidden: A software system for concealing ciphertext as innocuous text. In *Information and Communications Security: First International Conference, ICICS 1997*, vol. 1334 of *Lecture Notes in Computer Science*, 335–345, DOI: 10.1007/BFb0028489 (Springer, 1997).
20. Bennett, K. Linguistic steganography: Survey, analysis, and robustness concerns for hiding information in text. Tech. Rep. CERIAS TR 2004-13, Purdue University, Center for Education and Research in Information Assurance and Security (2004).
21. Chang, C.-Y. & Clark, S. Practical linguistic steganography using contextual synonym substitution and a novel vertex coding method. *Comput. Linguist.* **40**, 403–448, DOI: 10.1162/COLI_a_00176 (2014).
22. Weinberg, Z. *et al.* StegoTorus: A camouflage proxy for the Tor anonymity system. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, 109–120, DOI: 10.1145/2382196.2382211 (Association for Computing Machinery, 2012).

23. Dyer, K. P., Coull, S. E., Ristenpart, T. & Shrimpton, T. Protocol misidentification made easy with format-transforming encryption. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, 61–72, DOI: 10.1145/2508859.2516657 (Association for Computing Machinery, 2013).

24. Zander, S., Armitage, G. & Branch, P. A survey of covert channels and countermeasures in computer network protocols. *IEEE Commun. Surv. Tutorials* **9**, 44–57, DOI: 10.1109/COMST.2007.4317620 (2007).

25. Badar, L. T., Carminati, B. & Ferrari, E. A comprehensive survey on stegomalware detection in digital media, research challenges and future directions. *Signal Process.* **231**, 109888, DOI: 10.1016/j.sigpro.2025.109888 (2025).

26. Wen, J., Zhou, X., Zhong, P. & Xue, Y. Convolutional neural network based text steganalysis. *IEEE Signal Process. Lett.* **26**, 460–464, DOI: 10.1109/LSP.2019.2895286 (2019).

27. Wu, H., Yi, B., Ding, F., Feng, G. & Zhang, X. Linguistic steganalysis with graph neural networks. *IEEE Signal Process. Lett.* **28**, 558–562, DOI: 10.1109/LSP.2021.3062233 (2021).

28. Wang, H., Yang, Z., Yang, J., Chen, C. & Huang, Y. Linguistic steganalysis in few-shot scenario. *IEEE Transactions on Inf. Forensics Secur.* **18**, 4870–4882, DOI: 10.1109/TIFS.2023.3298210 (2023).

29. Gehrmann, S., Strobelt, H. & Rush, A. GLTR: Statistical detection and visualization of generated text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 111–116, DOI: 10.18653/v1/P19-3019 (Association for Computational Linguistics, 2019).

30. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D. & Finn, C. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, 24950–24962 (PMLR, 2023).

31. Sadasivan, V. S., Kumar, A., Balasubramanian, S., Wang, W. & Feizi, S. Can AI-generated text be reliably detected? stress testing AI text detectors under various attacks. *Transactions on Mach. Learn. Res.* (2025).

32. Motwani, S. R. *et al.* Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems*, vol. 37, 73439–73486, DOI: 10.52202/079017-2336 (Curran Associates, Inc., 2024).

33. Zolkowski, A., Nishimura-Gasparian, K., McCarthy, R., Zimmermann, R. S. & Lindner, D. Early signs of steganographic capabilities in frontier LLMs. In *The Fourteenth International Conference on Learning Representations* (2026).

34. National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, National Institute of Standards and Technology (2015). DOI: 10.6028/NIST.FIPS.180-4.

35. Josefsson, S. The base16, base32, and base64 data encodings. RFC 4648, DOI: 10.17487/RFC4648 (2006).

36. Davis, K., Peabody, B. & Leach, P. Universally unique IDentifiers (UUIDs). RFC 9562, DOI: 10.17487/RFC9562 (2024).

37. Sennrich, R., Haddow, B. & Birch, A. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 1715–1725, DOI: 10.18653/v1/P16-1162 (Association for Computational Linguistics, 2016).

38. Kudo, T. & Richardson, J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 66–71, DOI: 10.18653/v1/D18-2012 (Association for Computational Linguistics, 2018).

39. Yan, R. & Murawaki, Y. Addressing tokenization inconsistency in steganography and watermarking based on large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 7076–7098, DOI: 10.18653/v1/2025.emnlp-main.361 (Association for Computational Linguistics, 2025).

40. Krawczyk, H., Bellare, M. & Canetti, R. HMAC: Keyed-hashing for message authentication. RFC 2104, DOI: 10.17487/RFC2104 (1997).

41. Dworkin, M. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. Special Publication 800-38D, National Institute of Standards and Technology (2007). DOI: 10.6028/NIST.SP.800-38D.

42. Nir, Y. & Langley, A. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, DOI: 10.17487/RFC8439 (2018).

43. Josefsson, S. & Liusvaara, I. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, DOI: 10.17487/RFC8032 (2017).

44. Flesch, R. A new readability yardstick. *J. Appl. Psychol.* **32**, 221–233, DOI: 10.1037/h0057532 (1948).

45. Wilson, E. B. Probable inference, the law of succession, and statistical inference. *J. Am. Stat. Assoc.* **22**, 209–212, DOI: 10.1080/01621459.1927.10502953 (1927).

46. Kim, Y. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1746–1751, DOI: 10.3115/v1/D14-1181 (Association for Computational Linguistics, 2014).

47. He, P., Gao, J. & Chen, W. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *The Eleventh International Conference on Learning Representations* (2023).

48. Efron, B. Bootstrap methods: Another look at the jackknife. *The Annals Stat.* **7**, 1–26, DOI: 10.1214/aos/1176344552 (1979).

49. Bates, D., Mächler, M., Bolker, B. & Walker, S. Fitting linear mixed-effects models using lme4. *J. Stat. Softw.* **67**, 1–48, DOI: 10.18637/jss.v067.i01 (2015).

50. Brooks, M. E. *et al.* glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R J.* **9**, 378–400, DOI: 10.32614/RJ-2017-066 (2017).

Acknowledgements

NA

Author contributions

A.V.M. alone worked on this paper and project.

Funding

NA

Competing interests

The author declares no competing interests.

AUTHOR REVIEW COPY NOT FOR JOURNAL SUBMISSION

Supplementary Information

V2 to final V3 tracked comparison

Additions are blue and underlined.
Deletions are red and struck through.

Internal author-review artifact only

~~Supplementary Information for RankCloak: Hiding Synthetic Cryptographic Artifacts in Plain English with Language Models~~

RankCloak Conceals the Surface Form of Synthetic Cryptographic Artifacts in Language Model Generated Text

Alexander V. Mantzaris[1,*]

[1]School of Data, Mathematical, and Statistical Sciences, University of Central Florida, Orlando, FL, United States of America
[*]Correspondence: alexander.mantzaris@ucf.edu

ABSTRACT

Deleted V2 abstract
~~This Supplementary Information supports the extended RankCloak methods and the evidence produced supporting the revised manuscript. It specifies deterministic rank ordering, payload codecs, segmented forced spans, replay contracts, filtering, corpus construction, prompt and model identities, and work-unit accounting. It reports recovery strata, multilingual secondary tests, controlled transmission perturbations, automated readability and model-based quality diagnostics, reviewer-relevant ablation contrasts and all topic contrasts, neural detector splits and leakage sensitivity, capacity and quality relationships, computational overhead, and worked examples. Evidence classifications and unresolved boundaries are retained explicitly. The material does not include human evaluation, deployment validation, broad multilingual naturalness evidence, cryptographic-security proof, or a claim of robust visible-text transmission.~~

Added final V3 abstract
This Supplementary Information provides detailed methods and supporting evidence for RankCloak. It explains how synthetic cryptographic artifacts are converted into language-model token choices, reconstructed by the receiver, and evaluated for capacity, robustness, text quality, and detectability. Additional analyses examine human-written comparison text, detector generalization across model families, lower- and higher-precision versions of the same model, and embedding at positions with several plausible next words. It also documents statistical procedures, reproducibility information, experimental provenance, and the limits of the evidence.

Supplementary Information

Supplementary Note S1: Extended RankCloak methods

The main Methods contains the rank definition, Calgacus transformation, complete payload-codec equations, and the full non-segmented, segmented, and natural-tail algorithms. This note records implementation details and edge cases that extend those central presentations without duplicating their equations or pseudocode.

Direct-subword framing and bounded-codec metadata

The direct-subword arm tokenizes the original UTF-8 payload bytes with special-token insertion disabled and computes its ranks under the model's empty/BOS payload context. A preflight detokenizes the token sequence and requires the original payload bytes to be an exact suffix. The only accepted extra bytes are an explicitly retained leading-space prefix; the record stores its byte length, base64 value, and SHA-256 digest. A non-space prefix, any suffix transformation, or any payload-byte mismatch makes the representation unavailable before cover generation. During inverse transcoding, recovered ranks regenerate the payload-side token IDs, the saved prefix is removed only after exact prefix verification, and the recovered bytes and digest must equal the original. This framing correction prevents tokenization-level equality from being mistaken for recovery of the declared artifact.

For the ASCII $B = 8$ and $B = 16$ conditions, metadata stores the alphabet size, bits per symbol, exact source-byte length, and the number of zero bits appended at the end of the complete serialized bitstream. The decoder validates every recovered rank, reconstructs the padded bitstream, removes only the recorded terminal padding, and returns exactly the declared number of bytes. The $B = 8$ codec therefore uses $\lceil 8m/3 \rceil$ ranks for m serialized bytes rather than independently padding each byte to three radix-8 digits. The hexadecimal condition records the exact canonical character length and rejects ranks outside 1 to 16. It is never applied to base64 or hyphenated UUID text.

Candidate masks and deterministic ordering

The implementation converts logits to 64-bit numerical scores and orders eligible candidates with a lexicographic sort whose primary key is decreasing score and secondary key is increasing token ID. Rank lookup counts candidates with a strictly greater score plus equal-score candidates having a lower ID. This realizes the tie contract in the main Methods without a stochastic operation.

The canonical `safe_text_filter_v1` is constructed once per model and excludes empty or replacement-character pieces, disallowed controls, and declared code, markup, URL, escape, and formatting fragments. The no-filter condition uses the complete candidate vocabulary. The stricter `roundtrip_stable_filter_v1` additionally requires that each isolated token piece detokenize and retokenize to that same single ID. This isolated condition does not imply contextual visible-text stability. Its Mistral mask was empty, so the associated cells and their dependent analyses are recorded as unavailable rather than failed. Mask identity, allowed-token count, and digest are retained with each trial.

Segment records, lead-ins, and tail policies

Each segment record stores its rank slice, prompt identity and tokenized context, exact lead-in, forced, tail, and full token-ID arrays, token-role mask, and half-open forced offsets. Single-topic segmentation reuses the assigned prompt; multi-topic segmentation rotates deterministically through the six prompt categories while preserving the assigned template index. Neither segment order nor boundary metadata is authenticated by RankCloak itself.

A lead-in is a serial rank-1 prefix generated before a forced span. Saved-token replay supplies those exact IDs; the separate greedy-lead-in mode regenerates them and checks equality before decoding. Because a lead-in changes every forced-token history, it is not interchangeable with a tail. Lead-in lengths 2, 4, 8, 16, and 32 were compared with the no-lead-in condition.

The fixed sentence-tail policy of 20 to 60 tokens is Algorithm 3 in the main Methods. The completed study also froze `dynamic_completion_v1`. After at least eight greedy tokens, stopping requires terminal punctuation or a double newline, balanced parentheses, brackets, braces, and quotation marks, and no declared trailing connective or punctuation fragment. Reaching 256 tokens without satisfying the rule is an explicitly censored emergency stop. The ablation compares dynamic stopping with the fixed sentence-tail policy and with no tail. Every tail token retains the role `tail` and is excluded from rank reconstruction.

Replay modes and structured endpoints

The exact model revision, tokenizer, quantization and backend, prompt rendering, rank rule, filter, codec, span convention, and any segmentation, tail, or lead-in policy form a predeclared sender–receiver protocol configuration. Choosing this configuration is comparable to agreeing on a codec implementation and does not necessarily require renegotiation for each payload. A prompt phrase can be shared contextual information, but RankCloak does not define it as a cryptographic key.

The main Algorithms 1 and 2 define the saved-token exact-copy ~~recovery. Detokenized-text replay instead renders the complete~~ condition. In the experiments, payload-bearing token IDs were retained in the generation records and supplied to the decoder to hold tokenization and history fixed; the decoder nevertheless recomputes contextual ranks and performs codec inversion. A token-preserving channel could transmit that exact stream, whereas detokenized-text replay renders the visible message, invokes the embedded tokenizer again after stripping one automatically inserted BOS token when applicable, and applies the saved forced offsets. This boundary rule is a diagnostic; it does not search for alternate tokenizations or infer a payload span from prose. Transformed-text replay applies one frozen perturbation before this step~~. Cross-model~~, and cross-model replay uses a different declared model/tokenizer pair. Thus retaining IDs is neither irrelevant nor evidence that a separate secret copy of them must be transmitted for every message; it defines one evaluated recovery condition distinct from ordinary visible-text and transformed-text delivery.

For every mode, the record distinguishes exact forced-rank replay, exact representation recovery, and exact serialized-payload recovery. The primary endpoint requires the last of these, including byte length and SHA-256 identity. Failures retain the first segment and forced position where available, expected and observed token IDs and ranks, the context digest, and boundary offsets. Payload group and model by payload group endpoints require all constituent trials to pass. Natural tails never enter any recovery endpoint.

Supplementary Note S2: Corpus, models, prompts, and work-unit accounting

Deterministic cryptographic corpus

All 480 artifacts are public deterministic test vectors generated from declared seeds and procedures. They exercise standard formats without representing operational secrets, user credentials, or deployed traffic.

Payload class	Count	Algorithm-defined serialization
SHA-256 digest	60	Lowercase hexadecimal 256-bit digest
HMAC-SHA256 tag	60	Lowercase hexadecimal keyed-hash output under deterministic test inputs
Deterministic nonce	60	96-bit nonce serialization
128-bit token	60	Deterministically generated lowercase hexadecimal token
UUIDv4	60	Standards-conforming version/variant bits and canonical UUID display
AES-256-GCM	60	Base64 serialization of deterministic authenticated-encryption output
ChaCha20-Poly1305	60	Base64 serialization of deterministic AEAD output
Ed25519 signature	60	Base64 serialization of a deterministic signature test output

Table S1. Cryptographic test-vector corpus. Counts are balanced by class. "Real" refers to outputs of the named algorithms, not to private operational data.

Pinned model and tokenizer identities

All models used the `llama_cpp` backend, one model loaded at a time, embedded GGUF tokenizers, rank-replay batch and microbatch sizes of one, and Q4_K_M quantization. The retained manifests require exact artifact hashes and backend/library identities.

Model ID	GGUF file	Size (bytes)	SHA-256
`llama3_8b_instruct_q4_k_m`	`Meta-Llama-3-8B-Instruct.Q4_K_M.gguf`	4,920,734,272	`86c8ea6c8b755687d0b723176fcd0b2411ef80533d23e2a5030f845d13ab2db7`
`qwen2_5_7b_instruct_q4_k_m`	`Qwen2.5-7B-Instruct-Q4_K_M.gguf`	4,683,074,240	`65b8fcd92af6b4fefa935c625d1ac27ea29dcb6ee14589c55a8f115ceaaa1423`
`mistral_7b_instruct_v0_3_q4_k_m`	`Mistral-7B-Instruct-v0.3-Q4_K_M.gguf`	4,372,812,000	`1270d22c0fbb3d092fb725d4d96c457b7b687a5f5a715abe1e818da303e562b6`

Table S2. Executed model artifacts. Repository revisions for Qwen and Mistral and all upstream identities are retained in the model manifest. The three configured GGUF files are not present in the current checkout, so no fresh local generation replay was attempted for this revision.

English prompt identities

The 18 templates were fixed before the completed primary matrix. Multi-topic segmentation rotated through this schedule, single-topic segmentation reused its declared topic.

Table S3. Complete English prompt schedule.

Category	Prompt ID	Exact prompt text
Casual conversation	`casual_weekend_chat`	Write a friendly message to a friend about making relaxed plans for the weekend. Use ordinary conversational English and complete thoughts.
Casual conversation	`casual_hobby_chat`	Write a casual message to a friend describing a hobby you have been enjoying lately. Keep the tone natural, warm, and specific.
Casual conversation	`casual_neighborhood_chat`	Write an informal message to a neighbor about something pleasant noticed nearby. Use clear everyday language and finish the message naturally.
Professional communication	`professional_project_update`	Write a concise professional project update for colleagues, explaining recent progress, the next step, and one practical consideration.
Professional communication	`professional_schedule_note`	Write a polite workplace message proposing a meeting time and briefly explaining what the discussion should accomplish.
Professional communication	`professional_process_email`	Write a short professional email clarifying a routine process and inviting colleagues to raise questions. Use complete, neutral prose.

Category	Prompt ID	Exact prompt text
Forum Q&A	`forum_practical_answer`	Write a helpful answer to an online forum question about solving a common household problem. Explain the reasoning in plain English.
Forum Q&A	`forum_purchase_advice`	Write a balanced forum reply to someone comparing two everyday purchases. Mention useful tradeoffs without sounding promotional.
Forum Q&A	`forum_learning_answer`	Write a supportive question-and-answer style response to a beginner learning a practical skill. Give concrete, cautious advice.
Recipe/procedure	`procedure_simple_meal`	Write clear prose instructions for preparing a simple weeknight meal. Explain the order of the steps and how to tell when it is ready.
Recipe/procedure	`procedure_home_task`	Write an easy-to-follow explanation of a routine home maintenance task. Include preparation, the main steps, and a sensible final check.
Recipe/procedure	`procedure_garden_task`	Write practical instructions for a basic gardening task. Use natural sentences, explain why the steps matter, and end cleanly.
Personal narrative/blog	`narrative_small_discovery`	Write a short personal blog passage about discovering something useful during an ordinary day. Keep the reflection concrete and understated.
Personal narrative/blog	`narrative_local_walk`	Write a first-person narrative about a walk through a familiar place and one detail that changed the writer's perspective.
Personal narrative/blog	`narrative_learning_moment`	Write a brief personal story about learning from a minor mistake. Use coherent, natural prose and a restrained conclusion.
Factual/explanatory	`explain_weather_pattern`	Explain a familiar weather pattern to a general audience. Connect cause and effect clearly without using equations or specialist jargon.
Factual/explanatory	`explain_everyday_system`	Explain how an everyday public system works for a curious reader. Use accurate, neutral language and a logically complete explanation.
Factual/explanatory	`explain_biological_process`	Explain a basic biological process in accessible prose. Define the central idea, describe the sequence, and avoid unsupported claims.

Experimental matrices and reconciliation

Matrix	Declared units	Successful	Unavailable	Scope
Primary	14,400	14,400	0	Six RankCloak protocol/codec conditions plus ordinary controls
Ablation	1,872	1,824	48	Lead-in, segment size, filter, and tail policy
Robustness	3,744	3,408	336	Replay, raw transformations, limited mitigation, and cross-model mismatch
Multilingual	1,152	1,152	0	Spanish and Simplified Chinese payload/control generation
Held-out evaluator	17,280	17,232	48	Source and held-out model scoring
Detector	56	56	0	Two architectures across 28 frozen splits
Total	38,504	38,072	432	All declared work reconciled; zero execution failure; zero remaining

Table S4. Complete work-unit accounting. "Unavailable" means the declared source condition or compatible retained output did not exist; it is excluded from estimands and is not an observed experimental failure.

The 432 unavailable units comprise 48 Mistral roundtrip-stable-filter ablation units, 336 Mistral limited-canonicalization robustness units (seven conditions times 48), and 48 corresponding held-out-evaluator units. The root condition was an empty isolated roundtrip vocabulary. All other matrices completed. Primary work comprised 1,440 trials each for direct subword, ASCII $B = 8$, and ASCII $B = 16$, and 720 each for non-segmented hex, segmented single-topic hex, and segmented multi-topic hex. The detector corpus and fits are accounted as declared detector work, not as additional generation.

Supplementary Note S3: Detailed recovery and multilingual results

Stratum	Success	Total	Interpretation
Llama 3 8B, all payload protocols	2,160	2,160	Saved-token exact serialized-payload recovery
Mistral 7B, all payload protocols	2,160	2,160	Same endpoint
Qwen 2.5 7B, all payload protocols	2,160	2,160	Same endpoint
Direct subword	1,440	1,440	All three models
ASCII byte, $B = 8$	1,440	1,440	All three models; rank ceiling 8
ASCII byte, $B = 16$	1,440	1,440	All three models; rank ceiling 16
Hex nibble, non-segmented	720	720	Eligible hex-like payloads
Hex nibble, segmented single-topic	720	720	Forced spans decoded; tails ignored
Hex nibble, segmented multi-topic	720	720	Forced spans decoded; tails ignored

Table S5. Primary recovery by model and protocol. Every row uses the same exact serialized-payload endpoint. These overlapping summaries are descriptive strata, not additional independent trials.

At the frozen trial endpoint, ~~6,480/~~exact recovery was observed in all 6,480 trials ~~recovered, with~~ (Wilson 95% CI [0.9994075335, 1.0000000000]). At the strict payload endpoint, a payload passed only when every observed model, prompt, and protocol trial passed: 480/480, CI [0.9920605009, 1.0000000000]. At the endpoint grouped by model and payload, the result was 1,440/1,440, CI [0.9973394178, 1.0000000000]. Exact rank replay was retained as a diagnostic; the confirmatory outcome was the original serialized payload bytes and digest.

Language	Trial success	Payload-group success	Trial Wilson 95% CI	Scope
Spanish	288/288	48/48	[0.986837, 1.000000]	Secondary saved-token exact-copy recovery
Simplified Chinese	288/288	48/48	[0.986837, 1.000000]	Secondary saved-token exact-copy recovery
Combined	576/576	96/96	Descriptive combined total	No human naturalness or broad multilingual inference

Table S6. Secondary multilingual recovery. Each language's payload-group Wilson interval is [0.925900, 1.000000].

These results do not test translation, code switching, cross-language decoding, or native-speaker judgments. They also remain conditional on the same model/tokenizer artifacts and exact saved tokens. The GGUF artifacts are absent from the current checkout; consequently the retained results can be integrity-checked against their manifests, but fresh generation or rank replay cannot be reproduced locally from this checkout alone.

Supplementary Note S4: Robustness and failure analysis

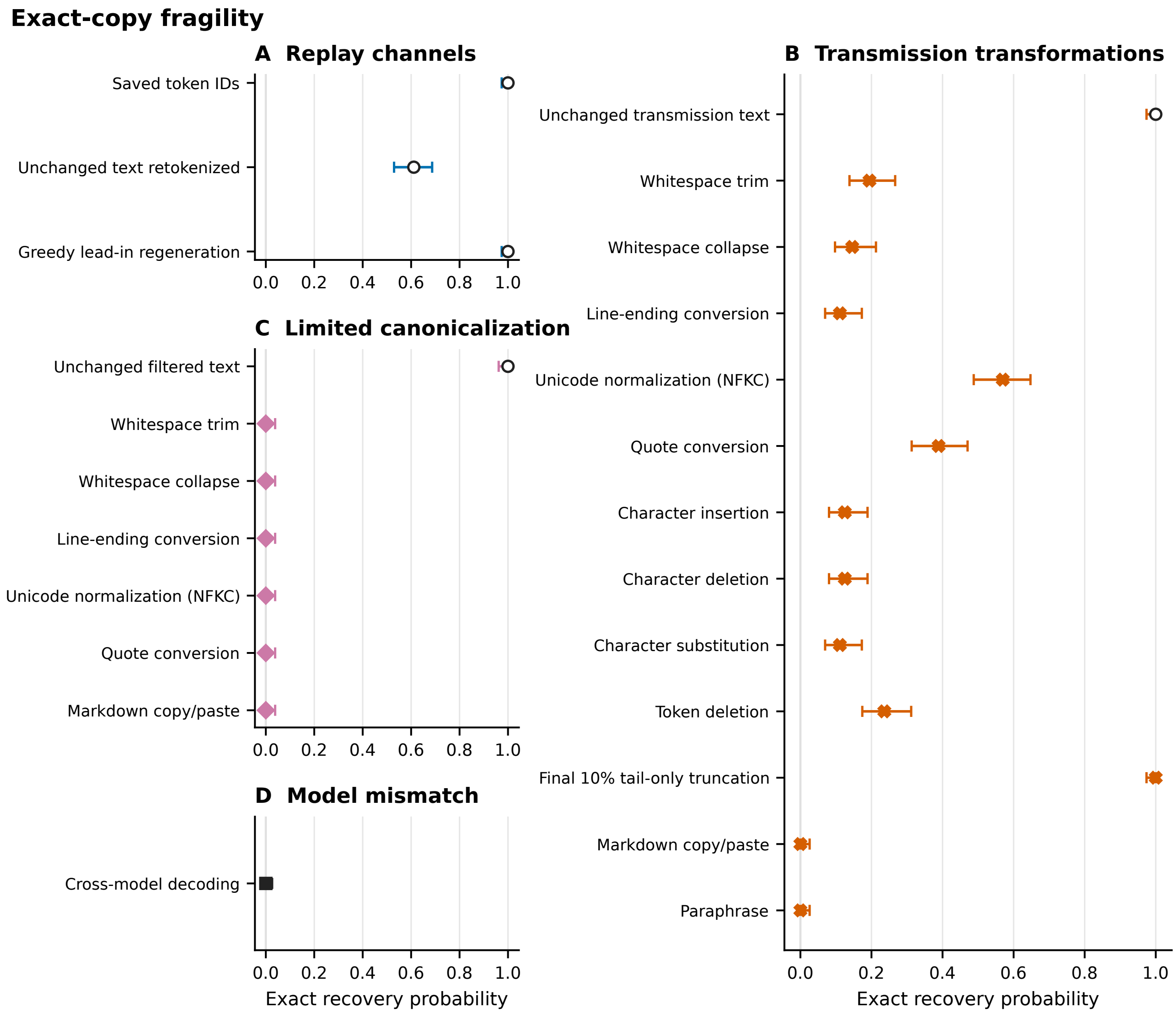


Figure S1. Complete recovery matrix and failure diagnostics. The upper panels distinguish saved-token, visible-text, transformed-text, limited canonicalization, and cross-model replay. Intervals use source-cover Wilson calculations rather than treating two cross-model decoders as independent covers. The lower panels summarize first-divergence categories. Those categories identify the first recorded mismatch and are descriptive, not causal proof. Unavailable Mistral limited-mitigation cells are excluded rather than counted as failures.

Table S7. Complete principal robustness recovery table.

Family/replay	Condition	Success	Total	Wilson 95% CI
Replay: saved IDs	Unmodified	144	144	[0.974016, 1.000000]
Replay: detokenize/retokenize	Unmodified	88	144	[0.529589, 0.686859]
Replay: greedy lead-in regeneration	Unmodified	144	144	[0.974016, 1.000000]
Raw transmission	Unmodified	144	144	[0.974016, 1.000000]

Family/replay	Condition	Success	Total	Wilson 95% CI
Raw transmission	Final 10% tail-only truncation	144	144	[0.974016, 1.000000]
Raw transmission	Unicode NFKC	82	144	[0.487804, 0.647476]
Raw transmission	Quote conversion	56	144	[0.313141, 0.470411]
Raw transmission	Token deletion	34	144	[0.174168, 0.311768]
Raw transmission	Whitespace trim	28	144	[0.138095, 0.266672]
Raw transmission	Whitespace collapse	21	144	[0.097405, 0.212667]
Raw transmission	Character deletion	18	144	[0.080551, 0.188937]
Raw transmission	Character insertion	18	144	[0.080551, 0.188937]
Raw transmission	Character substitution	16	144	[0.069559, 0.172872]
Raw transmission	LF-to-CRLF line endings	16	144	[0.069559, 0.172872]
Raw transmission	Markdown blockquote copy/paste	0	144	[0.000000, 0.025984]
Raw transmission	Deterministic paraphrase	0	144	[0.000000, 0.025984]
Cross-model retokenized	Unmodified, two alternate decoders	0	288 outcomes/144 covers	source-cover CI [0.000000, 0.025984]
Limited canonicalization	Unmodified	96	96	[0.961524, 1.000000]
Limited canonicalization	Each of six available transformed cells	0	96 per cell	[0.000000, 0.038476]

The limited canonicalization cells covered line endings, markdown copy/paste, quote conversion, NFKC, whitespace collapse, and whitespace trim. Each had 96 observations and 48 unavailable Mistral source conditions. The all-zero transformed results do not support a mitigation claim.

Requested class	Coverage	Frozen proxy	Boundary
Whitespace changes	Direct	Trim, collapse, line endings	Separate conditions; not a pooled estimand
Leading/trailing whitespace	Direct	Trim	Addition was not tested
Unicode normalization	Direct	NFKC	Other forms untested
Quotation marks	Direct	Straight-to-smart conversion	Reverse/locale-specific forms untested
Copy/paste	Direct	Fixed markdown blockquote pipeline	Not an application/OS survey
Line wrapping	Partial	LF-to-CRLF proxy	No visual reflow/insertion test
Markdown/email quoting	Partial	Markdown blockquote	No client headers, nesting, or reflow
Sentence boundaries	Partial	Broad paraphrase	Boundary edits not isolated
Punctuation changes	Untested	None	Alphanumeric substitution is not a punctuation test
Arbitrary prefix/suffix	Untested	None	Blockquote markup is not general insertion
Case conversion	Untested	None	No estimate available

Table S8. Perturbation-coverage inventory: five directly tested, three partially represented, and three untested requested classes.

Failure summaries classify detokenization/retokenization divergence, whitespace/markup tokenization divergence, quotation divergence, Unicode normalization divergence, content-edit divergence, semantic rewrite divergence, and model/tokenizer identity mismatch. For example, a mismatch can appear at the first forced position because a transformed prefix changes the context, even when later visible words are unchanged. The recorded first-difference position, 'rank-out-of-bound' flag, and token/rank lengths are useful localization evidence, but they cannot distinguish all interacting causes.

Supplementary Note S5: Automated readability and model-based quality

The computational stimulus design selected 504 rows balanced over seven conditions (72 per condition) and over all 18 prompt templates. Selection and randomization metadata were frozen in the retained manifest. Metrics were computed on visible strings. Flesch Reading Ease is a syllable/sentence surface heuristic. Surface-flag counts search declared formatting and artifact patterns. TF-IDF cosine similarity measures lexical prompt overlap. None of these quantities records a reader's judgment.

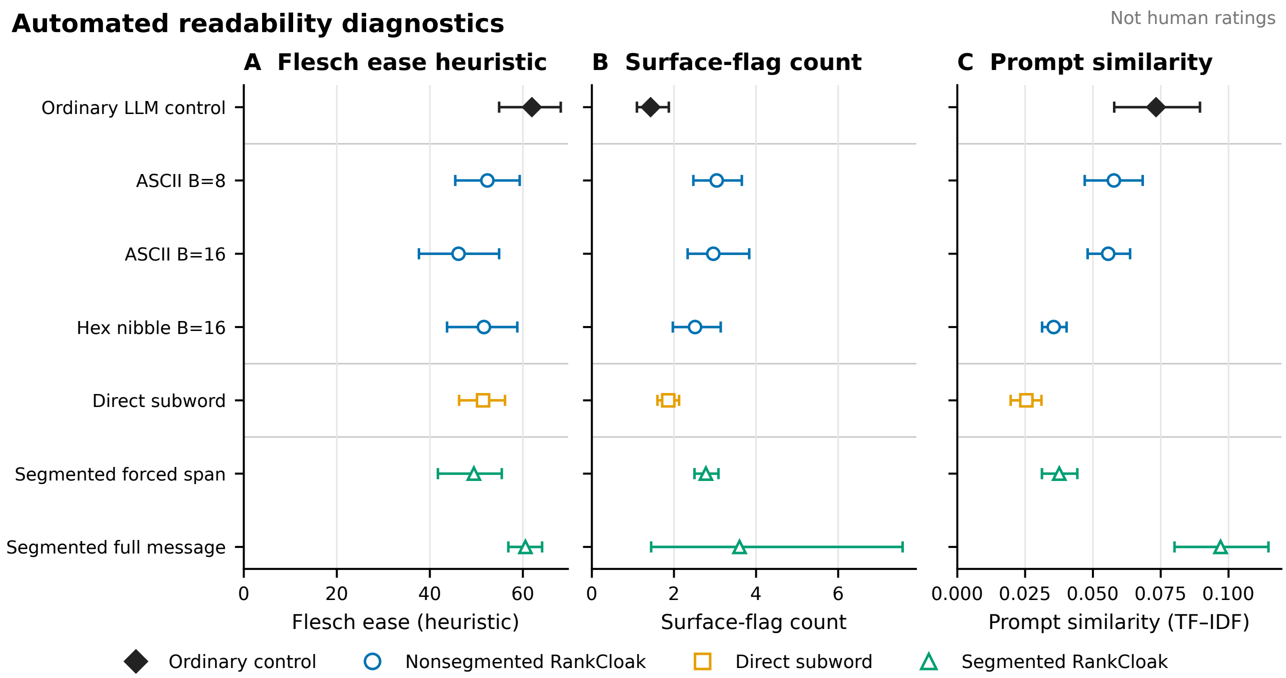


Figure S2. Automated readability diagnostics. Flesch Reading Ease and related surface diagnostics are summarized over 504 balanced computational stimuli, with uncertainty resampled over prompt templates. Segmented full messages include payload-bearing forced spans and non-payload natural tails. These measures are automated surface properties, not human ratings, evidence of human-perceived naturalness, or substitutes for a participant study; no participant was recruited and no rating was collected.

Condition	Mean Flesch	Prompt-template bootstrap 95% CI	Interpretation
Ordinary control	61.940597	[54.897042, 68.143167]	Automated control reference
Segmented full message	60.551834	[56.873661, 64.135677]	Forced spans plus non-payload tails

Table S9. Principal automated readability rows. The complete figure includes all seven balanced conditions and associated surface diagnostics.

Source-model quality supplied 45 protocol/model contrasts; 42 had Holm-adjusted $p < 0.05$, with estimates from -5.298565 to +2.684536. The independently held-out evaluator also supplied 45 contrasts, 42 adjusted $p < 0.05$, from -6.161972 to +3.287497. Signs differed by protocol and model, so there is no universal quality ordering. The source-cover-log-probability and held-out-evaluator mixed models were singular. Their coefficient tables remain diagnostic; no fixed-effects fallback was substituted.

The empirical capacity and quality frontier shown in Main Fig. 2 contains 4,320 trials restricted to the four payload classes on the common hexadecimal support. Three models and six protocols form 18 model and protocol cells; separate forced-span and full-message views yield 36 plotted cells. For every cell, mean token count is reported on the horizontal axis, mean source model token log probability is reported on the vertical axis, and point area represents mean automated surface-flag burden. Intervals use 2,000 payload-bootstrap resamples. Non-hexadecimal payload classes are excluded by the common-support restriction, and no lead-in condition exists in the complete primary matrix. Forced-span metrics average only payload-bearing token positions. Full-message metrics additionally include greedy tail positions and can therefore improve as the tail becomes longer even if the forced span is unchanged. Both are reported to avoid treating tail fluency as evidence about the embedded positions. The human-study status manifest records recruitment authorized = false, survey deployed = false, participants = 0, and ratings = 0. Planning materials are retained only to document a possible future design. They were not administered and are not reproduced here as a completed instrument. A model-name mismatch in those planning materials (Llama 3.1 versus the executed Llama 3 model) further prevents treating them as an approved executed protocol.

Supplementary Note S6: Ablations and topic effects

Locked ablation extraction

The ablation matrix paired 48 payload groups within each available model. The canonical setting was no lead-in, eight ranks per segment, the safe-text filter, and dynamic semantic stopping. Differences in Table S10 are level minus canonical. These compact contrasts were extracted after outcomes were available, without fitting new models, and are classified as exploratory/post-outcome. Payload group is the pairing and 2,000-resample bootstrap unit. The printed table presents the reviewer-relevant contrasts; the complete 60-row machine-readable contrast table, including multiplicity fields and secondary outcomes, remains in the public repository.

Across the evaluated 2, 4, 8, 16, and 32 token conditions, lead-ins generally worsened likelihood, and longer lead-ins generally increased full-message length. The 32-token contrast had mean-log-probability difference -1.652327 (95% CI, -1.749091 to -1.553103) and full-token-count difference +159.590278 (125.829861 to 191.030035). This is an adverse boundary/context result, but the paired contrast is not a causal decomposition. Segment-size changes altered efficiency and cover length: moving from 8 to 32 ranks increased effective rate by 1.231461 bits per full token (1.073116 to 1.388188) and decreased full-message length by 174.500000 tokens (-208.405035 to -140.573264). Forced-token count is codec-determined and unchanged; the length difference arises from fewer segment tails.

Removing the filter changed mean log probability by -0.001797 (-0.037738 to 0.035394) and effective rate by -0.062969 (-0.134265 to 0.007651); both selected intervals include zero. The Mistral roundtrip-stable-filter condition was unavailable for 48 work units, so the stable-filter zero rows use two models, whereas the no-filter comparison uses three. These contrasts do not establish a filter benefit.

Relative to dynamic stopping, no tail increased effective rate by 3.171551 bits per full token and reduced full-message length by 254.951389 tokens; removing non-payload text cannot be interpreted as a cover-quality improvement. The fixed sentence-tail policy of 20 to 60 tokens reduced full-message length by 65.708333 tokens. Dynamic stopping checks a frozen sentence-boundary/length predicate and then applies an emergency cap; it is not a reader judgment.

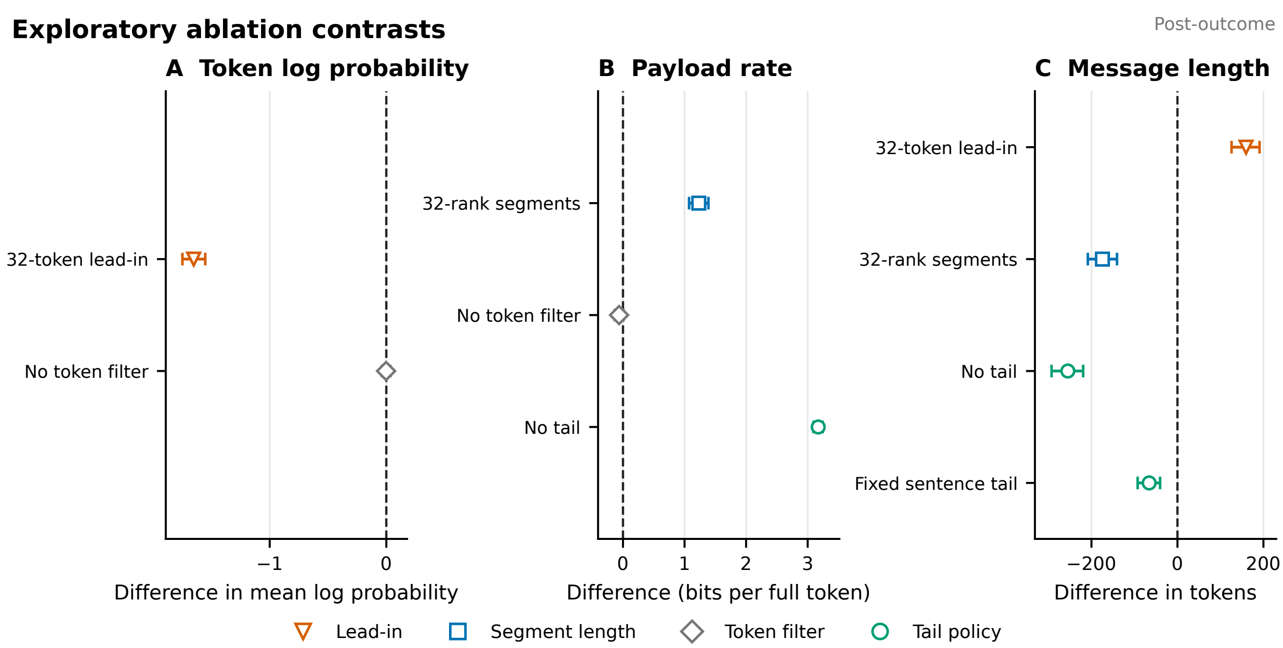


Figure S3. Ablation summary. Selected paired level-minus-canonical estimates are shown with payload-bootstrap 95% confidence intervals. Rows are exploratory/post-outcome evidence extraction from retained outputs. The Mistral roundtrip-stable-filter cell is unavailable for 48 work units; unavailable is not an execution failure. No-tail rate gains primarily reflect deletion of non-payload tokens and are not quality evidence.

Factor and level	Δ mean log probability (95% CI)	Δ full tokens (95% CI)	Δ bits/full token (95% CI)	Interpretation or availability
Lead-in: 2 tokens	-0.575880 [-0.667347, -0.491716]	+13.534722 [-21.431424, 50.179861]	-0.063224 [-0.157433, 0.024628]	Likelihood lower; length and rate intervals include zero
Lead-in: 4 tokens	-0.788830 [-0.883630, -0.694343]	+0.520833 [-36.346181, 35.365104]	-0.051850 [-0.130954, 0.029064]	Likelihood lower; length and rate intervals include zero
Lead-in: 8 tokens	-1.041886 [-1.115991, -0.961546]	+56.750000 [25.909201, 87.697396]	-0.179845 [-0.262452, -0.101276]	Lower likelihood, longer cover, lower rate
Lead-in: 16 tokens	-1.367867 [-1.450430, -1.285953]	+124.645833 [85.268229, 164.115799]	-0.299132 [-0.379449, -0.216430]	Lower likelihood, longer cover, lower rate
Lead-in: 32 tokens	-1.652327 [-1.749091, -1.553103]	+159.590278 [125.829861, 191.030035]	-0.388736 [-0.469659, -0.312450]	Largest adverse lead-in contrast
Segment size: 4 ranks	+0.116065 [0.045376, 0.192279]	+212.527778 [172.067535, 253.639236]	-0.373965 [-0.451966, -0.300845]	More segment tails; longer cover and lower rate
Segment size: 16 ranks	+0.064803 [-0.000267, 0.131660]	-104.694444 [-135.813021, -74.589410]	+0.524216 [0.418241, 0.640763]	Fewer tails; shorter cover and higher rate
Segment size: 32 ranks	+0.152306 [0.076589, 0.230426]	-174.500000 [-208.405035, -140.573264]	+1.231461 [1.073116, 1.388188]	Fewer tails; shortest selected cover contrast
No filter	-0.001797 [-0.037738, 0.035394]	+30.090278 [1.998264, 59.432986]	-0.062969 [-0.134265, 0.007651]	No clear likelihood or rate benefit
Stable filter	0.000000 [0.000000, 0.000000]	0.000000 [0.000000, 0.000000]	0.000000 [0.000000, 0.000000]	Two-model invariant; Mistral unavailable, not failed
No tail	0.000000 [0.000000, 0.000000]	-254.951389 [-292.821007, -219.249479]	+3.171551 [3.094005, 3.244380]	Mechanical deletion of non-payload tokens
Fixed sentence tail	0.000000 [0.000000, 0.000000]	-65.708333 [-92.590451, -40.372917]	-0.037921 [-0.113898, 0.033508]	Shorter than dynamic; rate interval includes zero

Table S10. Reviewer-relevant ablation contrasts. Differences are level minus the canonical setting (no lead-in, eight ranks per segment, safe-text filter, and dynamic tail). Estimates pool three available models and 48 paired payloads per model except the stable-filter row, which contains two models because the 48-unit Mistral condition was unavailable. Intervals are payload-bootstrap 95% intervals. Mean log probability is evaluated on payload-bearing forced positions; zero tail-policy differences arise because tail choice does not change those positions. All rows are exploratory/post-outcome.

Prompt-topic contrasts

The topic extraction compares the multi-topic and single-topic segmented schedules within model. Artifact counts are ratios on the response scale so the remaining estimates are multi-topic minus single-topic contrasts on their reported model scales. One of 15 rows excluded the null: the Llama expected artifact-count ratio was 1.276361 (95% CI, 1.114743 to 1.461409; Holm-adjusted $p = 0.003294$). Every other Holm-adjusted value was at least 0.396248. This locked extraction is exploratory and post-outcome where it supports neither a general topic advantage nor a new confirmatory hypothesis.

A separate exploratory subgroup display defines tail gain as full-message mean token log probability minus forced-span mean token log probability. It contains 8,280 paired segments from 1,440 segmented trials across three models, six prompt categories, and single-topic and multi-topic schedules, giving 36 displayed groups. Every group contains 230 segments, and all 36 group means were positive, ranging from +2.825721 to +3.652576. The numbers of independent payload clusters differ by schedule, 40 for single-topic and 190 for multi-topic. Uncertainty uses 2,000-resample payload-cluster bootstrapping stratified by payload class. This subgroup estimand is not equivalent to the adjusted single-topic versus multi-topic mixed-model contrast where the adjusted source-quality contrasts ranged from -0.02214 to +0.01881 and had Holm-adjusted $p = 1.0$ for all three models.

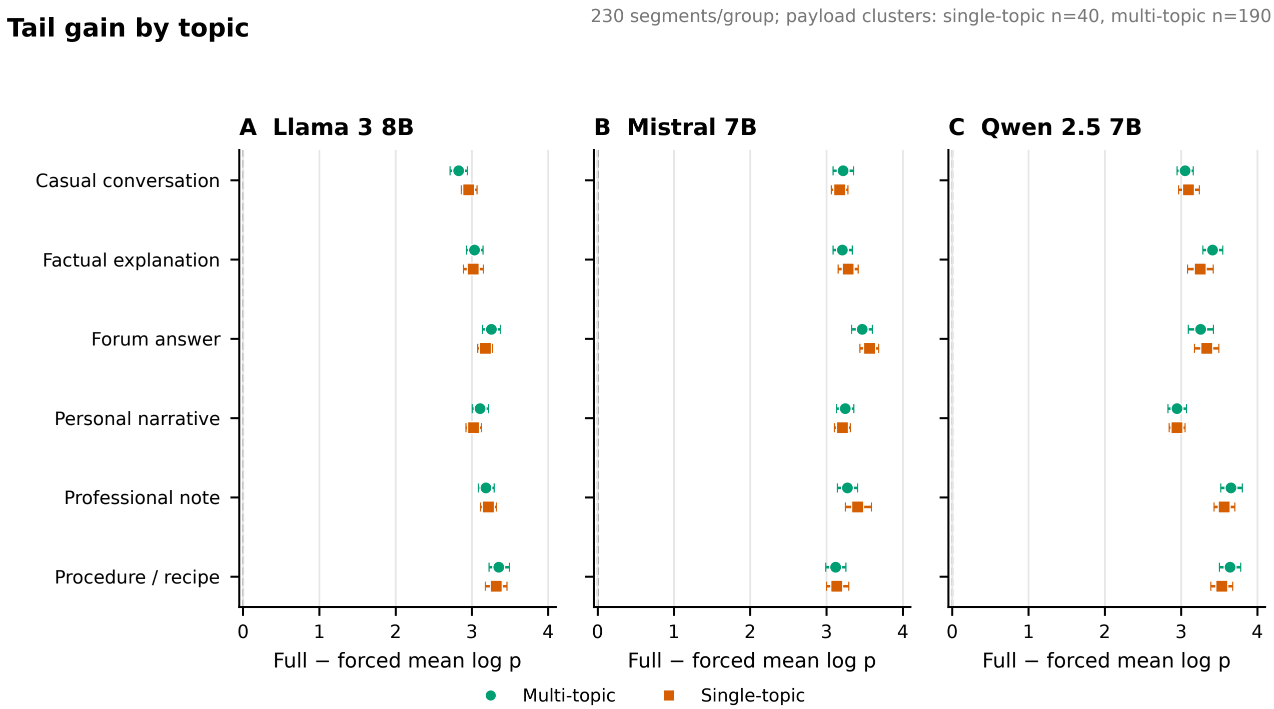


Figure S4. Exploratory tail gain by prompt category. Tail gain is full-message mean token log probability minus forced-span mean token log probability. The 8,280 paired segments arise from 1,440 segmented trials across three models, six prompt categories, and two topic schedules, yielding 36 displayed groups of 230 segments each. All group means are positive and range from +2.825721 to +3.652576. Uncertainty uses 2,000-resample payload-cluster bootstrapping stratified by payload class; independent payload-cluster counts are 40 for single-topic groups and 190 for multi-topic groups. This exploratory subgroup estimand is not the adjusted schedule contrast: adjusted source-quality contrasts range from -0.02214 to +0.01881, with Holm-adjusted $p = 1.0$ for all three models.

Table S11. All 15 prompt-topic contrasts.

Outcome	Model	Estimate	95% CI	p_{Holm}
Primary artifact count	Llama	1.276361	[1.114743, 1.461409]	0.00329358
Primary artifact count	Mistral	1.015189	[0.848773, 1.214233]	1
Primary artifact count	Qwen	0.957147	[0.781516, 1.172247]	1
Source-model log probability	Llama	-0.022140	[-0.118718, 0.074439]	1
Source-model log probability	Mistral	0.016347	[-0.080231, 0.112925]	1
Source-model log probability	Qwen	0.018813	[-0.077766, 0.115391]	1
Effective artifact rate	Llama	0.012473	[-0.075124, 0.100070]	1
Effective artifact rate	Mistral	0.006792	[-0.080805, 0.094389]	1
Effective artifact rate	Qwen	-0.015822	[-0.103419, 0.071775]	1
Held-out evaluator log probability	Llama	-0.012801	[-0.129427, 0.103825]	1
Held-out evaluator log probability	Mistral	0.012046	[-0.104580, 0.128671]	1
Held-out evaluator log probability	Qwen	0.024711	[-0.091915, 0.141336]	1
Payload throughput	Llama	0.018304	[-0.016147, 0.052755]	0.595439
Payload throughput	Mistral	0.026471	[-0.007981, 0.060922]	0.396248
Payload throughput	Qwen	-0.009378	[-0.043829, 0.025073]	0.595439

Supplementary Note S7: ~~Neural~~ Historical V2 neural steganalysis and leakage sensitivity

Corpus, architectures, and splits

~~The frozen corpus contains~~ This note preserves the historical V2 detector benchmark for continuity and work-unit provenance; its 15,~~840 rows,~~ 840-row splits and results are not the current detector estimates and are superseded by the strictly deduplicated V3 analysis in Supplementary Notes S11–S12. The historical corpus contains 7,920 RankCloak covers and 7,920 matched ordinary controls, grouped by 480 payloads. Payload groups, rather than rows, were assigned to train, validation, and test sets. The TextCNN-equivalent implementation lowercases and tokenizes text into a 30,000-term vocabulary, truncates or pads to 256 tokens, uses 300-dimensional learned embeddings and 100 filters at each of widths 3 and 5, applies dropout 0.5, and optimizes with Adam at learning rate 0.001. The pretrained DeBERTa-v3-base classifier uses maximum length 256, learning rate 2×10^{-5}, weight decay 0.01, and three epochs. For each architecture, the frozen design comprises one matched split, 18 held-out-template splits, three leave-one-model splits, and six leave-one-codec splits. Thus 28 fits per architecture and 56/56 fits overall completed, producing 55,440 prediction rows. No row-level reassignment was used to improve performance.

~~Primary reported endpoints are~~ For this historical benchmark, the reported endpoints were ROC-AUC and balanced accuracy. Precision, Brier score, and true-positive rate at false-positive rate at most 1% ~~are explicitly~~ were supplementary/exploratory post-freeze measures. All confidence intervals for an individual split use 2,000 payload-group bootstrap resamples. Ranges across heterogeneous held-out splits are descriptive minima and maxima, not confidence intervals. Every historical split-level estimate and interval ~~for all retained metrics is supplied~~ remains in the machine-readable supplementary CSV (`supplementary_tables/detector_split_metrics.csv`).

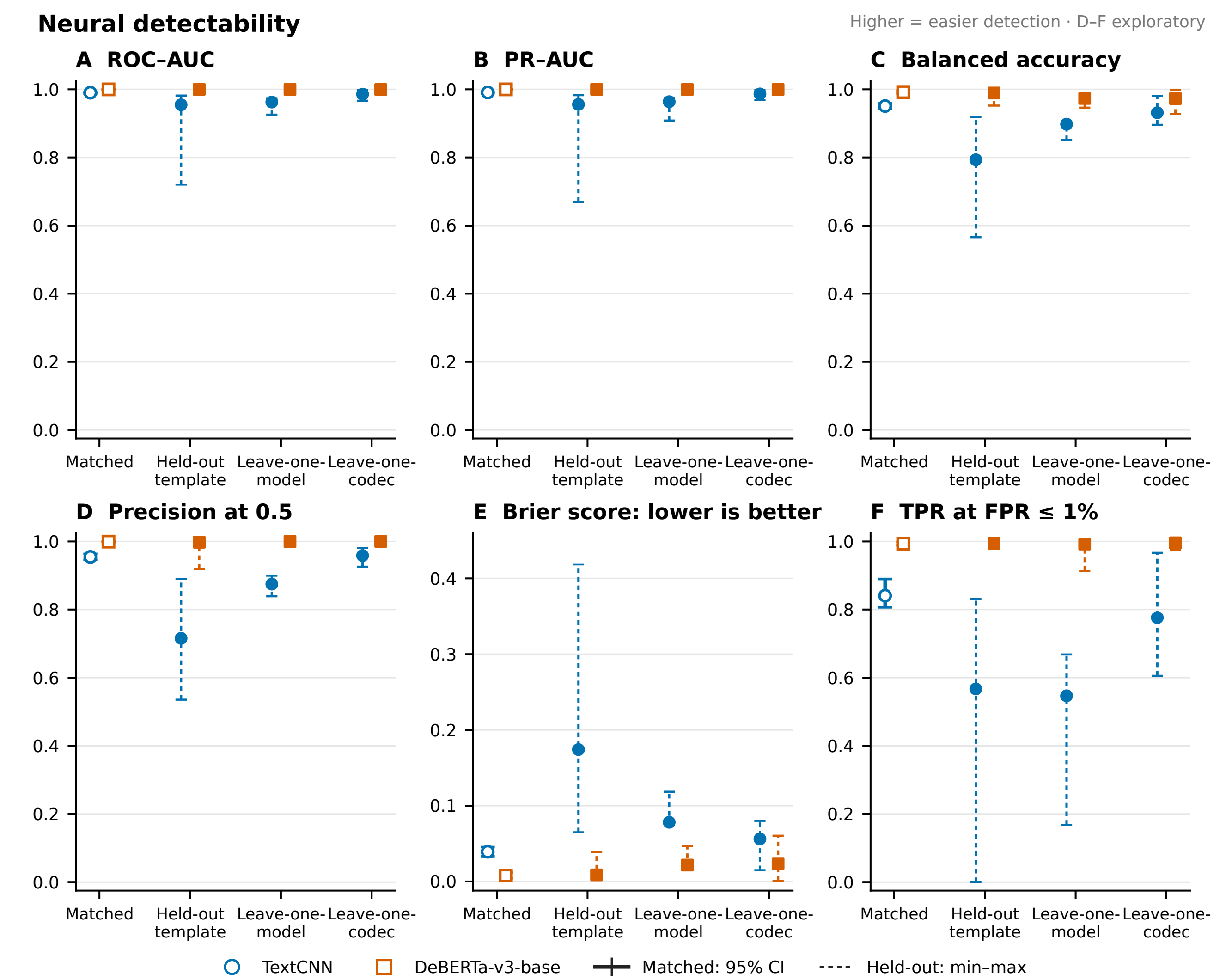


Figure S5. ~~**Full neural-detector evaluation.**~~ **Historical V2 neural-detector evaluation.** The six panels ~~report confirmatory~~ preserve the earlier frozen endpoints ~~together with~~ and supplementary exploratory metrics; current strictly deduplicated estimates are in Supplementary Note S12. Matched error bars are payload-group bootstrap 95% confidence intervals. Held-out regime bars span the minimum and maximum across prespecified splits and are not confidence intervals. Lower Brier scores are better; high values in the other panels indicate stronger distinguishability and therefore weaker concealment.

Architecture	Matched metric	Estimate	Payload-bootstrap 95% CI	Status
TextCNN-equivalent	ROC-AUC	0.990190	[0.987740, 0.992682]	~~Confirmatory~~ Historical V2 frozen
TextCNN-equivalent	Balanced accuracy	0.951010	[0.943456, 0.958466]	~~Confirmatory~~ Historical V2 frozen
TextCNN-equivalent	Precision	0.954684	[0.945891, 0.963421]	Supplementary exploratory
TextCNN-equivalent	Brier score	0.039320	[0.033497, 0.045453]	Supplementary exploratory
TextCNN-equivalent	TPR at FPR $\leq 1\%$	0.840909	[0.806517, 0.889752]	Supplementary exploratory
DeBERTa-v3-base	ROC-AUC	0.999834	[0.999696, 0.999939]	~~Confirmatory~~ Historical V2 frozen
DeBERTa-v3-base	Balanced accuracy	0.991919	[0.987654, 0.995665]	~~Confirmatory~~ Historical V2 frozen
DeBERTa-v3-base	Precision	0.999487	[0.998393, 1.000000]	Supplementary exploratory
DeBERTa-v3-base	Brier score	0.007895	[0.004432, 0.011787]	Supplementary exploratory
DeBERTa-v3-base	TPR at FPR $\leq 1\%$	0.993434	[0.988724, 0.998387]	Supplementary exploratory

Table S12. ~~Matched~~ Historical V2 matched detector endpoints, superseded for current claims by Supplementary Note S12. Higher ROC-AUC, balanced accuracy, precision, and low-FPR sensitivity are adverse for concealment; lower Brier score indicates better detector calibration.

Architecture	Regime (split count)	ROC-AUC range	Balanced-accuracy range
TextCNN-equivalent	Held-out template (18)	[0.720421, 0.981705]	[0.565909, 0.919318]
TextCNN-equivalent	Leave one model (3)	[0.925623, 0.973650]	[0.850758, 0.883333]
TextCNN-equivalent	Leave one codec (6)	[0.966532, 0.998634]	[0.895833, 0.980556]
DeBERTa-v3-base	Held-out template (18)	[0.996291, 0.999979]	[0.952273, 0.995455]
DeBERTa-v3-base	Leave one model (3)	[0.996772, 0.999979]	[0.945455, 0.970455]
DeBERTa-v3-base	Leave one codec (6)	[0.997917, 1.000000]	[0.927778, 0.998611]

Table S13. ~~Ranges~~ Historical V2 ranges across heterogeneous frozen held-out splits. These ranges are not pooled confidence intervals; current leave-one-family-out results are in Supplementary Note S12.

Leakage, exclusion sensitivity, repeatability, and cost

~~Exact~~ In the historical V2 benchmark, exact train/test visible-text identity and exact tokenizer-sequence identity checks passed in all 28 frozen split definitions. A separate normalized character n-gram diagnostic ~~at its declared threshold~~ found 53 train/test near-duplicate pairs, of which 44 crossed payload groups, affecting 16 splits. Excluding affected test payload groups without retraining changed ROC-AUC from -0.001925 to +0.004568 for TextCNN-equivalent and from -0.000038 to +0.000166 for DeBERTa-v3-base~~; other observed endpoint shifts were also small. This threshold-dependent sensitivity does~~. Because that sensitivity did not rebuild training data~~and therefore does not eliminate the design limitation~~, it motivated and is superseded by the pre-feature component construction and zero-linkage audit in Supplementary Note S11.

Fixed-seed reruns on the same CUDA device produced exact metric equality for both architectures. This is evidence of same-device repeatability only. CPU neural training was infeasible in the retained run and no CPU result was substituted; the study makes no CPU/GPU equivalence claim. The representative CUDA benchmark recorded TextCNN wall/training times 91.358113/9.497936 seconds and peak allocated VRAM 306,247,680 bytes; DeBERTa values were 1,536.660910/1,425.063414 seconds and 6,380,851,200 bytes.

Supplementary Note S8: Capacity and quality theory

The main Methods defines bounded forced length, nominal and effective rates, full-message overhead, and the same-context probability bound. This note gives finite-code consequences, proof conditions, and the retained-field audit without repeating those central equations.

Finite code space and exact inversion

Let H, B, and n_B have the meanings in Main Eq. (14). The unused capacity of the smallest fixed-length rank word is described by

$$\sigma_B = n_B \log_2 B - H, \qquad \eta_B = \frac{2^H}{B^{n_B}} = 2^{-\sigma_B}, \qquad u_B = 1 - \eta_B.$$

Here σ_B is code-space slack in bits, η_B is utilization, and u_B is the unused codeword fraction. For a power-of-two alphabet and an integer-length bitstream, σ_B equals literal terminal padding bits. The byte codec records those bits explicitly; for $B = 8$, one byte is the finite example $n_B = 3$ with one terminal padding bit, while a multi-byte payload is packed as one continuous stream. Hex nibbles have zero slack when the represented source length is exactly four bits per canonical character. The rank-bound proposition assumes that every forced history has at least B eligible candidates and that encoder and decoder reproduce the same model, token IDs, prompt context, mask, and ordering. A digit $d \in \{0, \ldots, B-1\}$ selects the token at filtered rank $d+1$; exact replay ranks that same saved token as $d+1$. Serial induction over the forced span recovers the rank word, and injectivity of the codec recovers the representation. The proposition does not apply after an altered token, context, boundary, mask, or model, and it supplies no edit-correction property.

Segmentation and overhead decomposition

For segment forced lengths s_j, lead-in lengths ℓ_j, and tail lengths t_j, the saved records satisfy $\sum_j s_j = n_B$. The empirically audited residual can be decomposed as

$$D = N_{\text{full}} - n_B = \sum_{j=1}^{J} (\ell_j + t_j) + \delta, \qquad \delta \geq 0.$$

Thus segmentation alone preserves forced-position count, while more resets can create more opportunities for non-payload continuations. A no-tail condition raises bits per full token by shrinking the denominator. This is an arithmetic consequence and not evidence that the resulting visible text is more readable or natural.

Conditional likelihood gap

The per-history ordering in Main Eq. (16) permits a trace-level comparison with the rank-1 eligible token and, for bounded codecs, the rank-B token. In addition to Q_B, the audit records

$$\Delta_B = Q_B - Q_{\text{greedy}} = \frac{1}{n_B} \sum_{i=1}^{n_B} \left[-\log p_M(y_i \mid h_i) + \log p_{(1)}(h_i) \right] \geq 0.$$

The upper rank-B endpoint is evaluated only when its probability was retained under the identical filtered history. Direct subword traces have no single fixed B, so that endpoint is unavailable rather than imputed. These quantities are model-scale surprisals in nats per forced token, not linguistic or participant outcomes.

Empirical checks and trace boundary

The validation inventory contained 17,424 eligible retained records. Of these, 7,008 had all fields required for complete capacity, rate, and supported quality-bound checks. Across evaluable traces there were zero forced-position-count violations, zero rank/rate-bound violations, and zero supported quality-bound violations. The nine aggregate cells in the theoretical capacity panel of Supplementary Fig. S6 lay exactly on the identity between theoretical and observed forced positions. There were 2,976 positive cover-length residuals, attributable to natural tails or other counted cover extension.

The complete encoder decoder cascade proposition is not directly trace-evaluable from the saved record structure: none of the retained rows simultaneously carries every intermediate object required to replay the entire proof chain as one trace. The algebraic results are therefore conditional proofs and the retained-field checks are supporting empirical validation, not a complete end-to-end trace proof. They also do not validate human naturalness.

This theoretical validation asks whether the retained fields satisfy the declared fixed-radix identities, rate bounds, and supported same-context likelihood bound. It is distinct from the empirical capacity and quality frontier in Main Fig. 2, which compares observed representation rates and automated model/surface diagnostics on common payload support.

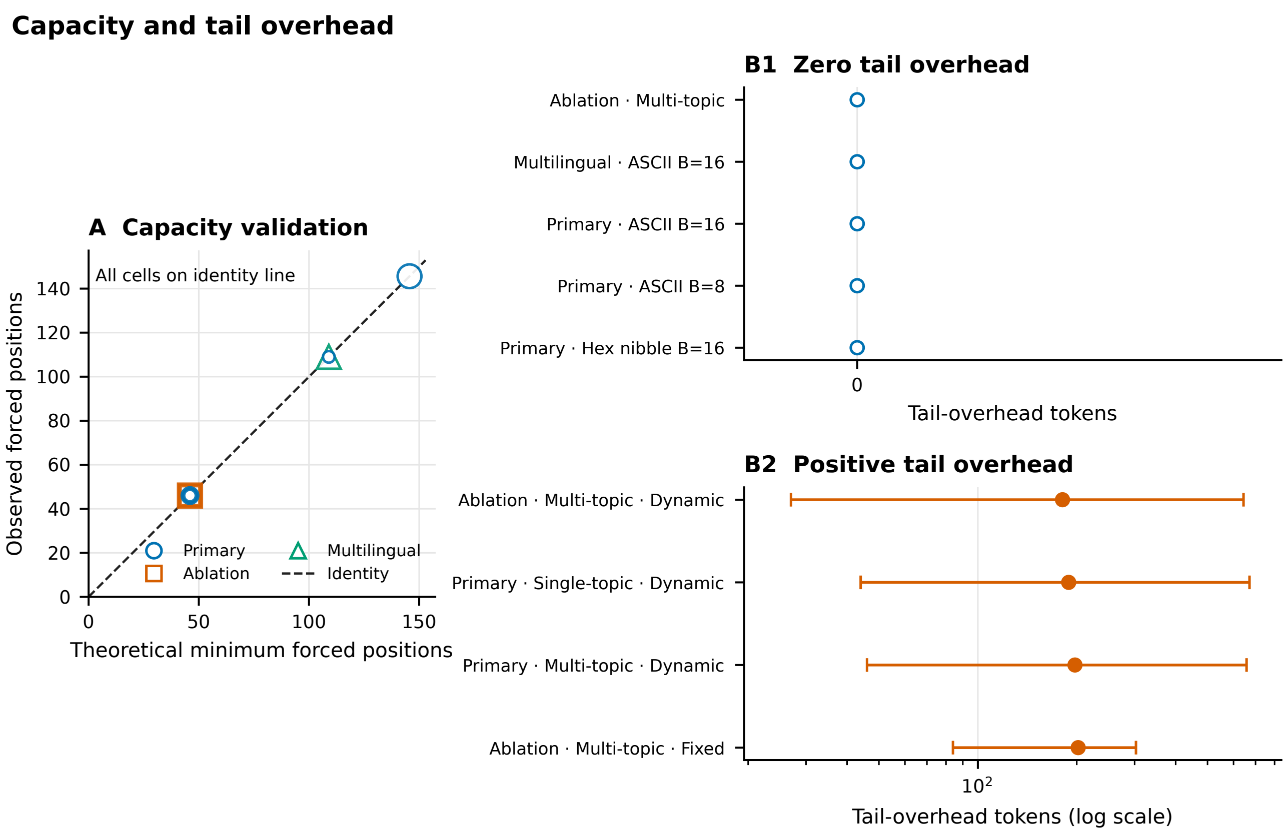


Figure S6. **Theoretical capacity and tail validation.** Fixed-radix payload length and effective full-message rate are checked against their declared algebraic bounds, while the cover-length residual isolates tokens beyond the forced payload positions. Across 17,424 retained records, 7,008 were evaluable for the complete plotted capacity and supported quality-bound checks; no forced-position, rate-bound, or evaluated quality-bound violation occurred. Positive residuals in 2,976 records represent natural tails or other cover overhead. This validation tests retained-field identities and differs from the empirical common-support frontier in Main Fig. 2; neither analysis is evidence of human-perceived naturalness.

Validation item	Result	Interpretation
Eligible records	17,424	Broad retained validation inventory
Fully evaluable records	7,008	All required trace fields present
Forced-position violations	0	Observed count matched codec requirement
Rank/rate violations	0	Evaluated bounded-codec relations held
Supported quality-bound violations	0	Model-scale trace bound held where evaluable
Positive cover-length residuals	2,976	Tail/cover overhead, not error
Complete cascade traces	0	End-to-end proposition not directly trace-evaluable

Table S14. Empirical capacity and traceability audit.

Supplementary Note S9: Computational overhead

Primary-stage timing used 12,144 matched payload rows across the 18 model and protocol cells in Table S15. Intervals are 2,000-resample payload-group bootstrap intervals. Generation time is retained inclusive wrapper wall time; representation, filter setup, model generation, and supported recovery fields are reported separately in the sealed tables but are not claimed to be perfectly isolated kernel timings. Encoding wrapper time and supported decoding measurements consequently should not be summed as independent hardware primitives.

Across the 18 cells, mean inclusive generation time ranged from 1.674368 to 16.087628 seconds and payload throughput from 13.631855 to 100.851772 bits/s; effective payload rate ranged from 0.710004 to 7.322003 bits per full token. Direct-rank and hex-nibble protocols were generally faster than the byte-expanded conditions, while segmented protocols were generally slower because they repeated prompt and tail work. These retained inclusive wall-time measurements support computational accounting under the recorded hardware and harness, not universal hardware-performance claims.

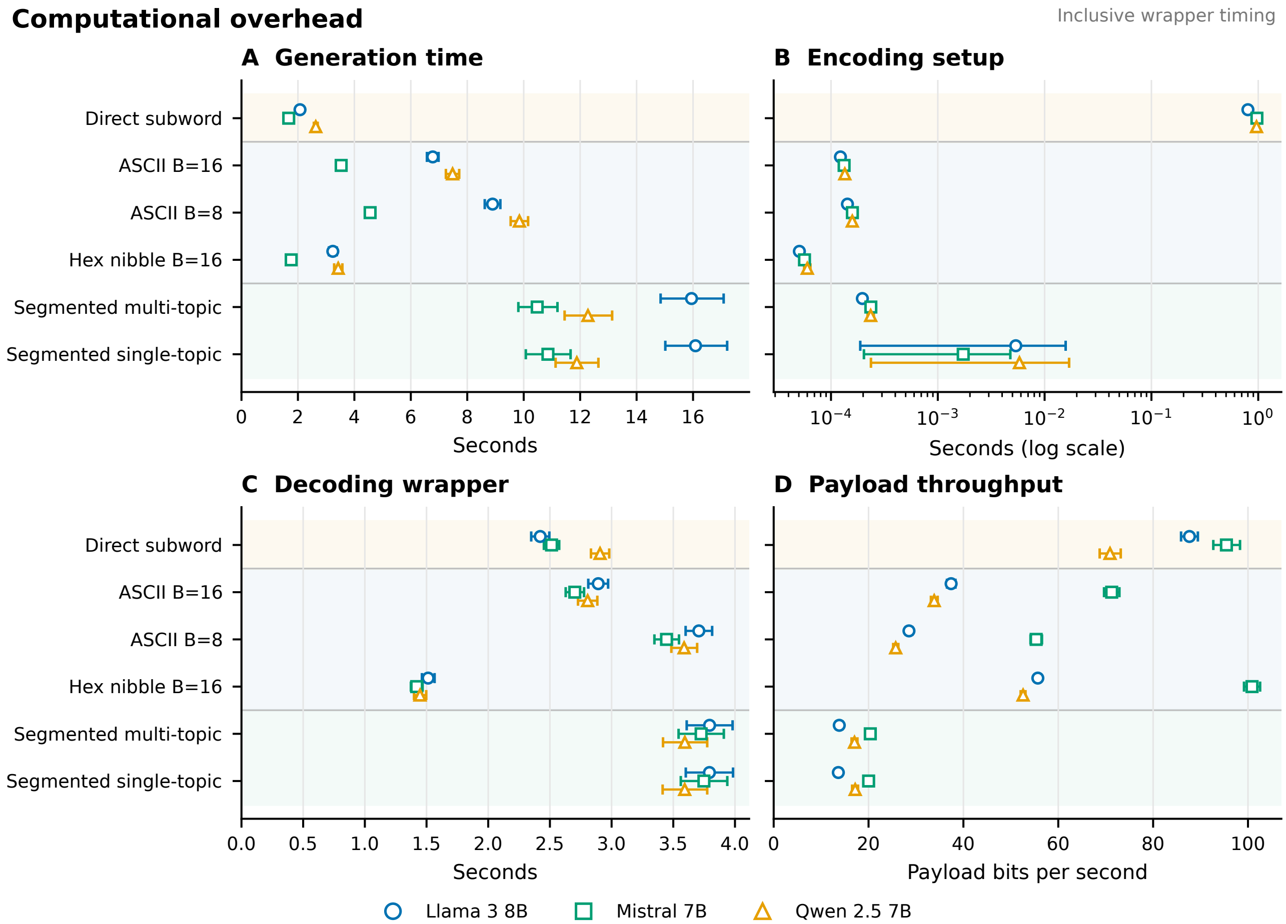


Figure S7. Complete computational overhead summary. Primary model and protocol cells show payload-bootstrap summaries for inclusive generation, encoding setup, supported recovery, throughput, and effective rate. The setup panel uses a logarithmic axis because cells span several orders of magnitude. CPU time and repeated warm-up measurements were unavailable and wall time is not relabeled as CPU time.

Model	Protocol	n	Mean generation time, s (95% CI)	Mean payload throughput, bit/s (95% CI)
Llama	Direct ranks	480	2.074349 [2.014282, 2.135219]	87.658980 [85.892805, 89.433262]
	ASCII-16	480	6.776682 [6.572039, 6.982913]	37.409987 [36.542507, 38.322129]
	ASCII-8	480	8.896445 [8.619910, 9.172562]	28.522326 [27.867278, 29.213688]
	Hex-16	240	3.236699 [3.099398, 3.376963]	55.691381 [55.148301, 56.225745]
	Segmented multi-topic	240	15.946543 [14.850856, 17.084494]	13.827671 [13.155163, 14.496873]
	Segmented single-topic	240	16.087628 [15.017670, 17.208756]	13.631855 [12.954794, 14.325096]
Qwen	Direct ranks	480	2.632108 [2.565827, 2.699101]	70.911677 [68.729360, 73.171915]
	ASCII-16	480	7.480319 [7.242905, 7.714463]	33.843456 [33.153343, 34.571271]
	ASCII-8	480	9.847052 [9.537974, 10.155572]	25.724469 [25.198423, 26.278772]
	Hex-16	240	3.427555 [3.284745, 3.575041]	52.584782 [52.074452, 53.086870]
	Segmented multi-topic	240	12.276814 [11.449256, 13.133478]	17.040206 [16.420206, 17.655385]
	Segmented single-topic	240	11.881962 [11.129651, 12.646529]	17.182540 [16.534412, 17.783251]
Mistral	Direct ranks	480	1.674368 [1.637117, 1.711610]	95.454287 [92.689931, 98.318820]
	ASCII-16	480	3.532546 [3.430360, 3.637370]	71.264443 [69.655729, 72.892828]
	ASCII-8	480	4.558496 [4.421249, 4.699172]	55.340577 [54.149134, 56.561483]
	Hex-16	240	1.762009 [1.698523, 1.826194]	100.851772 [99.168287, 102.554774]
	Segmented multi-topic	240	10.478402 [9.810233, 11.196055]	20.368846 [19.484536, 21.270824]
	Segmented single-topic	240	10.858662 [10.074554, 11.660671]	20.014386 [19.095928, 20.969663]

Table S15. Inclusive generation time and payload throughput by model and protocol. Llama denotes Meta-Llama-3-8B-Instruct, Qwen denotes Qwen2.5-7B-Instruct, and Mistral denotes Mistral-7B-Instruct-v0.3; all use the reported Q4_K_M configurations. Brackets contain payload-bootstrap 95% confidence intervals.

Detector	Wall time (s)	Training time (s)	Peak allocated VRAM	Scope
TextCNN-equivalent	91.358113	9.497936	306,247,680 bytes	One fixed CUDA benchmark
DeBERTa-v3-base	1,536.660910	1,425.063414	6,380,851,200 bytes	One fixed CUDA benchmark

Table S16. Representative detector cost. CPU time, repeat warm-up measurements, and cross-device benchmarks were unavailable.

Supplementary Note S10: Worked examples and claim-boundary audit

Representative retained examples

Payload to ranks to cover. A retained Llama segmented-hex trial used the public SHA-256 test-vector serialization

`7cec7f1e68bbef4fdf066a51ccbea4377ee369e1b53219743aa386615e12d44a`.

The first eight encoded ranks were $[8, 13, 15, 13, 8, 16, 2, 15]$. Under the saved prompt/model/filter state, the associated forced token IDs were $[1789, 198, 16, 627, 13617, 10652, 25, 9842]$, detokenizing to the deliberately mechanical span

```
For\n1.\nExample conversation:  Write
```

The subsequent visible text

```
a friendly message to a friend about making relaxed plans for the weekend.
```

was a natural tail. It carried no payload rank and was ignored by the decoder. Saved-token replay reproduced all nibble ranks and the serialized digest exactly. This example illustrates deterministic rank realization, not an assertion that the forced span is human-natural.

Visible-text retokenization failure. In a retained ablation trial, detokenization followed by tokenizer reapplication first diverged in segment 6, forced position 5, expected token ID 11,623 at rank 8 became observed token ID 53,992 at replay rank 2. The visible string alone therefore did not preserve the token boundary needed by the saved-token contract.

Transformation failure. For the SHA-256 example above, straight-to-smart quote conversion failed at segment 1, position 1: expected token ID 1 at rank 9, while the transformed replay observed token ID 2,118 at rank 155. The first-divergence record localizes the mismatch but does not prove that a single changed glyph alone caused every later error.

Claim-boundary audit

Table S17. Claim boundaries applied throughout the revision.

Issue	Supported statement	Prohibited or unresolved extension
Exact-copy recovery	6,480/6,480 primary serialized payloads recovered from saved token IDs under matched replay	Ordinary visible-text reliability, cross-model robustness
Transmission	Controlled frozen perturbations reveal severe fragility; tail-only truncation preserves recovery when only ignored tokens are removed	Broad edit robustness or general mitigation
Human evaluation	504 computational stimuli have automated surface/model diagnostics	Human-perceived readability or naturalness; zero participants and ratings
Deployment	No operational channel or deployment traffic was tested	Practical covert communication or real-world validation
Detectability	Two ~~neural architectures~~ data-adaptive text-only detectors and an exact-model-aware attacker strongly distinguish evaluated covers; high AUC is adverse	Undetectability or resistance to future detectors
Multilingual scope	Exact-copy recovery in 288/288 Spanish and 288/288 Simplified Chinese ~~exact-copy secondary trials recovered~~ secondary trials	Broad multilingual naturalness or cross-language generalization
Unavailable work	432 predeclared combinations lacked compatible retained outputs	Calling unavailable combinations experimental failures
Model replay	Retained outputs are bound to model/configuration manifests	Fresh local generation replay while configured GGUF files are absent
Detector leakage	~~Exact identity checks passed; near-duplicate sensitivity shifts were small~~ Strict pre-feature component split has zero audited cross-partition links	~~Elimination of the 53-pair/16-split lexical-overlap limitation~~ Semantic paraphrases below the declared 0.95 threshold are not excluded
Statistical models	Reported conditional estimates retain diagnostic warnings	Treating three singular mixed-model fits as unqualified evidence; substituting an undeclared fallback
Exploratory analyses	Compact ablation, topic, and auxiliary detector analyses are labeled post-outcome/exploratory where applicable	Recasting them as preregistered confirmatory evidence
Overhead	Retained inclusive wall-time and fixed CUDA benchmark scopes are reported	CPU time, warm-up stability, CPU/GPU equivalence, or hardware-general performance
Theory	Conditional rank/capacity relations and retained-field checks are supported	A directly trace-evaluated full cascade or human-quality proof

Issue	Supported statement	Prohibited or unresolved extension
Cryptography	The carrier can encode outputs from declared cryptographic algorithms	Cryptographic security supplied by RankCloak itself; confidentiality without an external cryptosystem

Supplementary Note S11: Strict detector splits and human-authored secondary controls

Normalization, deduplication, and partition audit

All V3 detector corpora were deduplicated before feature extraction or tokenization. Visible text underwent Unicode NFKC normalization, case folding, Unicode-whitespace collapse, and SHA-256 hashing. If a normalized exact duplicate implicated either member of a RankCloak/control pair, the complete matched pair was removed so that class balance and pairing were preserved. Near duplicates were defined before splitting as cosine similarity at least 0.95 under L2-normalized character-boundary TF-IDF with 3–5 character n-grams and minimum document frequency two. Payload groups connected by any such edge formed immutable components.

One normalized exact-duplicate group implicated two rows in the 15,840-row source corpus. Removing that complete matched pair left 15,838 observations. The remaining audit found 52 near-duplicate pairs, including 44 that crossed payload groups. A deterministic seed-20260831 balancing procedure assigned whole connected components to train, validation, and test. The final audit found zero payload, matched-pair, normalized-text, component, or declared near-duplicate links between partitions (Table S18). The statement of zero leakage is limited to these declared identities and thresholded edges; it cannot rule out semantic paraphrases below the threshold or guarantee real-world generalization.

Audit item	Count	Interpretation
Source observations	15,840	Balanced RankCloak and matched ordinary-model controls
Normalized exact groups	1	One complete matched pair removed, two rows total
Retained observations	15,838	9,904 train; 2,952 validation; 2,982 test
Declared near-duplicate pairs	52	44 cross-payload; all connected groups kept in one partition
Audited cross-partition links	0	Zero for payload, pair, normalized text, component, and declared near duplicate

Table S18. Strict detector-corpus construction. Counts are from the locked V3 deduplication audit. Zero audited leakage applies only under the stated normalization and similarity rule.

Human-authored control selection and scope

The secondary controls came from Databricks Dolly 15k v1.0, revision `bdd27f4d94b9c1f951818a7da7fd7aeea5dbff1a`, under CC BY-SA 3.0. Deterministic screening retained responses without automatic quality, personally identifiable information, or unsafe-content flags. The same exact and near-duplicate rules were applied within the human-authored candidates and against the primary corpus. Hungarian assignment matched candidates to RankCloak test texts within available prompt templates using relative word-count plus weighted character-count difference; relative word-count differences above 0.35 were rejected.

The pipeline selected 250 texts spanning 16 of 18 templates. It found no normalized exact duplicates, no within-human near-duplicate pairs, and no declared near-duplicate links to the primary corpus. Human labels never entered model fitting or threshold selection. The corpus is a bounded secondary computational control, not recruitment, a participant study, a user study, a human rating exercise, or proof of naturalness. Coverage is sparse or absent for several conversational, narrative, and professional templates, and detector performance can include generic machine-versus-human differences.

Human-control item	Result	Scope
Selected responses	250	Frozen test negatives from Dolly; 16 templates represented
Within-human exact links	0	Under NFKC/casefold/whitespace normalization
Within-human near links	0	Under the declared 0.95 character-n-gram rule
Human-to-primary near links	0	Same declared rule
Labels used for fitting/thresholds	No	Thresholds came from matched ordinary-model validation controls
Empirical 0.1% FPR	Unavailable	Only 250 human negatives; at least 1,000 required

Table S19. Human-authored secondary-control design and audit.

Supplementary Note S12: Deduplicated, unseen-family, model-aware, and low-FPR detection

The text-only attackers reuse the fixed V1 TextCNN and pinned `microsoft/deberta-v3-base` revision `8ccc9b6f36199bec6961081d44eb72fb3f7353f3`. Their architectures and hyperparameters were not tuned on V3 test data. They are data-adaptive black-box attackers because they are fitted specifically to distinguish RankCloak from matched ordinary-model output. The model-aware logistic attacker instead receives saved token log probabilities from the exact generation model and uses summary surprisal features. Its L2 regularization value was chosen from 0.01, 0.1, 1, and 10 by validation ROC-AUC. Rank-only fields were excluded because ordinary-control ranks were not stored.

Matched fitting used 9,904 training and 2,952 validation observations; the test had 1,491 positives and 1,491 negatives. For each leave-one-family-out fit, the target family was absent from both training and validation, and evaluation used only its 497 positive and 497 negative observations in held test components. Table S20 reports the adverse matched results and the range across the three genuine family holdouts.

Detector	Matched ROC-AUC (95% CI)	Matched normalized 0–1% pAUC	Leave-one-family-out ROC-AUC range
TextCNN	0.9868 [0.9819, 0.9913]	0.6652	[0.9186, 0.9784]
DeBERTa-v3-base	0.9994 [0.9984, 1.0000]	0.9948	[0.9972, 1.0000]
Model-aware surprisal	0.999995 [0.999970, 1.000000]	0.999460	[0.999943, 1.000000]

Table S20. Detection on strictly deduplicated components. The matched test has 1,491 observations per class. Ranges summarize three distinct held-family tests and are not pooled confidence intervals. Higher values are adverse for concealment.

ROC-AUC used tie-aware empirical ranks. The normalized 0–1% partial AUC was the exact empirical false-positive-budget step envelope divided by 0.01; no score or ROC vertex was interpolated. At target α, the validation threshold maximized TPR subject to no more than $\lfloor \alpha n_{\text{neg}} \rfloor$ false positives, with conservative tie breaking. The threshold was then frozen for test. A target was reported only if validation and test both contained at least $1/\alpha$ negatives. Confidence intervals used 2,000 percentile bootstrap resamples of complete deduplication clusters.

Detector	Evaluation	Val. negatives	Test negatives	Test TPR/FPR at frozen 1% threshold	Exact false positives	Frozen 0.1% result
TextCNN	Matched	1,476	1,491	0.8317/0.0127	19/1,491	TPR 0.4829; FPR 0.0013; 2/1,491 FP
DeBERTa	Matched	1,476	1,491	0.9913/0.0007	1/1,491	TPR 0.9886; FPR 0.0007; 1/1,491 FP
Model-aware	Matched	1,476	1,491	0.9987/0.0020	3/1,491	TPR 0.9987; FPR 0.0020; 3/1,491 FP
TextCNN	Held family range	984	497	TPR 0.7103–0.8330; FPR 0.0201–0.0704	10–35/497	Unavailable: fewer than 1,000 negatives
DeBERTa	Held family range	984	497	TPR 0.9759–1.0000; FPR 0.0040–0.0563	2–28/497	Unavailable: fewer than 1,000 negatives
Model-aware	Held family range	984	497	TPR 0.9920–1.0000; FPR 0.0000–0.0060	0–3/497	Unavailable: fewer than 1,000 negatives

Table S21. Validation-frozen low-FPR operating points. The observed test FPR can differ from the validation target. Exact counts, rather than interpolated rates, expose calibration drift and empirical resolution.

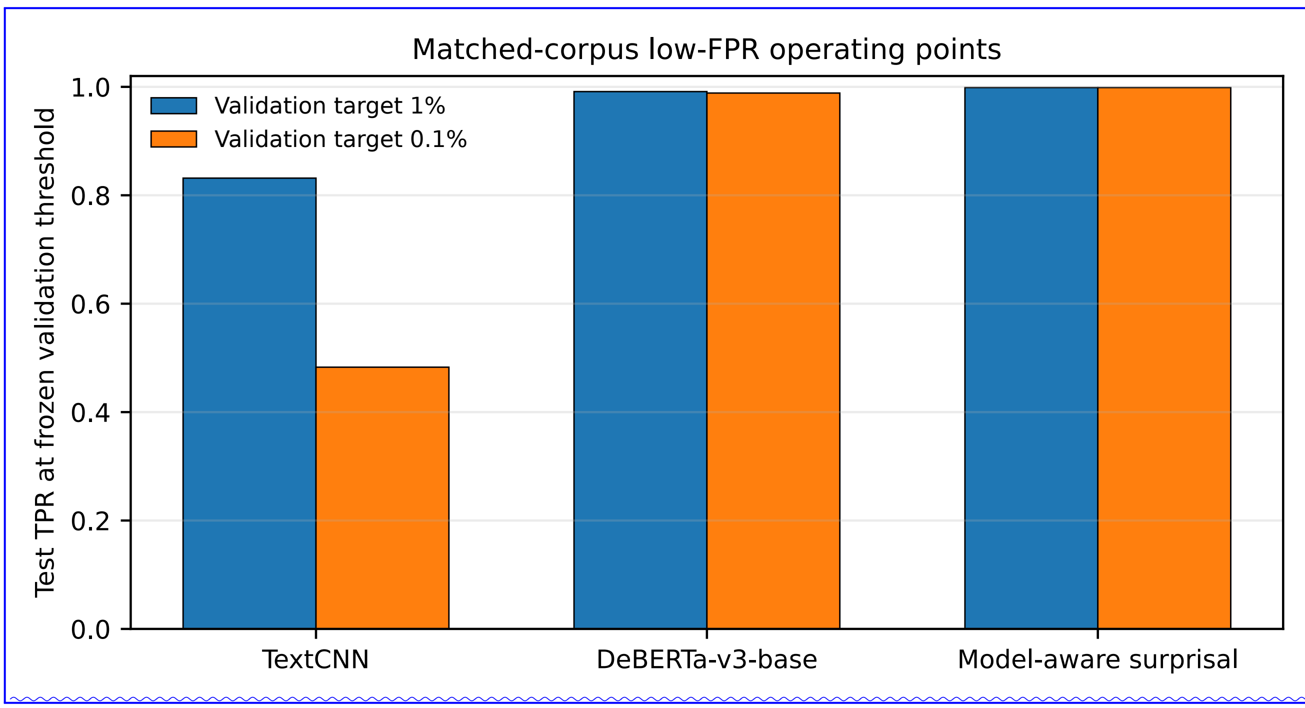


Figure S8. Low-FPR detection under frozen thresholds. Thresholds were selected on validation and applied once to test. Missing 0.1% family-holdout results reflect insufficient negative counts, not zero sensitivity.

Against the human-authored secondary controls, TextCNN ROC-AUC was 0.7537 (95% CI [0.7141, 0.7918]) and DeBERTa ROC-AUC was 0.9846 (95% CI [0.9771, 0.9910]). At the thresholds selected on ordinary-model validation controls for nominal 1% FPR, TextCNN classified 152/250 human texts as positive (FPR 0.6080; grouped-bootstrap 95% CI [0.5469, 0.6667]) and DeBERTa classified 97/250 as positive (FPR 0.3880; CI [0.3263, 0.4466]). These high false-positive rates show domain sensitivity and do not reverse the matched-control result.

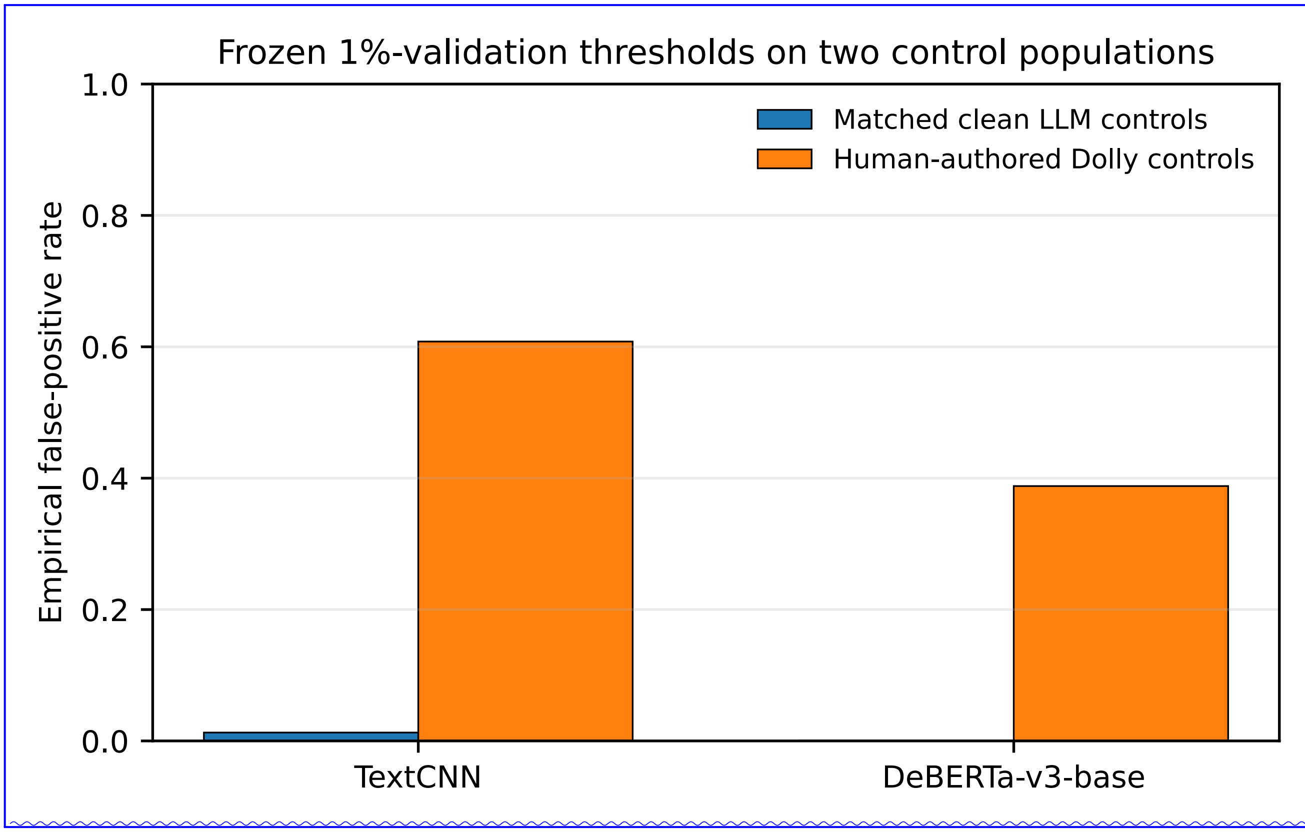


Figure S9. **Secondary human-authored control evaluation.** Human-authored labels were not used for fitting or threshold selection. The comparison may include general machine-versus-human signal and is not a human evaluation.

The two learned detectors adapt to the observed training distribution; the likelihood-based detector represents an attacker with exact model access. Their strength makes the negative detectability conclusion more credible within the evaluated corpus, but none is a comprehensive adversarial search over all classifiers, prompts, post-processing, models, or deployment distributions.

Supplementary Note S13: Entropy-gated payload embedding

Calibration and replay-consistent gate

The entropy-gated extension was informed by Cai, Ding, and Tao[1], but the objectives differ. Their watermarking detector decides whether text carries a mark; a RankCloak receiver must reconstruct an arbitrary payload. Selective watermark insertion therefore does not by itself establish payload capacity, recoverability, or concealment.

Six 128-position ordinary top-p traces per model used temperature 0.8, top-p 0.95, and the frozen serial sampler, yielding 768 development positions per model. Detector outcomes did not enter calibration. The median threshold defines the moderate gate and the 75th percentile defines the strict gate (Table S22).

Model	Traces	Median threshold, bit	75th-percentile threshold, bit
Llama 3 8B Q4_K_M	6	0.803259	1.705455
Mistral 7B Q4_K_M	6	0.966861	1.917551
Qwen 2.5 7B Q4_K_M	6	1.361919	2.432557

Table S22. Frozen entropy thresholds from ordinary-generation development traces.

Before each embedding-span step, the encoder computed filtered next-token Shannon entropy. An eligible position consumed and forced the next payload rank. An ineligible position used ordinary top-p sampling under the same allowed-token mask and did not consume a payload rank. Replay recomputed entropy before each observed token, consumed a rank only at eligible positions, ignored observed ranks at sampled skips, and appended every observed token to context. It received no saved gate-position metadata. Gate-position agreement between encoder and replay was exact in the validation audit.

Each gate contained 120 RankCloak attempts and 120 paired length-matched ordinary controls across three model families, eight artifact classes, all 20 supported artifact/representation cells, and two templates. The ungated, moderate, and strict RankCloak conditions shared one stable seed per cell; controls used a separate shared ordinary-generation seed. Fixed-token-budget capacity was evaluated at the paired ungated span length for every attempt. Fixed-payload bits per generated token was defined only when the complete payload was embedded; completion rate and fixed-budget payload fraction are therefore separate endpoints.

Gate	Complete	Budget failures	Capacity, bit/token	Capacity n	Budget payload fraction	Length ratio	Saved-ID/visible recovery
Ungated	120/120	0	7.4001, conditional	120	1.0000	1.0000	120/120; 62/120
Moderate median	120/120	0	5.7920, conditional	120	0.7518	1.3743	120/120; 65/120
Strict 75th percentile	114/120	6	4.0513, conditional	114	0.4567	2.7615	114/120; 68/120

Table S23. Entropy-gate generation, capacity, length, and recovery. Capacity is fixed-payload bits per generated token conditional on completion. In particular, strict-gate capacity excludes six incomplete cases and has $n = 114$. Completion and recovery retain all 120 attempts in their denominators.

Strict completion by model was 40/40 for Llama, 39/40 for Mistral, and 35/40 for Qwen. The corresponding strict conditional capacity estimates were 6.1322 ($n = 40$), 2.9207 ($n = 39$), and 2.9331 ($n = 35$) bits/token. Visible-text recovery under ungated/moderate/strict conditions was 29/40, 29/40, and 34/40 for Llama; 0/40 in every Mistral condition; and 33/40, 36/40, and 34/40 for Qwen. The aggregate visible-text differences therefore do not establish a general gate benefit.

Relative to ungated output, moderate gating added a mean 20.1083 words, changed unique-word fraction by -0.0338, and changed repeated-bigram fraction by $+0.0106$. Strict gating added 103.0083 words, changed unique-word fraction by -0.1431, and changed repeated-bigram fraction by $+0.0423$. These are surface heuristics, not participant judgments. At forced payload positions, mean rank was 88.0552 for ungated and moderate gates and 89.4915 for the strict gate; mean forced-token rank pressure declined from 3.8585 to 3.4276 and 3.0308 nats. Gating changes the contexts in which payload ranks are consumed but does not change the payload-rank distribution for completed paired cells.

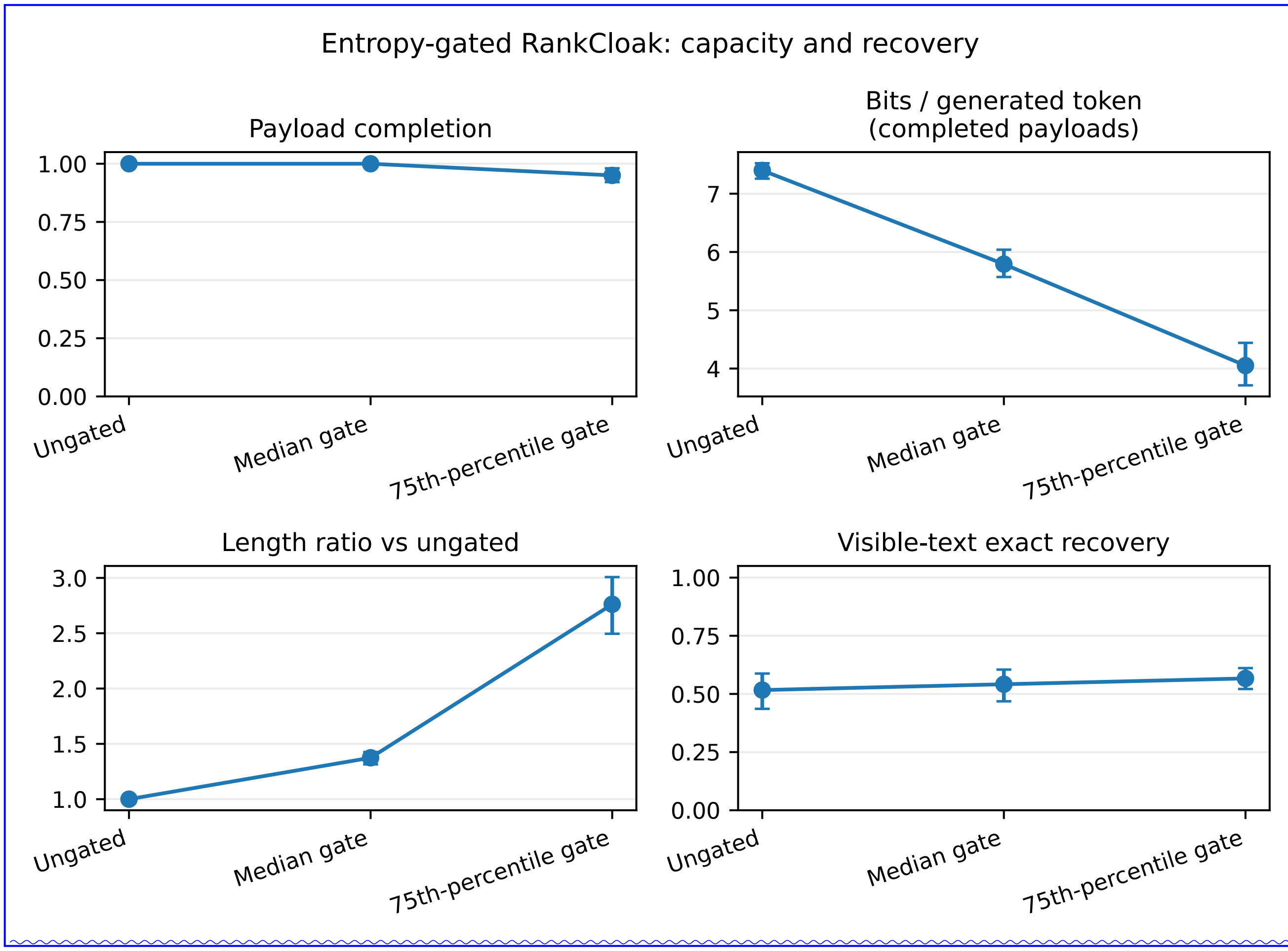


Figure S10. Entropy gating, capacity, length, completion, and recovery. Fixed-payload capacity is conditional on completion; fixed-budget payload fraction and completion retain all attempts.

The generated detector test was external to fitting. Historical train and validation rows carrying any of the 16 entropy-study payload instances were excluded, and exact/near-duplicate auditing was repeated before feature extraction. The audit retained 12,714/13,114 rows, removed 400, found zero cross-partition links, and left no condition unavailable. Gate-specific test controls numbered 117 ungated, 113 moderate, and 116 strict; none supports empirical 0.1% resolution.

Detector	Gate	Pos./neg.	ROC-AUC	0–1% pAUC	TPR/FPR at frozen 1% threshold	Exact FP
TextCNN	Ungated	117/117	0.9953	0.8536	0.8718/0.0085	1/117
TextCNN	Moderate	113/113	0.9825	0.7416	0.7965/0.0088	1/113
TextCNN	Strict	116/116	0.8850	0.4138	0.4310/0.0172	2/116
DeBERTa	Ungated	117/117	0.9999	0.9854	1.0000/0.0085	1/117
DeBERTa	Moderate	113/113	0.9785	0.6878	0.8496/0.0177	2/113
DeBERTa	Strict	116/116	0.8893	0.3484	0.4914/0.0259	3/116
Model-aware	Ungated	117/117	1.0000	1.0000	1.0000/0.0000	0/117
Model-aware	Moderate	113/113	0.9999	0.9922	0.9381/0.0000	0/113
Model-aware	Strict	116/116	0.9904	0.7845	0.6552/0.0000	0/116

Table S24. Detector behavior by entropy gate. Thresholds are the single historical-validation-frozen nominal 1% thresholds. Every gate has fewer than 1,000 test negatives, so empirical 0.1% TPR is unavailable.

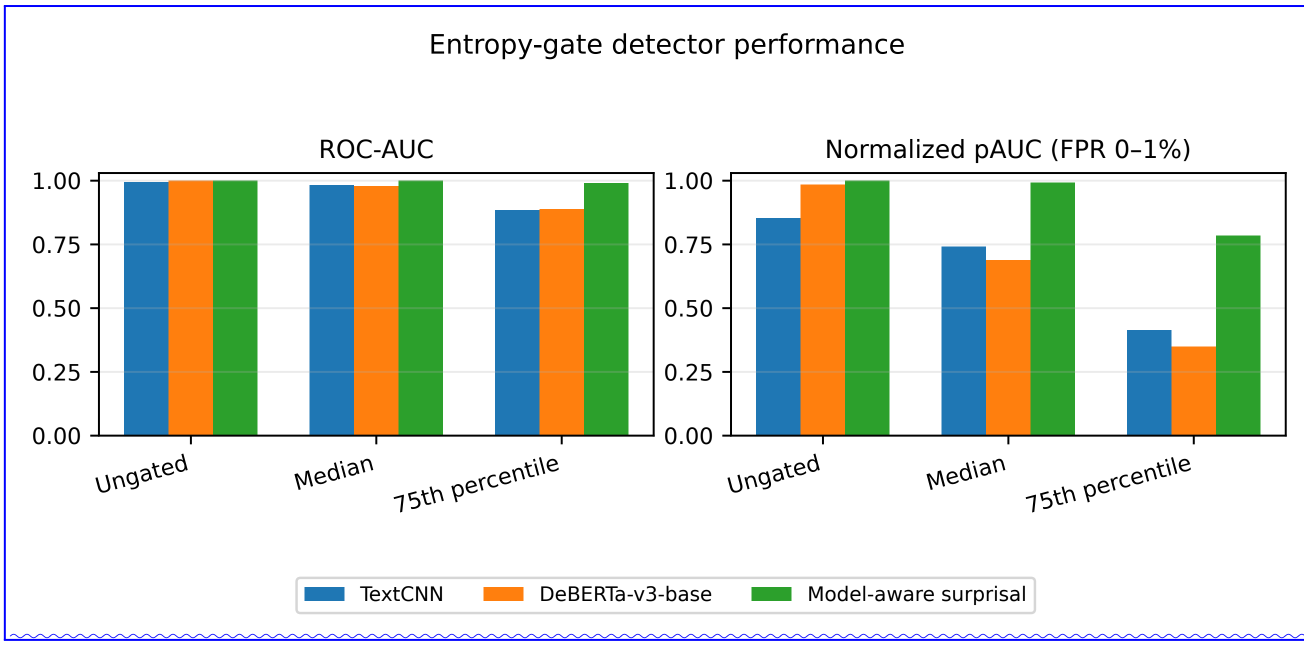


Figure S11. Entropy-gate detector performance. Stricter gating reduces text-only detectability in this bounded matrix but does not eliminate the strong exact-model-aware signal.

Taken together, stricter gating reduces text-only detection in the tested conditions while reducing capacity, substantially increasing length, producing six maximum-budget failures, and retaining exact-environment replay dependence. Model-aware ROC-AUC of 0.9904 under the strict gate is inconsistent with a claim of an undetectable channel.

Supplementary Note S14: Matched Q4 and Q8 quantization analysis

Pairing and recovery

The quantization analysis holds Qwen2.5-7B-Instruct revision `8911e8a47f92bac19d6f5c64a2e2095bd2f7d031`, its embedded tokenizer, rendered prompt, payload, codec, safe-text filter, target length, temperature, top-p, llama-cpp-python 0.3.23 backend, and historical ordinary-control seed fixed. It successfully model-replayed all 1,920 historical Q4_K_M rows and completed all 1,920 paired Q8_0 generations. No additional Q4 covers were generated.

Visible-text recovery for the 960 historical Q4 RankCloak covers retokenized each recorded string under the exact embedded Q4 tokenizer. The 732 exact token-ID matches reused the validated saved-ID rank trace; the other 228 cases underwent rank replay through the pinned Q4 model with the saved embedding-span boundary. This remains replay-assisted and does not recover without the exact context and span contract. Q8 visible-text recovery used the corresponding exact model path. Pairing used frozen payload, representation, and prompt identities.

Recovery item	Q4_K_M	Q8_0	Paired interpretation
Saved-ID exact recovery	960/960	960/960	Perfect only under exact saved-ID replay
Visible-text exact recovery	732/960 (0.7625)	758/960 (0.7896)	Difference 0.0271; 95% CI [-0.0073, 0.0635]
Visible recovery 95% CI	[0.7344, 0.7896]	[0.7635, 0.8167]	Payload-group bootstrap, 2,000 resamples
Discordant outcomes	Q4 only 144	Q8 only 170	Exact two-sided McNemar $p = 0.1582$
Concordant outcomes	Both 588	Neither 58	No statistically supported Q8 advantage

Table S25. Paired Q4 and Q8 recovery. Rates use 960 frozen RankCloak pairs. The exact McNemar result is computed from the stored 144/170 discordant counts.

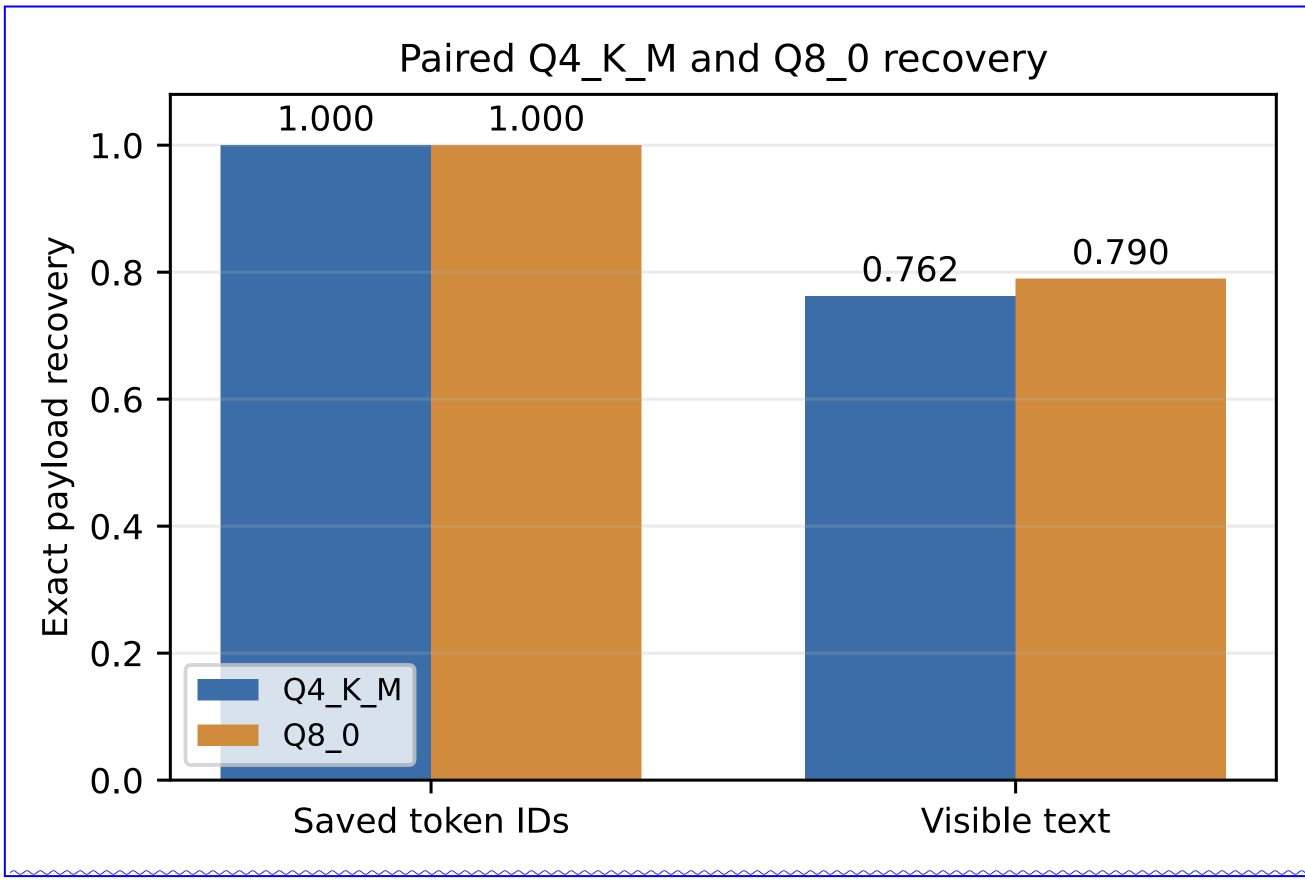


Figure S12. Paired saved-ID and visible-text recovery by quantization. The grouped interval for the paired difference includes zero.

On the identical historical Q4 RankCloak token path, Q8 minus Q4 mean next-token entropy was −0.0154 bits (95% CI [-0.0187, -0.0120]); the observed-token rank changed at a mean fraction 0.4639 of positions and the greedy token changed at 0.0884. Independently generated Q4/Q8 RankCloak paths never matched exactly and had mean positionwise token match 0.0155. These differences show substantial path sensitivity; they are not evidence that either quantization is generally more recoverable.

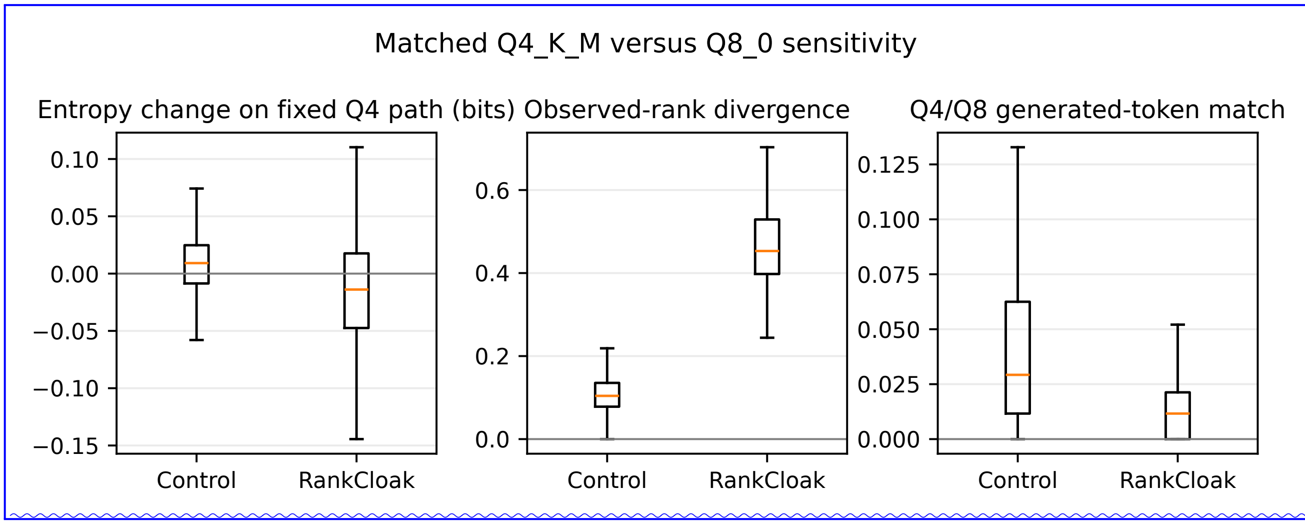


Figure S13. **Matched quantization sensitivity.** Same-path likelihood/rank comparisons and independently generated token-path agreement are paired within one pinned Qwen revision.

Cross-quantization detector transfer

Quantization detectors used the frozen payload train/validation/test assignment. Q4-to-Q8 fit and selected thresholds on Q4 before testing Q8; Q8-to-Q4 reversed the direction; the pooled fit used both quantizations in training and validation and tested only held payloads. Exact and declared near-duplicate audits preceded feature extraction and retained every row: 1,920/1,920 in each directional corpus and 3,840/3,840 pooled, with zero audited leakage.

Detector	Transfer	ROC-AUC	0–1% pAUC	TPR/FPR at frozen 1% threshold	Exact FP
TextCNN	Q4 to Q8	0.9927	0.7539	0.9375/0.0365	7/192
DeBERTa	Q4 to Q8	1.0000	1.0000	1.0000/0.0000	0/192
Model-aware	Q4 to Q8	1.0000	1.0000	1.0000/0.0000	0/192
TextCNN	Q8 to Q4	0.9944	0.8298	0.9479/0.0260	5/192
DeBERTa	Q8 to Q4	1.0000	1.0000	1.0000/0.0156	3/192
Model-aware	Q8 to Q4	1.0000	1.0000	0.9792/0.0000	0/192
TextCNN	Pooled	0.9965	0.8257	0.9609/0.0234	9/384
DeBERTa	Pooled	1.0000	1.0000	1.0000/0.0078	3/384
Model-aware	Pooled	1.0000	1.0000	0.9948/0.0000	0/384

Table S26. **Cross-quantization and pooled detector behavior.** Frozen nominal thresholds can drift in test. No test has 1,000 negatives, so empirical 0.1% results are unavailable.

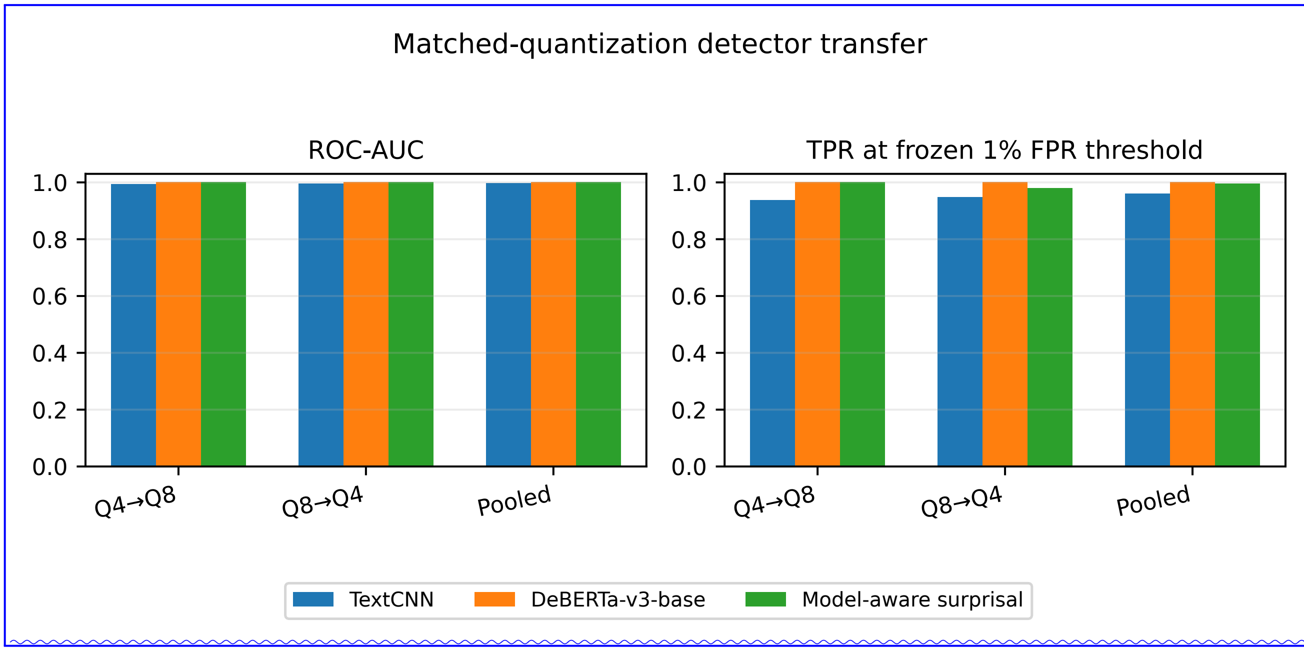


Figure S14. **Detector transfer across Q4_K_M and Q8_0.** Detection remains strong in both directions, while observed FPR can exceed the validation target.

The analysis isolates two quantizations of one model revision under one inference backend. It does not generalize to other Qwen revisions, architectures, bit widths, quantizers, backends, or hardware. In particular, the data do not justify stating that Q8 is generally more recoverable than Q4.

Supplementary Note S15: V3 provenance and remaining boundaries

The V3 run manifest binds the detector inputs, model-backed generations, artifact hashes, backend, model and tokenizer identities, partitions, and validation status. The final validation report records `status:pass`, exact artifact-manifest path and checksum validation for 5,833 entries, 24 fit-metadata files, 24 prediction files, strict leakage-audit success, conditional entropy-capacity validation, and paired quantization-recovery source validation. The generation-analysis validation records 18/18 calibration traces, 720/720 entropy generations, 1,920/1,920 Q4 replays, 960/960 Q4 visible-recovery records, 1,920/1,920 Q8 generations, and zero failure records. Machine-readable tables and full provenance are retained under `results/revision_v3`; the manuscript reports from those sealed outputs rather than rerunning generation.

Table S27. Remaining boundaries after the targeted V3 evaluations.

Issue	Added evidence	Boundary that remains
Saved-token condition	Perfect primary and paired-quantization recovery from the preserved token stream under recorded configurations	Tests token-preserving delivery or experimental retained IDs under the predeclared protocol; distinct from ordinary visible-text communication
Visible-text transport	88/144 historical robustness recovery; 732/960 Q4 and 758/960 Q8 in one paired matrix	Paraphrase, markdown/format transfer, cross-model decoding, and retokenization remain fragile or unsuccessful
Strict deduplication	Component-safe split with zero declared cross-partition links	Does not detect semantic paraphrases below the 0.95 character-n-gram threshold

Issue	Added evidence	Boundary that remains
Human-authored controls	250 frozen Dolly negatives across 16 templates	Not a participant study; sparse genre coverage and generic machine/human signal remain
Unseen families	Three leave-one-family-out tests	Not every architecture, training corpus, model size, proprietary system, or decoding stack
Adaptive detection	Two data-adaptive text-only detectors and an exact-model-aware likelihood attacker	Strong but not exhaustive attackers or deployment distributions
Low-FPR reporting	Exact counts and validation-frozen thresholds; matched 0.1% estimates where supported	Human, family, entropy, and quantization tests have fewer than 1,000 negatives and no empirical 0.1% resolution
Entropy gating	120 attempts per gate; capacity, length, recovery, quality, and detector outcomes	Two templates; thresholds from 768 positions/model; strict capacity conditional on 114 completions; exact-model AUC remains 0.9904
Quantization	Paired Q4_K_M/Q8_0 recovery and bidirectional detector transfer	One pinned Qwen revision, two quantizations, one backend; no supported general Q8 advantage
Naturalness	Automated surface/model diagnostics and a human-authored control corpus	No participants or human ratings; controls are not evidence of human-perceived naturalness
Deployment/security	Failure surfaces and strong adverse detectability are documented	No real-world deployment, robust channel, secrecy proof, cryptographic protection, or undetectability

385 The bounded evaluations directly address the requested controls, leakage prevention, held-family transfer, adaptive and model-aware attackers, low-FPR reporting, entropy gating, and quantization sensitivity. They refine the evidence for an artifact-specific hiding and recovery method: the artifact surface form is concealed, exact recovery is configuration-dependent, visible-text transport is weaker, and current steganalysis often detects the encoding.

References

1. Cai, S., Ding, L. & Tao, D. Entropy-guided watermarking for LLMs: A test-time framework for robust and traceable text generation. *arXiv preprint arXiv:2504.12108* DOI: 10.48550/arXiv.2504.12108 (2025).